
Machine Learning Methods for Hit-Level Electron Shower Reconstruction under Pile-Up Conditions for the Light Dark Matter eXperiment

Eliot M. Petré

Lund University – Faculty of Engineering (LTH)

`eliot@mpetren.se`

Supervisor: Lene Kristian Bryngemark

Co-supervisor: Ruth Pöttgen

Master's Thesis, 30 ECTS credits

Master of Science in Engineering: Engineering Physics

Abstract

Baseline: Supervised hit-to-particle clustering

"MLPF-lite": Multi-task hit-level shower reconstruction

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetur id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

11th August 2026

Acknowledgements

My sincerest gratitude goes to my supervisor Lene Kristian Bryngemark and my co-supervisor Ruth Pöttgen...

I am also grateful to everyone who has contributed with discussions, advice, and encouragement during the course of this work, including the other diploma students at LDMX during my time there.

I want to direct my thanks to my partner Tilde Bonnevier Wallstedt for her support and source of inspiration to write a thesis in particle physics. My friend Axel Helgstrand also deserves thanks for the same reason and for directing me to LDMX especially.

I would also like to thank my two cats Shiro and Nemo, who assisted my work by performing random walk on my keyboard, thus expanding the search in the space of possible solutions.

Contents

Acknowledgements

1	Introduction	2
2	Background	3
2.1	Dark Matter Particle Physics	3
2.1.1	A Brief History of Dark Matter: from Astronomy to Particle Physics	3
2.1.2	Thermal Relics and the Sub-GeV Frontier	4
2.2	The Light Dark Matter eXperiment	6
2.2.1	Search Strategy and Physics Goal	6
2.2.2	Detector Layout	8
2.2.3	Electromagnetic Calorimeter and Electron Showers	11
2.2.4	Trigger Scintillator and Trigger-Pad Tracks	12
2.2.5	Triggering	13
2.2.6	Pile-Up, Reconstruction and Physics Reach	14
2.3	Machine Learning Methods for Reconstruction in High Energy Physics	15
2.3.1	Computational Context in Experimental High Energy Physics	17
2.3.2	MLPF, a Recent Example of ML-based Reconstruction in HEP	17
2.3.3	The GNN and the Transformer under Pile-Up Conditions	18
2.3.4	Graph Neural Networks and GravNet	19
2.3.5	Transformer Encoders and Self-Attention	20
2.3.6	An analogy between the Transformer and the GNN	21
2.4	things that I want to include but have maybe not found a suitable place for yet	22
2.4.1	ML-based Reconstruction in CMS	23
3	Methodology	23
3.1	Data Generation Overview and Reconstruction Formulation	24
3.1.1	Simulation and Data Pipeline Overview	24
3.1.2	Reconstruction Formulation	25
3.2	Simulation	26
3.2.1	Single Electron ($1e$) Event Simulations	26
3.2.2	Simulating Events with Electron Multiplicity ($2e$, $3e$)	26
3.3	Simulation Information and ROOT Extraction	28
3.3.1	Selected Detector Data and Simulation Truth	29
3.3.2	Chunked ROOT Reading and Hit Matching	30
3.4	Supervised Targets and Simplifications for the Baseline Models	31

	3.4.1	Dominant-Origin and Energy-Fraction Targets	32
	3.4.2	Noise Handling and Electron Ordering	32
3.5		Tensorization and Arriving at ML-Ready Storage	33
	3.5.1	Combined Event Tensor	33
	3.5.2	Preprocessing of Energy Input Features	34
	3.5.3	ML-Ready Shards and Stored Event Structure	34
	3.5.4	Data Split and Training-Time Preparations	36
3.6		Model Architectures	36
	3.6.1	Capacity-Selection Strategy	36
	3.6.2	GravNet Baseline	37
	3.6.3	Transformer Baseline	38
	3.6.4	MLPF-Inspired Contributor-Set Slot Model	38
3.7		Training Procedure	39
	3.7.1	Dataset splitting	39
	3.7.2	Mini-batch construction for variable-sized events . . .	39
	3.7.3	Optimizer and Regularization	41
	3.7.4	Early Stopping and Checkpoint Selection	41
3.8		Training Objective and Optimization	41
3.9		Evaluation Methodology	42
	3.9.1	Hit-origin classification metrics	42
	3.9.2	Projected shower geometry	43
	3.9.3	Difficulty profiles and event inspection	44
	3.9.4	Interactive 3D <i>electromagnetic calorimeter</i> (ECal) Event Display	45
4		Results and Discussion	45
4.1		Event Characteristics and the Shower-Overlap Problem	46
	4.1.1	Detector Occupancy and Longitudinal Shower Devel- opment	46
	4.1.2	Truth-Level Transverse Shower Geometry	48
	4.1.3	Trigger-Pad Multiplicity as Event-Level Context . . .	50
	4.1.4	Permutation Invariance of Electron Shower Labels . .	51
4.2		Baseline Models	53
	4.2.1	Baseline Architecture Comparison	53
4.3		Baseline Models	54
	4.3.1	Baseline Architecture Comparison	54
	4.3.2	Representative Training Behaviour	57
	4.3.3	Hit Assignment Performance	58
	4.3.4	Performance Across the Electron Shower Depth	61
	4.3.5	Dependence on Shower Separation	63
	4.3.6	Confidence and Selective Performance	66
	4.3.7	Trigger-Pad Ablation	69
4.4		MLPF-Inspired Slot Model	70
	4.4.1	MLPF-Inspired Contributor-Set Reconstruction	70

4.4.2	Electron Counting and the Contribution of Trigger-Pad Information	73
5	Conclusions and Outlook	74
5.1	Summary of Contributions	74
5.2	Future Work	74
6	References	76
	Abbreviations and Glossary	80
	Glossary	80
	Acronyms	80
7	Appendix	83
7.1	WILL REMOVE THIS: LDMX-SW pull request #2032 - <i>Fix origin ID handling in OverlayProducer</i>	83
7.1.1	Changes in <code>OverlayProducer.cxx</code>	83
7.1.2	Changes in <code>SimCalorimeterHit.cxx</code>	83
7.2	ROOT reading and tensorization logic	84
7.3	Interactive 3D ECal Event Display	86

Notes and stuff

Reference test

Here is my repo. [1]

I want to (maybe) phase in eventually

Chapter A from partienet paper [2] argues efficiently why GNNs are likely preferred in this application as compared to CNNs.

This paper summarizes the problem that the project aims to work with pretty well MLPF. The prospect of a ML based model may improve the "physics reach" of X experiment. This is in essence the PileUp problem and the prospect of a GNN based method! It is also a strong counter-argument to doubters who are deterred by the idea of ML in physics. MLPF github. A continuation of MLPF using transformers. Another related paper by the same authors.

1 Introduction

Dark matter (DM) is an invisible form of matter whose subatomic nature remains unknown, motivating searches for physics beyond the *Standard Model* (SM). The *Light Dark Matter eXperiment* (LDMX) searches for sub-GeV dark matter by comparing the momentum of an incoming electron with the visible final state after a fixed-target interaction. Missing momentum could indicate the production of invisible dark matter particles, making accurate reconstruction of the beam electrons and their detector activity essential [3–5].

In experimental particle physics, *reconstruction* uses detector measurements to infer physically meaningful properties of the underlying particle event. At increased beam intensity, several electrons can enter LDMX within the same readout window and produce overlapping showers in the ECal subdetector. This obscures which detector activity originated from each electron. Rejecting every such event would discard part of the delivered beam, whereas reliable reconstruction could make a larger fraction of recorded events useful and thereby improve the practical physics reach [5].

This thesis investigates supervised *machine learning* (ML) methods for separating overlapping showers in simulated two-electron and three-electron events. The principal hit-level task assigns each selected ECal hit to the electron responsible for the largest share of its deposited energy, as determined from simulation truth. An event is represented as a variable-size collection of ECal hits and supplemented with reconstructed trigger-pad-track observations from the *Trigger Scintillator* (TS) subdetector. Information from both subdetectors can therefore be processed jointly through representations learned by the models, without projecting the event onto a fixed image.

Two controlled baseline models are studied: a *graph neural network* (GNN) using GravNet convolutions and a Transformer encoder. Both receive ECal hits and reconstructed trigger-pad-track observations, with the value of the latter evaluated through inference-time ablation. Both architectures learn relations within unordered, irregular detector data. Their use is motivated by recent machine-learned particle-flow methods that interpret heterogeneous detector information with graph-based or Transformer-based models [6–8]. A secondary exploratory multitask Transformer also predicts energy fractions and electron count alongside hit assignment, following an approach inspired by particle-flow reconstruction.

The study asks how reliably the models can assign hits in two-electron and three-electron pile-up events, which shower properties limit the separation, whether TS inputs add useful context beyond the ECal, and how GravNet and Transformer encoders compare in reconstruction quality and computational properties.

The scope is limited to offline reconstruction of simulated inclusive non-signal events. It does not perform a DM search or deploy a real-time trigger algorithm. This thesis presents the reusable `ml_ldmx` platform developed for the study, which prepares LDMX simulation data, trains models and produces hit-level and event-level diagnostics [9].

2 Background

2.1 Dark Matter Particle Physics

Dark matter (DM) is the name given to a proposed form of invisible matter that does not emit, absorb or scatter light. Astronomical and cosmological measurements reveal gravitational effects that, within general relativity, cannot be explained by the observed distribution of ordinary matter alone. Dark matter is the leading explanation for these observations, but their cause has not been directly established. In particular, no DM particle or non-gravitational interaction with the particles of the *Standard Model* (SM) has been confirmed in an experiment. This gap between well-established astronomical observations and an unknown subatomic explanation makes dark matter one of the largest unsolved problems in modern physics [3, 10].

This subsection briefly traces the introduction of DM into particle physics before presenting thermal freeze-out, light dark matter and the dark bremsstrahlung process that is the basis for the *Light Dark Matter eXperiment* (LDMX).

2.1.1 A Brief History of Dark Matter: from Astronomy to Particle Physics

At the frontier of observation, a new tool can change what is visible. When Galileo Galilei pointed his telescope at the sky in the 17th century, he resolved the faint glow of the Milky Way into stars and observed four satellites in orbit around Jupiter, all previously invisible to the naked eye. These were not discoveries of DM, but they illustrate two important ideas: nature may contain matter beyond the reach of ordinary perception, and a new instrument can move that reach [10]. Just as the telescope did not create Galileo’s moons, a reconstruction algorithm does not create new detector information either, it attempts to make better use of information that is already there.

DM as a concept encapsulates this idea of matter that exists, but cannot be perceived by ordinary means. The history of the physical phenomenon now called dark matter, however, begins with measurements that revealed more gravitating mass than astronomers could see. Astronomers could estimate the visible mass of a galaxy or galaxy cluster from its light and independently estimate the total mass required to explain how its stars or galaxies moved. Several studies independently found that the visible mass was not enough. These studies included the recently rediscovered work by Lund University astronomer Knut Lundmark, whose 1930 comparison of the luminosities and mass estimates of five galaxies preceded Fritz Zwicky’s more famous 1933 study of the Coma cluster. Zwicky borrowed the virial theorem from thermodynamics, modeling celestial bodies as particles, to show that the galaxies in Coma moved too rapidly for the cluster to be held together by its visible mass alone. Later measurements of galactic rotation curves showed a related discrepancy: stars far from galactic centres moved faster than expected from the visible matter [10–12]. For these astronomers, however, “dark matter” generally meant unseen astronomical material such as faint stars, cold gas or other non-luminous bodies, not necessarily a new elementary particle.

The transition from unseen astronomical matter to particle physics was neither immediate nor automatic. It required both new evidence and a change in scientific culture. Half a century ago, cosmology was still regarded by many physicists as a "fringe science" with limited predictive power and testability, and the two communities rarely contributed to each other's fields [10].

As cosmology became a recognized quantitative science, measurements of primordial element abundances and the cosmic microwave background showed that baryonic matter could not provide all the required mass. Within the standard cosmological model, baryonic matter, meaning ordinary matter composed primarily of protons and neutrons, accounts for only about 16% of the total matter in the Universe. The remaining 84% is attributed to dark matter [3]. Studies of structure formation further indicate that the dominant component must have moved slowly enough to behave as cold matter. These results helped transform dark matter from a problem of unseen astronomical objects into a problem that could require new particle physics.

Modified-gravity explanations, which attempt to reproduce these observations without introducing a new non-baryonic matter component, have also been explored, but none has accounted for the full range of observations as successfully as the dark matter framework [3, 10].

The SM describes the known elementary particles and their strong, weak and electromagnetic interactions, while gravity is disconnected described separately by general relativity. No known SM particle simultaneously has the properties required to account for the observed cold dark matter. A particle interpretation therefore points beyond the SM, to possibly a DM sector. A viable DM particle candidate must have three key properties:

- 1) **Mass.** A DM particle candidate must have mass higher than zero, accounting for the invisible mass in the Universe, and it explains its gravitational influence.
- 2) **Stability.** Observations in the cosmic microwave background imply the existence of DM in the early Universe and it remains abundant today. It must therefore be stable, or at least have a lifetime long enough to survive until the present day.
- 3) **Almost no interaction with the SM.** Gravity is described separately from the SM, but it is the only established interaction between DM and ordinary matter. *If* DM has any additional interactions with SM particles, they must be sufficiently weak to have escaped detection in previous experiments. In particular, any electromagnetic interaction must be extremely weak.

2.1.2 Thermal Relics and the Sub-GeV Frontier

Possible dark matter candidates span an enormous range of masses. Figure 1 gives a schematic overview of several frequently discussed regions. In this thesis, *light dark matter* (LDM) refers to particle dark matter below approximately the GeV mass scale. The range of greatest relevance to LDMX is roughly from MeV to GeV masses. [5, 13].

The *thermal freeze-out* hypothesis is one possible explanation for the amount of dark matter observed today. In this model one assumes that there exists an interaction that

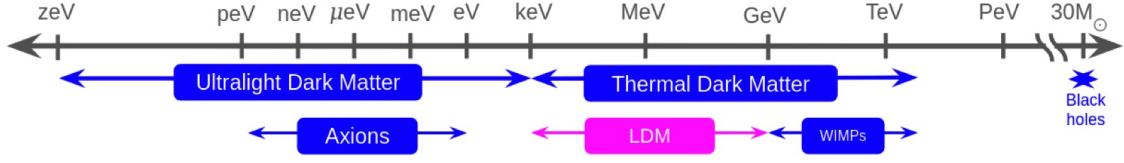


Figure 1. Representative mass ranges for several DM candidates and frameworks. The LDM region relevant to LDMX is highlighted in magenta, while WIMP benchmarks occupy a higher-mass part of the broader thermal dark matter region. The boundaries are illustrative and model-dependent. Reproduced from Figure 4 of Ref. [14], which was inspired by Figure 1 of Ref. [13].

connects the SM and the dark sector. Given this assumption, in the hot and dense early Universe, particle collisions were frequent and energetic enough to create dark matter particles. Dark matter particles could also annihilate back into SM particles. These reactions occurred rapidly enough to keep the two sectors in *thermal equilibrium*.

As the Universe expanded, it became cooler and less dense. Creating massive dark matter particles became increasingly difficult, while existing dark matter particles continued to annihilate. Eventually, the particles became so dilute that the annihilation rate fell below the expansion rate of the Universe. Annihilation then became rare, and the number of dark matter particles in a volume expanding with the Universe remained approximately constant. Their abundance had *frozen out*, and the surviving population is called a *thermal relic*.

The surviving thermal relic depends on the strength of the interaction. A stronger annihilation process remains effective for longer and leaves fewer dark matter particles, while a weaker process causes an earlier freeze-out and leaves more particles. For a specified particle model and mass, the observed dark matter abundance therefore defines a target interaction strength [3]. This is referred to as the *thermal target*. Thermal freeze-out does not define one universal dark matter mass range because the result also depends on the particle model, its interactions and the history of the early Universe.

The historically best-known thermal relic candidate is the *weakly interacting massive particle* (WIMP). A particle with a mass near the electroweak scale with a weak-strength interaction can freeze out with approximately the observed abundance, a coincidence known as the *WIMP miracle*. Searches have strongly constrained many WIMP models without finding a confirmed signal, although WIMPs as a class have not been excluded. Experimental limits depend on both particle mass and interaction strength. Below the GeV scale, the nuclear recoil produced by a dark matter particle becomes difficult for traditional WIMP searches to detect, motivating alternative searches for LDM [3, 5, 15].

One way to connect LDM to the SM introduces a new dark-sector gauge-boson mediator called the *dark photon*, denoted by A' in Figure 2. It can be thought of as a dark-sector relative of the ordinary photon. A small kinetic mixing with the ordinary photon allows the dark photon to interact weakly with electrically charged SM particles. In the invisible dark photon benchmark considered here, the dark photon decays into a pair of dark matter particles, denoted by $\chi\bar{\chi}$ [3, 5].

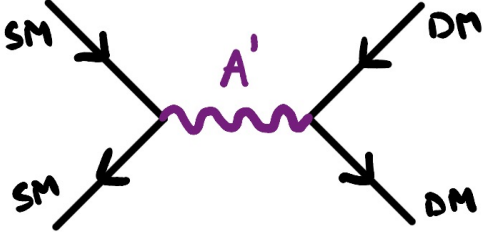


Figure 2. Feynman diagram of conversion of a pair of SM particles into a pair of DM particles, mediated by a dark photon A' .

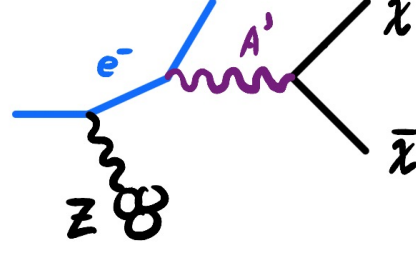


Figure 3. Feynman diagram-sketch of dark bremsstrahlung. The electron scatters and radiates a dark photon A' , which decays into an invisible dark matter particle-antiparticle pair $\chi\bar{\chi}$. The recoiling electron provides a visible signature by the law of conservation of momentum. Inspired by Ref. [16].

In an electron fixed-target experiment, the benchmark production reaction is *dark bremsstrahlung*, illustrated in Figure 3. It is a dark-sector version of ordinary bremsstrahlung. An incoming electron scatters in the field of a target nucleus and radiates a dark photon, which then decays into $\chi\bar{\chi}$. The dark matter particles leave no measurable detector signal. Their production can instead be inferred from the imbalance between the incoming electron momentum and the reconstructed visible final state. Unexplored thermal targets motivate the missing-momentum search for which LDMX is designed [4, 5].

Dark bremsstrahlung turns the original astronomical question into a tangible experimental one: can missing momentum reveal an invisible final state after ordinary sources of imbalance have been rejected? The next subsection introduces the detector built to answer that question.

2.2 The Light Dark Matter eXperiment

2.2.1 Search Strategy and Physics Goal

LDMX is an electron fixed-target *missing momentum* experiment designed to search for sub-GeV DM through the dark-bremsstrahlung process introduced above. The experiment is planned for End Station A at *SLAC National Accelerator Laboratory* (SLAC) outside of San Francisco, California and is developed by an international collaboration with institutions in the United States and Sweden. An 8 GeV electron beam will strike a thin tungsten target. Tungsten is used because its high atomic number gives a large production rate for low-mass dark photons and helps reduce nuclear backgrounds at the chosen target thickness [5].

In the dark bremsstrahlung benchmark, the beam electron radiates a dark photon, denoted A' . The mediator couples weakly to electrically charged SM particles and can decay into a pair of dark-sector particles, denoted by χ . It therefore provides a possible bridge between the SM and a dark sector. The DM particles do not produce a measurable

detector signal. LDMX instead tests the model by searching for the recoil of the electron and the missing momentum left by their production [4, 5].

In the benchmark process, a candidate event has three main features. The electron leaves the target with less energy than it entered with, it receives a transverse (perpendicular to the beam direction) momentum kick, and no other visible final-state particle accounts for the lost momentum. The combination is important because ordinary SM interactions can mimic parts of this signature when a visible particle escapes detection. The trackers therefore measure the incoming and recoil-electron momenta, while the calorimeters test whether additional visible activity explains the apparent missing momentum [4, 5].

The planned beam structure delivers approximately 20 million filled buckets per second, with an average of one to two electrons in each bucket. This corresponds to roughly 20 to 40 million electrons reaching the target per second during operation. The exposure, measured in *electrons on target* (EoT), is not a universal event count required to reject a null hypothesis. Rather, increasing the exposure increases sensitivity to rarer production processes and weaker interactions. The attainable exposure also depends on running time, detector performance, backgrounds and radiation tolerance. LDMX is designed for a staged programme beginning around 4×10^{14} EoT and extending towards exposures of order 10^{16} EoT, with projected sensitivity up to three orders of magnitude beyond earlier searches [5].

Useful exposure depends not only on how many electrons are delivered, but also on which events can be triggered, stored and reconstructed. The ability to retain events containing several electrons can therefore improve the practical physics reach. This connection is developed further in Section 2.2.6.

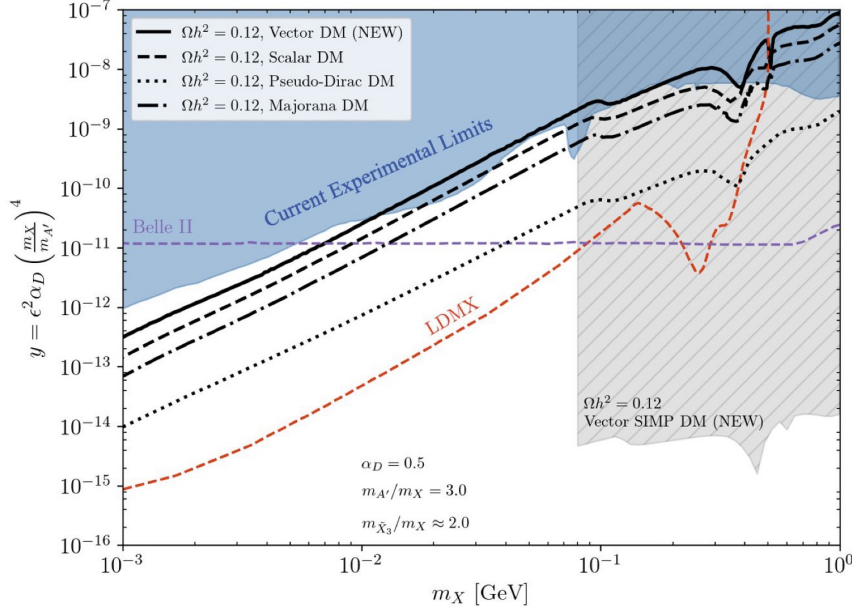


Figure 4. Representative comparison of thermal-relic targets with existing constraints and projected experimental sensitivity. The horizontal axis gives the dark-matter mass and the vertical axis gives the benchmark interaction quantity y . Blue shows approximate existing constraints, purple shows the Belle II projection, red shows the LDMX projection, and the black curves and grey region show several model-dependent DM thermal targets. Reproduced from Refs. [5, 17].

Figure 4 gives a representative view of the projected reach. The horizontal axis shows the dark-matter mass, while the vertical axis combines the assumed interaction strength and particle masses into the benchmark quantity y . The blue region represents parameter combinations already constrained by earlier searches. The red curves show the smaller interaction strengths that LDMX could test, thus expanding the frontier of search of DM, while the black curves and grey region show thermal-relic targets for several representative DM models. The figure is therefore a comparison within specified model assumptions, not a model-independent exclusion of dark matter by mass alone [5, 17, 18].

2.2.2 Detector Layout

Figure 5 shows the physical scale and overall arrangement of the detector. The conceptual drawing in Figure 6 illustrates the path of the beam electron and the missing-momentum signature. The labelled cutaway in Figure 7 shows the detector geometry along the beam direction.

Two *Trigger Scintillator* (TS) modules are placed upstream of the silicon tagging tracker, and a third is placed directly before the target. The tagging tracker measures the incoming electron momentum. The electron then passes through the thin tungsten target inside a dipole magnet. Its magnetic field bends charged particles transversely according to their momentum, so a recoil electron that has lost energy is deflected more strongly than an electron retaining the full beam energy. A silicon recoil tracker

downstream of the target measures the momentum and direction of the outgoing charged particles.

The baseline target is 0.1 radiation lengths thick and has a transverse size of $4\text{ cm} \times 10\text{ cm}$. Its small thickness limits multiple scattering while retaining a useful production rate. The nominal $20\text{ mm} \times 80\text{ mm}$ beam spot reduces peak detector occupancy and helps to separate the showers from different electrons [5].

The *electromagnetic calorimeter* (ECal) is placed behind the recoil tracker. It measures electrons, photons and their shower shapes. The HCal surrounds the ECal at the sides and rear. Its main role in the dark-matter search is to identify penetrating hadrons and muons that could otherwise imitate missing momentum. Together, the trackers and calorimeters measure the recoil electron and *veto*, meaning reject, candidate events containing other visible particles. This combined information distinguishes an invisible final state from ordinary interactions in which visible particles were missed by one detector system [5].

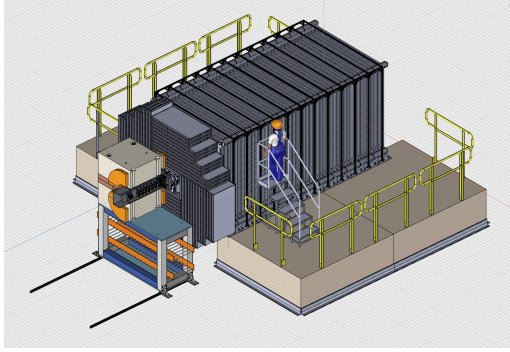


Figure 5. Solid-model overview of the LDMX detector installation. The compact tracking and target region is located at the left, in front of the calorimeter systems. Sourced from Figure 3.1 of the 2025 LDMX design report [5].

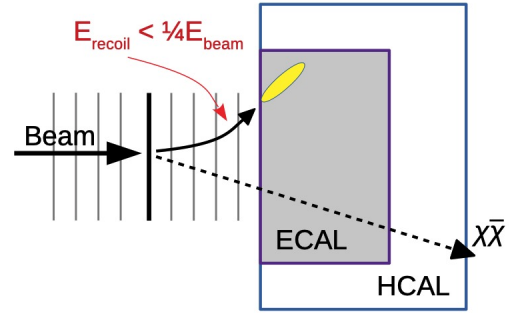


Figure 6. Conceptual illustration of the LDMX missing-momentum signature. After the beam electron passes through the tagging tracker and scatters in the thin tungsten target, a recoil electron carrying less than one quarter of the beam energy enters the ECal, while an invisible $\chi\bar{\chi}$ pair carries away momentum. The ECal and HCal test for additional visible activity. Sourced from Figure 8 of the 2018 LDMX design report [4].

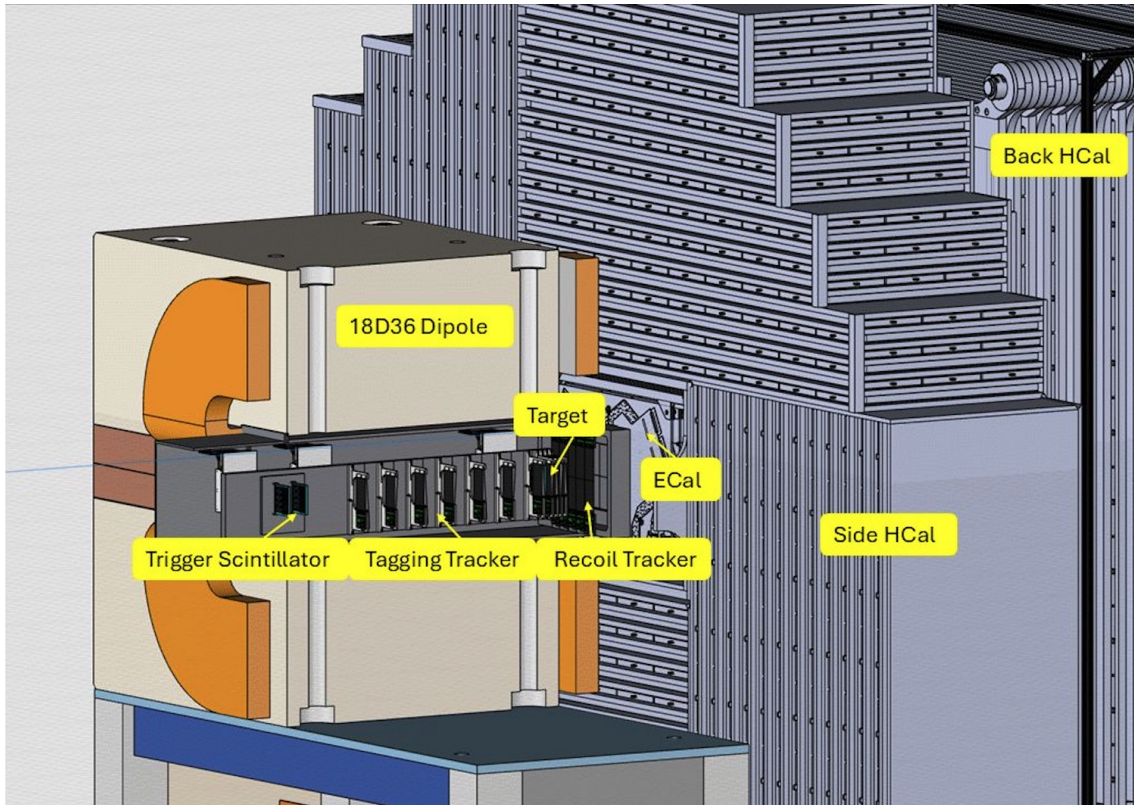


Figure 7. Cutaway view of the LDMX detector. From left to right, it shows the Trigger Scintillator, tagging tracker, target inside the dipole magnet, recoil tracker, ECal, and the side and back HCal. Sourced from Figure 3.2 of the 2025 LDMX design report [5].

2.2.3 Electromagnetic Calorimeter and Electron Showers

The main purpose of the ECal is to measure the energy and spatial development of electrons and photons, allowing the visible final state to be reconstructed and backgrounds to be rejected. Its high-granularity silicon-tungsten design is a compact "sibling detector" of the *High Granularity Calorimeter* (HGCAL) at the *Compact Muon Solenoid* (CMS) experiment, from which LDMX adopts detector-module and readout technology [5].

The ECal is a sampling calorimeter with a total depth of about 40 radiation lengths. A radiation length X_0 is the characteristic material scale for energy loss by a high-energy electron through bremsstrahlung. Tungsten layers initiate and develop electromagnetic showers, while interleaved silicon layers sample their energy deposits. The final design contains 32 silicon layers arranged as 16 bilayers, where each bilayer pairs two silicon sampling planes around a common support and cooling structure. The `ldmx-det-v15-8gev` geometry used in this thesis likewise produces reconstructed ECal hits at 32 layer positions. The performance simulations in the current (and newer) LDMX design report instead used an earlier geometry with 34 layers in 17 bilayers [5].

An energetic electron radiates photons through *bremsstrahlung* as it passes through tungsten. These photons can produce electron-positron pairs, which radiate further photons. Repetition of these processes creates an *electron shower*. One incoming electron therefore produces energy deposits in many cells and layers rather than a single detector hit. The energy contributions in the 3D pattern captures information about the electron position, energy and shower development [19].

The transverse size of an electromagnetic shower is commonly described by the Molière radius R_M . For a homogeneous material, it is calculated as

$$R_M = X_0 \frac{E_s}{E_c}, \quad E_s \simeq 21 \text{ MeV}, \quad (1)$$

where X_0 is the radiation length, E_s is a material-independent scale energy, and E_c is the critical energy at which radiative and ionisation energy losses are comparable. On average, about 90% of the shower energy is contained within a cylinder of radius R_M [19]. For the layered LDMX ECal, the design report gives an effective Molière radius of approximately 2.5 cm. It also reports that 68% of the shower energy is contained within less than 1 cm during approximately the first 15 layers [5]. These values provide useful reference scales for interpreting shower overlap.

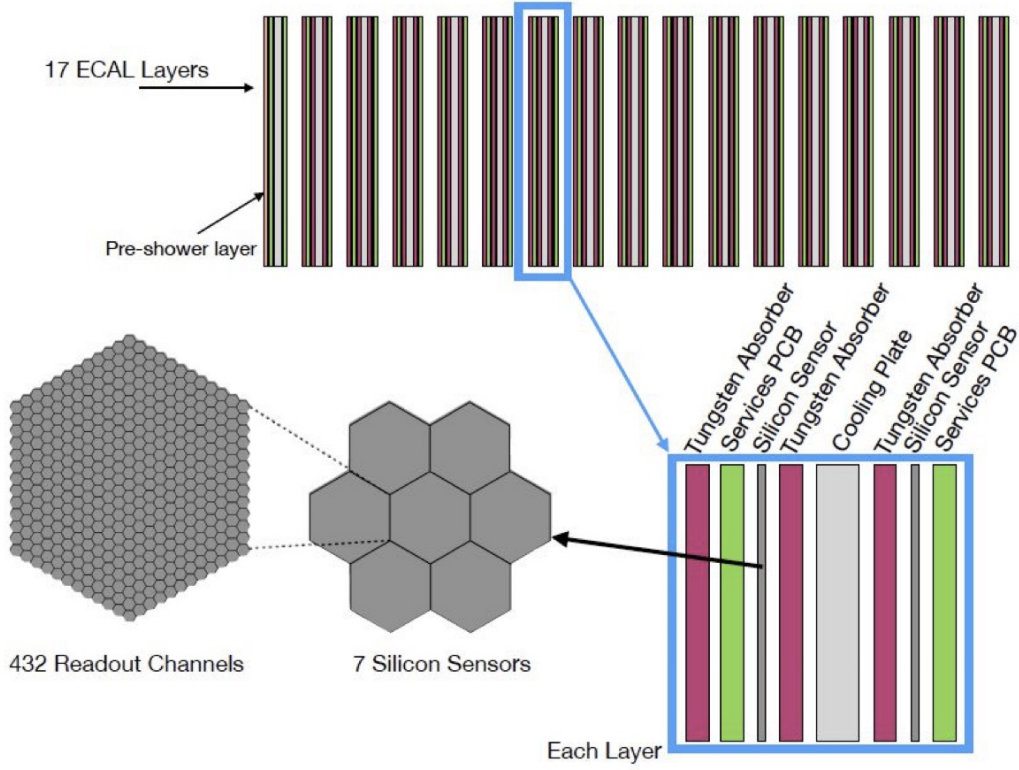


Figure 8. Schematic of the full-resolution LDMX ECal modules and the components of a bilayer. Sourced from the LDMX design report [5]

Each ECal sampling plane contains seven hexagonal modules, and each module contains 432 silicon readout cells. A local cell identifier connects every measured energy deposit with a fixed detector position. Figure 8 shows this full-resolution segmentation and the bilayer construction. For the real-time trigger, groups of nine neighbouring readout cells are summed into 48 coarser *trigger cells* per module. This grouping, which is not drawn in the figure, retains less spatial detail for the *online reconstruction* compared to the individual-cell readout available to *offline reconstruction* [5].

The fine segmentation helps separate nearby showers and identify small energy deposits from background particles, making it central to this thesis. When several beam electrons arrive in the same readout window, their showers may overlap in the same layers or cells. Reconstruction must then use the spatial and energy pattern of the full-resolution hits to determine which electron produced each part of the event and recover per-electron shower information from the combined response.

2.2.4 Trigger Scintillator and Trigger-Pad Tracks

The TS estimates how many beam electrons enter the target in the same readout window and where they pass through the beam region. It consists of three modules: two directly upstream of the tagging tracker and one directly upstream of the target. Each module contains two staggered layers of plastic scintillator bars. A traversing electron deposits energy in a bar and produces scintillation light that is measured as a photoelectron

signal. Adjacent bar hits are joined into clusters, and clusters across the three modules are combined when their positions are compatible with an electron travelling along the expected beam direction [5]. Forming these track candidates is itself a rule-based reconstruction method: low-level bar signals are converted into estimates of electron position and multiplicity.

In this thesis, the reconstructed TS track candidates are referred to by their name in the research-group codebase, `TriggerPadTracks`. The term describes objects derived from TS measurements, not a separate detector, and should not be confused as so. Each candidate provides a one-dimensional position estimate and a photoelectron count. Together, the candidates provide information about the positions, energy and the multiplicity of the incoming electrons. They are reconstructed detector objects rather than simulation-truth labels, and correspond one-to-one with the incident electrons only when the reconstruction makes correct prediction [5].

2.2.5 Triggering

The beam is divided into short time slots called radio-frequency buckets, which are delivered in groups called trains. Within a train, neighbouring buckets are separated by 26.9 ns, corresponding to a 37.14 MHz clock for the detector electronics. The nominal beam structure contains 20 filled buckets in each train at a train rate of 1 MHz, with an average of approximately one to two electrons per filled bucket.

Storing the full detector response for every filled bucket is neither feasible nor desirable: the input rate is much larger than the available readout and storage rate, and almost all events arise from ordinary processes not displaying the benchmark interaction. The online trigger therefore receives compact information from the TS, ECal and *hadronic calorimeter* (HCal) at the full clock rate and decides in real time whether the complete event should be read out and stored. The Trigger Scintillator supplies an incoming-electron count n , and the ECal supplies coarse energy sums. A basic online estimate is

$$E_{\text{miss}}^{\text{trig}} = nE_{\text{beam}} - E_{\text{ECal}}^{\text{trig}}, \quad (2)$$

Here, $E_{\text{miss}}^{\text{trig}}$ is the trigger-level estimate of the missing energy, n is the number of incoming electrons estimated by the Trigger Scintillator, and E_{beam} is the nominal beam energy per electron. The product nE_{beam} therefore estimates the total incoming energy, while $E_{\text{ECal}}^{\text{trig}}$ is the energy measured by the ECal using its coarse trigger readout. The superscript *trig* distinguishes quantities available to the online trigger from those obtained through the more detailed offline reconstruction. Over-counting the electrons can create false missing energy, while under-counting can cause a real signal event to be rejected. The trigger reduces the input rate to an average readout rate of about 10 kHz. The primary missing-energy trigger uses part of this bandwidth, while the remainder supports other physics, calibration and monitoring selections [5].

The distinction between online and offline reconstruction is set by this decision. Online reconstruction occurs before storage and must satisfy strict limits on latency, memory and transferred information. It therefore uses reduced quantities such as ECal trigger-cell sums and Trigger Scintillator track candidates. Once an accepted event has

been fully read out, offline reconstruction (which technically has no deadline) can use individual ECal cells and more computationally demanding algorithms to construct calibrated hits, tracks, showers and analysis quantities [5].

The methods developed in this thesis use simulated ECal hits at the full cell granularity, together with reconstructed trigger-pad tracks. They therefore belong to offline reconstruction research. Their results can show whether electron showers can be separated under multi-electron conditions. Deployment in the online trigger would be a separate task because it would require reduced trigger inputs and a dedicated latency and hardware study.

2.2.6 Pile-Up, Reconstruction and Physics Reach

In this thesis, *pile-up* mainly refers to *in-time pile-up*, where two or more beam electrons arrive in the same time sample and their detector responses are recorded together. **True?: Trigger latency does not create in-time pile-up. It follows from the number of electrons delivered in the same bucket.** Lowering the beam intensity would reduce it, but would also reduce the number of EoT accumulated per unit time. The practical challenge is therefore to retain the advantages of higher beam intensity without losing events because their detector responses are combined [5].

To a very good approximation, separate beam electrons propagate independently: the probability that two beam electrons interact directly with each other is negligible. This simplifies simulation as their propagations can be simulated independently and then *overlayed* before simulating their detector responses. Let $Z^{(k)}$ denote the true ECal energy-deposition pattern from source electron k . Under this simulation assumption,

$$Z^{(k)} \perp\!\!\!\perp Z^{(\ell)} \quad \text{for } k \neq \ell. \quad (3)$$

After overlay and detector reconstruction, the observed hit collection from the detectors is a combined response,

$$\mathcal{H} = \mathcal{R}(Z^{(1)}, \dots, Z^{(m)}). \quad (4)$$

The response $\mathcal{R}$ is a nondeterministic mapping that includes detector granularity, thresholds, noise and hit reconstruction. Deposits from different electrons may then occupy the same cell or form overlapping showers. The observed collection $\mathcal{H}$ cannot in general be divided into its original components $Z^{(1)}, \dots, Z^{(m)}$ by a simple geometric cut.

The overlap affects both triggering and later analysis. An incorrect electron count can produce an incorrect missing-energy estimate. Overlapping showers also make the ECal topology harder to interpret and can cover small deposits that are needed to reject backgrounds. The trigger must therefore use thresholds that depend on the estimated electron count. Offline reconstruction can use the full stored detector response to determine which multi-electron events remain separable and how much per-electron information can be recovered. Such studies can later inform which event classes are useful enough to retain at trigger level, although deploying an online method requires a separate trigger study. This thesis addresses the offline problem by assigning the combined ECal activity to the different beam electrons [5].

Multi-electron events are also an opportunity. The design report estimates that 4×10^{14} EoT can be accumulated in about twelve months with an average of one electron per sample. Reaching exposures of order 10^{16} EoT efficiently requires higher beam intensity and the ability to use events containing several electrons. In a design study with an average multiplicity $\mu = 1$, accepting up to four reconstructed Trigger Scintillator tracks instead of only one increased the total signal trigger efficiency from 0.58 to 0.88 at approximately the same 1 kHz trigger budget. This result used a 0.1 GeV dark-photon benchmark and the energy in the first 20 ECal layers [5].

Rejecting every multi-electron sample would discard part of the delivered beam and extend the time needed to reach a given exposure. If overlapping showers can instead be reconstructed reliably, more recorded events become useful and the intended sensitivity can be reached sooner. This is how pile-up handling can increase the practical physics reach of LDMX.

The specific problem studied in this thesis is a step towards that goal. The study considers two-electron and three-electron events as direct extensions of the single-electron case; higher multiplicities are outside its scope. The samples are constructed from independent single-electron simulations, and the models attempt to assign ECal hits to their electron of origin. The study does not itself search for DM. It tests whether modern reconstruction methods can recover per-electron shower structure from the combined detector response produced by multi-electron operation.

2.3 Machine Learning Methods for Reconstruction in High Energy Physics

Short introductory text here stating that ML is a topic of rising interest in HEP and refer to output of ML-related papers in HEP. [20]. **I have realized that the relevancy of this section is heavily questioned... Just cherry-pick relevant parts from here and ommit the rest I have limited space and this is not a popular science summary this background should cover necessary theory and it should cover necessary motivation for my study in my thesis, and maybe provide some context for the viewer also but nothing else. The fact that it is a rising tool of interest is still interesting to mention, but only in the contexts of why that is not simply a "trend follower". The rising tool of interest should be 1 or max 2 sentences.**

text here

machine learning (ML) has become a tool of rising interest in *high-energy physics* (HEP), driven by the growing complexity of detector data, the need for efficient event selection, and the search for rare signatures of physics beyond the SM. At the same time, advances in deep-learning methods and accelerator-based computing architectures such as *graphics processing units* (GPUs) have made increasingly powerful ML and non-ML approaches practical for scientific workflows [21]. Machine learning and deep-learning methods are not new in HEP, but the recent rapid growth of machine-learning applications in particle physics is

reflected both in dedicated reviews and in the continuously updated Living Review of Machine Learning for Particle Physics [20, 22, 23]. Figure 9 shows the trend of machine-learning-related publications within HEP significantly increasing in the last few years.

The underlying motivation for ML in HEP is that such tools may increase the *physics reach* of experiments, meaning their ability to probe weaker signals, rarer processes, or more challenging regions of parameter space, either by improving sensitivity enabling new discoveries or by making reconstruction and selection more computationally efficient.

a) Reconstruction

Reconstruction in HEP refers to reconstructing the physical interpretables from a measured particle event based on the detector outputs. Modern HEP experiments produce high-dimensional and heterogeneous data, where each event may contain a variable number of detector signals distributed across several subdetectors. This makes reconstruction a quite natural application area for machine learning: the task is to transform heterogeneous low-level or intermediate detector information, such as hits, tracks, and calorimeter clusters, into physically meaningful quantities that can be used for triggering, event selection, and analysis.

A useful way to view reconstruction is as an approximate inverse problem. In simulation, a detector may be regarded as a map from an underlying particle-level event to a set of detector measurements. If the particle-level event, representing the physical "truth", is denoted by Y , and the corresponding detector response by X , the simulation can be written schematically as

$$S(Y) = X, \quad (5)$$

where S represents the detector response. Reconstruction, denoted R , then attempts to inverse the mapping, i.e

$$R(X) \approx S^{-1}(X) = Y. \quad (6)$$

In practice, this inverse is imperfect and often ambiguous, based on the fact that some information is always lost, has stochastic evolutions or only indirectly observed: detector signals have finite resolution and particles may overlap in the detector. These cases does not exclude relevance however, as it could also be useful to study these scenarios as well and infer where our tools of measurement might simply not be enough. Machine-learning-based reconstruction approaches formulate parts of this inverse problem as a learnable mapping from detector-level inputs to physics-level targets, typically using simulated events where truth information is available to us and constructing target information is completely feasible.

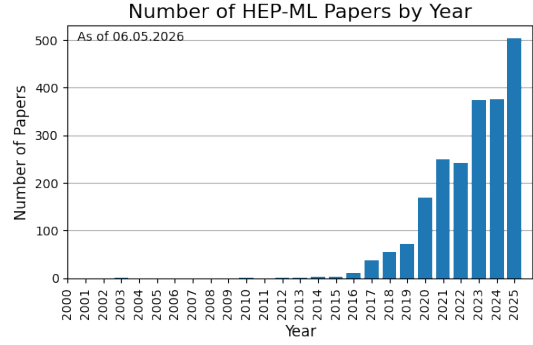


Figure 9. **Increase the size of labels here** Number of machine-learning-related publications on arXiv in the HEP category per year, based on the Living Review of Machine Learning for Particle Physics [20].

2.3.1 Computational Context in Experimental High Energy Physics

Let Nairit read this section!

The computational setting of HEP is also important for understanding the interest in machine-learning-based reconstruction. Large-scale HEP computing has traditionally been dominated by distributed, *Central Processing Unit* (CPU)-based workflows. [21] This is natural because collision or fixed-target events often are, to a good approximation, independent processes, analog to the electron case of LDMX $Z^{(k)} \perp\!\!\!\perp Z^{(\ell)}$ where $k \neq \ell$, following the notation introduced in Section 2.2.6. Simulation, reconstruction, and analysis can therefore often be parallelized over events with relatively little communication between tasks, making CPU clusters and grid computing highly effective, somewhat bypassing an immediate need of software engineering concurrency, which explains the longevity and success of this computing paradigm within HEP.

At the same time, the wider computing landscape has shifted toward heterogeneous architectures, in particular GPUs and other accelerator¹-based systems. This development is strongly connected to modern machine learning, where many operations reduce to dense linear algebra and can be executed efficiently on massively parallel hardware, including both training and inference tasks of deep learning ML models.

This shift creates both a challenge and an opportunity for HEP. Rule-based reconstruction algorithms have often been designed around detector-specific logic and sequential or iterative procedures. Such algorithms can be highly effective, but they may be difficult to port efficiently to accelerator hardware and may scale poorly with detector occupancy. Machine-learning models, by contrast, often express reconstruction as a fixed sequence of tensor operations, making inference naturally suited to GPUs or other accelerator architectures.

2.3.2 MLPF, a Recent Example of ML-based Reconstruction in HEP

All of the above stated reasons creates a modern context that make up the underlying motivations behind the *machine learning particle flow* (MLPF) reconstruction method, designed for the CMS experiment and the incoming *high luminosity phase* of the *Large Hadron Collider* (LHC). The original MLPF study published in 2021 used a graph-neural-network-based architecture for efficient particle-flow reconstruction in high-pile-up conditions [7]. The latest implementation from early 2026 demonstrates a transformer-based MLPF algorithm integrated into the CMS reconstruction software and evaluated on both simulation and collision data [8]. In the latter study, modern hardware, transformer-based computational architectures and recent progress of the transformer architecture are explicitly part of the motivation for scalable reconstruction in high-dimensional, variable-size event data.

The MLPF showcases the motivation for ML based reconstruction quite perfectly combining all of the above stated reasons for why ML methods have sparked an interest. The MLPF uses the modern model architecture, compute is accelerated with modern

¹“Accelerator” in this context refers to specialized hardware such as GPUs or even ML specialized architectures such as the relatively new *tensor processing unit* (TPU) designed by Google that accelerate the speed of computation for applicable processes[needed]

hardware, the method shows promise of improved speed of compute, resolution of reconstruction and it is motivated by the context of LHC high luminosity phase, which will increase complexity with a much higher rate of pile-up.

For the purpose of this thesis, the most relevant role of machine learning is therefore not only as a final classifier that separates signal-like and background-like events, but as a possible reconstruction method. Traditional reconstruction algorithms are often built from physics-motivated rules and detector-specific heuristics. Such methods remain essential, but they may become difficult to optimize in environments with high occupancy, irregular detector geometry, or overlapping detector signatures. Machine-learning-based reconstruction instead attempts to learn parts of the reconstruction mapping $R(X) \approx S^{-1}(X) = Y$ directly from simulated or measured data. It is also important to understand that rule-based and ML reconstruction is not either/or, they are the most effective when they work in junction and complement each other. The MLPF model takes some object-level inputs that are the results of rule-based reconstruction intermediate outputs such as clusters and particle candidates, in which the ML-based MLPF can utilize for stronger inference results [7].

2.3.3 The GNN and the Transformer under Pile-Up Conditions

ML-based methods are perhaps the most relevant under pile-up conditions, where detector hits from several particles or interactions may overlap and where the association between detector activity and the underlying particle origin becomes ambiguous. In this setting, a model that operates directly on hit-level or object-level detector information may provide additional information beyond a purely rule-based reconstruction, for example by assigning hit-level probabilities or by learning correlations that may be difficult to capture manually with code. One must still remember however that ML-methods are certainly also based on "rules" in the end, the strength of these methods in this context lie in that their discovery of efficient, or even *optimal* "rules", are somewhat automatic and a researcher can explore the space of solutions efficiently.

Architectures such as *graph neural networks* (GNNs) and *transformer* are particularly well suited to this type of problem because they can operate on variable-size collections of detector objects and learn correlations between them without requiring the data to be arranged on a regular grid. This is important for particle detectors, where the relevant information is often distributed over irregular detector geometries and across multiple subdetectors. By contrast, architectures such as *convolutional neural networks* (CNNs) are naturally adapted to regularly spaced image-like data and therefore impose an inductive bias that is not always well matched to detector readout. Graph-based models instead allow calorimeter hits, clusters, tracks, or other detector objects to be represented as nodes, with edges describing geometrical proximity or learned relationships. Transformers provide a related approach but it uses different analogies than nodes and edges.

In this thesis, the same general motivation is applied to the LDMX setting. The aim is not to perform full general-purpose particle-flow reconstruction, as in CMS, but to build and study a reconstruction platform for identifying electron-induced activity

in the electromagnetic calorimeter down to hit-level statistics, especially under pile-up conditions. This is the regime where reconstruction ambiguities are expected to be most relevant, and where machine-learning methods may offer the largest benefit over rule-based approaches.

This is kind of outdated, should I include all of this or not? The rest of this subsection introduces the machine-learning context needed for the reconstruction study in this thesis. First, the existing rule-based reconstruction chain in LDMX is reviewed, with emphasis on clustering, particle-flow-inspired reconstruction, and the distinction between online and offline reconstruction. The existing machine-learning-based veto strategy in LDMX is then briefly discussed, since it represents an already planned use of machine learning in the experiment **not relevant for my study?**. The subsection then turns to machine-learning-based reconstruction in CMS, in particular the development of MLPF, which provides a useful point of comparison for the approach studied here. Finally, the basic ideas behind GNNs and transformers are introduced and compared. These architectures form the methodological basis for representing calorimeter hits and other detector objects as unordered sets of nodes or tokens, allowing event reconstruction to be formulated as a learnable mapping from detector-level inputs to physics-level targets.

Further motivation of why not to use CNN to use somewhere maybe:

This representation preserves individual detector objects rather than projecting the event to an image. The number of hits is event dependent, the ECal is 3D, and overlapping showers need not be aligned with a fixed rasterization. Moreover, trigger-pad-track information is not an ECal pixel quantity. Sets, learned graphs and attention-based token sequences provide natural ways of relating these heterogeneous objects while remaining insensitive to an arbitrary ordering of the input rows.

2.3.4 Graph Neural Networks and GravNet

A graph represents a collection of objects as nodes and specifies which pairs of nodes are connected by edges. Applied for HEP in reconstruction, a node can represent information from a subdetector: a calorimeter hit, a track or another detector object (some detector-derived quantity, for example computed clusters), with its measured quantities stored in a feature vector $\mathbf{x}_i$ as input. The edges define which nodes may exchange information. This makes graphs suitable for variable-size data on irregular detector geometries, without assigning a physical meaning to the order in which the objects are stored, which may seem like an interesting proposition for a physicist in HEP explaining their rising popularity in this field [24].

In the message-passing formulation used here, each layer first constructs a message from every neighbour j of node i . The incoming messages are combined and used to update the representation of node i :

$$\mathbf{a}_i^{(\ell)} = \text{AGG}_{j \in \mathcal{N}(i)} M^{(\ell)}(\mathbf{h}_i^{(\ell)}, \mathbf{h}_j^{(\ell)}), \quad \mathbf{h}_i^{(\ell+1)} = U^{(\ell)}(\mathbf{h}_i^{(\ell)}, \mathbf{a}_i^{(\ell)}). \quad (7)$$

Here, $M^{(\ell)}$ and $U^{(\ell)}$ are learned functions and AGG is an order-independent operation such as a sum, mean or maximum. The updated feature therefore depends on the

node itself and on information received from its neighbourhood. Stacking layers allows information to travel farther through the graph [24, 25].

The GravNet layer is a message-passing layer in which the graph itself is learned. From every current node representation $\mathbf{h}_i$, two linear projections produce coordinates $\mathbf{s}_i$ in a learned space and features $\mathbf{f}_i$ to propagate. The coordinates are not physical positions, although physical detector coordinates may be included among the input features. Each node is connected to its k nearest neighbours in the learned space. Its messages can be summarized as

$$\begin{aligned} \mathbf{s}_i &= \mathbf{W}_s \mathbf{h}_i + \mathbf{b}_s, & \mathbf{f}_i &= \mathbf{W}_f \mathbf{h}_i + \mathbf{b}_f, \\ \mathbf{m}_{j \rightarrow i} &= \exp(-c \|\mathbf{s}_i - \mathbf{s}_j\|_2^2) \mathbf{f}_j, & j &\in \mathcal{N}_k(i), \end{aligned} \quad (8)$$

where $c > 0$ is a fixed distance scale and where..... Nearby nodes in the learned space therefore contribute more strongly. The `GravNetConv` operation takes the element-wise mean and maximum of these messages and combines both with a transformed version of the original node feature to produce $\mathbf{h}'_i$. A new neighbourhood is constructed in every GravNet layer, so the relations between detector objects can change as their representations develop. Figure ?? summarizes this sequence of operations [6].

Flowchart here? Good to include or not visualizing GNN or the gravnetconv specifically?

The models in this thesis use either ECal hits alone or ECal hits and trigger-pad-track observations as nodes. A stack of `GravNetConv` layers returns one representation per input node. A shared output head produces one prediction vector per node, after which only the predictions for ECal hits are retained for the hit-origin task. The exact input features and model configuration are specified in Sections 3.5 and 3.6, respectively.

2.3.5 Transformer Encoders and Self-Attention

A Transformer operates on input objects called *tokens*. A token need not represent a word. In this thesis, tokens represent selected ECal hits and, in the mixed-detector models, trigger-pad-track observations. A shared input network maps the feature vector $\mathbf{x}_i$ of each token to a latent representation $\mathbf{h}_i \in \mathbb{R}^{d_{\text{model}}}$. For an event containing n tokens, these representations form a matrix $\mathbf{H} \in \mathbb{R}^{n \times d_{\text{model}}}$.

The central operation of a Transformer encoder is *self-attention*. For one attention head, three learned linear projections produce a query, key and value for every token:

$$\mathbf{Q} = \mathbf{H} \mathbf{W}_Q, \quad \mathbf{K} = \mathbf{H} \mathbf{W}_K, \quad \mathbf{V} = \mathbf{H} \mathbf{W}_V. \quad (9)$$

The projection matrices $\mathbf{W}_Q$, $\mathbf{W}_K$ and $\mathbf{W}_V$ are learned during training. The queries, keys and values themselves depend on the input event. Scaled dot-product attention is then

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q} \mathbf{K}^T}{\sqrt{d_k}}\right) \mathbf{V}, \quad (10)$$

where d_k is the query and key dimension and the softmax is applied to each row [26]. The query of token i is compared with the key of every token j . The resulting weight determines how much of the value $\mathbf{v}_j$ contributes to the updated representation of token

i. Full self-attention therefore lets every detector object in an event receive information directly from every other object.

A multi-head attention layer repeats this operation with separate sets of projections. The head outputs are concatenated and projected, which allows different heads to learn different token-to-token relations. An encoder layer then applies the same two-layer feed-forward network, with a ReLU activation, independently to each token. Residual connections and layer normalization are used around both the attention and feed-forward operations, and several encoder layers can be stacked. One encoder layer is summarized in Figure ?? [26].

Figure here? Good to include or not visualizing GNN or the gravnetconv specifically?

The original Transformer adds a positional encoding because word order is meaningful. The row order of detector objects has no corresponding physical meaning, so the encoder models used in this thesis do not encode the arbitrary row index. Physical position is instead supplied through detector-coordinate features. Without a row-position encoding, self-attention is permutation equivariant: reordering the tokens produces the same reordering of the per-token outputs [27]. Shorter events are padded when events are batched, and a mask prevents real tokens from attending to the padded rows.

The models in this thesis use only the Transformer encoder, not the autoregressive decoder of the original architecture. The encoder returns one representation per token. Task-specific heads use these representations for per-hit predictions, while pooled representations can also support event-level predictions. Each attention head forms an $n \times n$ matrix of pairwise weights, so the memory and computational cost of full attention grow quadratically with the number of tokens. Its global information exchange also provides the connection to graph message passing developed in the next subsection.

2.3.6 An analogy between the Transformer and the GNN

Transformer ($O(N^2)$) Gravnet ($O(Nk)$)

The Transformer is a special case of a GNN. This analogy is relevant to be aware of, and the name GNN refers rather to a large catalogue of different implementations but what they all have in common that the input has a suitable analogy as being seen as nodes in a graph in the learned representation of the network, and then message passing is performed along these. The input to a transformer are called tokens, or even sloppily called words in the context of natural language processing, but one can mathematically show how a generic GNN could be molded into an equivalent network of a transformer, something Chaitanya K. Joshi has derived [28], adding to deconstructing the distinction around the terms nodes and tokens.

There is also another GNN architecture that is kind of the middle ground of a generic GNN and the transformer, known as *graph attention networks* (GAT), which implements the attention mechanism to GNNs, but the attention is not applied globally and being still restricted to local neighborhoods. The derivation in Joshi's paper molds a GNN into a transformer by considering a GNN with a fully connected graph (i.e, global) and

also implementing the attention mechanism to it, obtaining a mathematical equivalent between the two networks [28].

The architectural differences between the two networks might have been blurred, but the transformer architecture has one benefit over the GNN currently, namely of “winning the hardware lottery”². There is one practical difference between the two classes, and that is that the self-attention mechanism for all tokens in an input set $\mathcal{S}$ can be computed in a highly parallelized manner as follows:

$$\tilde{\mathbf{H}}^\ell = \text{softmax} \left(\mathbf{H}^\ell \mathbf{W}_Q^\ell (\mathbf{H}^\ell \mathbf{W}_K^\ell)^\top \right) (\mathbf{H}^\ell \mathbf{W}_V^\ell).$$

Where $\mathbf{H} \in \mathbb{R}^{n \times d}$ is a matrix of initial representations. **(i took these snippets directly and I dont know how much I want to include or how I can condense) snippet 1:** Transformers implement global multi-head attention through highly optimized dense matrix multiplication operations that leverage the parallel processing capabilities of modern GPUs and TPUs. [28]. **snippet 2:** The multi-head attention mechanism can also be similarly parallelized by performing a single linear transformation for all heads in parallel, followed by reshaping the queries, keys, and values per head to tensors of dimension $n \times k \times d/k$ each. The token-wise operations which follow multi-head attention are independent of other tokens and trivially parallelizable as well. In contrast, GNNs typically perform sparse message passing over locally connected structures, which is substantially less efficient on current GPUs for typical problem scales (with the exception of very sparse or billion-scale graphs). This involves maintaining an index of neighbours for each node, and performing gather and scatter operations to propagate messages [Fey and Lenssen, 2019]. As a result, GNNs are orders of magnitude slower to train and challenging to scale compared to standard Transformers on current hardware.

My text again: This means that if one would design one GNN-based model and one transformer-based model whose performance on their tasks are equally accurate, then the transformer-based model is, if no other apparent downsides, likely the better choice since inference and training will run faster and more efficient on the modern hardware, at least given the clear current shift in the hardware industry. The speed of inference is also especially important if one would design a ML-based online reconstruction model meant for being used, at some capacity, in real-time of the experiment, maybe even in triggering. The aspect of speed on modern hardware is specifically one of the mentioned reasons for why the latest iteration of the MLPF model abandoned GNNs for transformers instead [8].

2.4 things that I want to include but have maybe not found a suitable place for yet

* GNN, Transformer work with unordered sets of input compared to CNN that is on a regular grid. We are using different detectors and different objects and the ECal has a

²The term, introduced by Sara Hooker, describe when a research idea in computer science wins because it was suited to the available software and hardware and not because the idea is superior to alternative research directions. [29]

hexagonal grid and all of this adds to difficulty of using CNNs.

Briefly mention clustering and reconstruction methods that LDMX has already implemented and plans to potentially use. Describe these existing LDMX reconstruction approaches very shortly, and these are based on other methods developed at CMS CERN already (clustering CLUE, particle flow PF), where the the detector design such as the ECal is the inspiration and what LDMX is based at giving that the LDMX detector is a so-called "sibling detector".

Briefly mention that machine learning approaches have been explored by previous thesis students testing CNN, GNN and even RNN, and reference these.

Briefly mention and explain and a particle-net (GNN) implementation that was made for ML-based Veto in LDMX (explain what a veto is).

2.4.1 ML-based Reconstruction in CMS

comment: This is a title that I think was interesting when I thought to explain partienet, though I have come to the conclusion with my supervisor that explaining and detailing other reconstruction methods is not directly relevant to my work and there is not enough time to expand my analysis or to explain all of this in detail I can only briefly mention it for useful context for the reader and to motivate my study compared to previous students.

3 Methodology

The methodology was written before the finalised models and the mlpf inspired experimental model. In order to make this methodology as comprehensive and readable as possible, we should consider to move out content to the appendix to free up space here. Consider what parts of the methodology that would be best to move to the appendix. The parts that are mostly practical and that does not have too much importance in chosen research methodology related to the research question could be moved to the appendix. ROOT reading could probably be moved to appendix.

I should include a definition of permutation-invariant accuracy in methodology and I think that I should also include in this section as well the "can a global label swap recover the event?" plot that validates exactly this as an argument and then I should very clearly state that the accuracies that I report henceforth will be the permutation invariant accuracy, i.e the global swap permutation that yields the highest accuracy and then I should argue for why this accuracy is in fact the only interesting and relevant one for ldmx since electron has no inherent ordering and what we are in fact trying to achieve is electron shower separation and we are trying to form groups, there is no ordering difference the y-axis centroid ordering is a practical solution and just "arbitrarily" chosen

This chapter will answer the question for the reader "what was built, what was tested and how was it done?". Firstly, a concise overview is given defining the scope, the

experimental design, and the framework that was developed for this thesis. Afterwards, the simulation and data generation is specified and how that data is treated and carried through a pipeline. Lastly the chosen model architectures are covered and how they were trained and evaluated.

Rough structure overview:

- Simulation -> root file
- Supervised targets (approximations) (this tells us what we need to look at in data)
- Data extraction (root reading, IO), preprocessing, tensorization. Now we are ML ready.
- Models
- Training objective + optimization
- Eval + plots

3.1 Data Generation Overview and Reconstruction Formulation

This work develops a framework for a supervised ML pipeline for separating overlapping electron showers in the LDMX ECal, named "ml_ldmx" [9], and with the **MOVE TECHNICALITIES (PYTORCH) TO LATER DONT HAVE THAT IN THIS SUBSECTION** PyTorch package implements a couple of neural network models within this framework, based on the GNN and the Transformer architectures. The method spans across two frameworks, *LDMX-Software* (ldmx-sw) and ml_ldmx, and these are supported on UNIX systems: Linux, Mac OS or WSL ³ on Windows. Large files were stored at, and heavy computational tasks (including simulation and training) were run on the COSMOS *high-performance computing* (HPC) cluster that runs Linux, hosted at Lund University. [30]

3.1.1 Simulation and Data Pipeline Overview

Comment: this is basically a (necessary) transcript of the content in figure 11, that also provides a high level overview of the method

The experiments conducted starts outside of ml_ldmx, and begins with the simulation of the particle events with their simulated detector responses. The simulations were done in the open source ldmx-sw framework [31], developed by the LDMX collaboration for simulation and analysis of this specific experiment and detector. The ldmx-sw framework is built upon *GEometry ANd Tracking 4* (Geant4), a Monte Carlo based toolkit for simulating the passage of particles through matter [32].

Figure 11 provides a high-level overview useful for the reader at this stage, showcasing where the data generation starts, and how the simulation data is carried through a pipeline. In the pipeline the information required for the supervised ML tasks extracted and processed, going from ldmx-sw to ml_ldmx.

The simulations are configured with high-level *configuration scripts*, written in Python that utilizes Geant4 and LDMX-specific implementations and adjustments written in C++ from the ldmx-sw codebase. The simulations generate single electron

³Windows Subsystem for Linux, allow native Linux environment and programs to run on Windows

(1e) events of 1 electron present in one detector readout window and the data is stored as ROOT files (.root). These 1e events are afterwards *overlayed* onto each other creating pile up events containing 2 electrons (2e) or 3 electrons (3e) in each event. The particle events are assumed to be independent *mathhereofindependentevents*, purely physically speaking. The detector response, however, is not independent for overlapping showers so it was simulated again for the pile up events **regarding electron multiplicity non-independence detector response verify or formulate better? is it mentioned already somewhere else?**.

Once the simulations are done, the relevant information from the ROOT files has to be extracted, collected, relabeled and finally converted into *tensors* (i.e "tensorized"), and this is a suitable format for ML models that PyTorch (.pt) can handle. At this stage it was also natural to apply any other preprocessing steps before storing the "ML-ready" tensors in *shards* (collection of tensors into single .pt files) to be used for the different models developed.

3.1.2 Reconstruction Formulation

For one event, the observed ECal response is represented as the variable-size set

$$\mathcal{H} = \{(\mathbf{r}_i, E_i)\}_{i=1}^{N_{\text{ECal}}}, \quad \mathbf{r}_i = (x_i, y_i, z_i), \quad (11)$$

where E_i is the reconstructed energy of hit i . The standard event representation also includes the variable-size set of reconstructed TS track observations

$$\mathcal{T} = \{(c_j, n_j^{\text{pe}})\}_{j=1}^{N_{\text{TS}}}, \quad (12)$$

where c_j is the one-dimensional track centroid and n_j^{pe} is the corresponding reconstructed number of photoelectrons. The complete event input is therefore represented by the pair $\mathcal{X} = (\mathcal{H}, \mathcal{T})$. The ECal-only model variants use the restricted input $\mathcal{X} = \mathcal{H}$, omitting $\mathcal{T}$.

The principal supervised task is to assign each selected ECal hit to the origin electron responsible for the largest total deposited energy in that cell (since one ECal cell can have energy contributions from overlapping originating electron showers). The origin electron classification was the result of attempting to start with the most simple hit-level task that was thought of for shower separation and it produced two variants of classification models that were interesting to compare: one GNN model based on the GravNet architecture and one Transformer model with full self attention.

The practical use of the origin electron classifiers are inherently limited, and this motivated for experimenting with extended tasks that includes predicting whether an event is (2e) or (3e) and the contributed fraction of energy in shared hits. This resulted in one multi-task model, more experimental in nature, that was also based on the Transformer architecture.

3.2 Simulation

The samples used in this study were produced with `ldmx-sw`. Its versioned container release captures the framework and its software dependencies, including Geant4. The relevant configuration is summarized in table I; the exact simulation, overlay and job-launch scripts are archived with the thesis code [1].

Table I
SOFTWARE AND EXECUTION CONFIGURATION USED FOR EVENT PRODUCTION.

Item	Configuration
Simulation framework	<code>ldmx-sw container ldmx/pro:v4.7.3</code>
Detector geometry	<code>ldmx-det-v15-8gev</code> with the minimal scoring planes enabled
Beam configuration	One 8 GeV electron generated upstream of the tagging tracker
Execution environment	COSMOS Linux HPC cluster; jobs submitted with Slurm

3.2.1 Single Electron ($1e$) Event Simulations

The single-electron source sample was generated with the configuration in table I, using the `single_8gev_e_upstream_tagger()` beam generator. No interaction biasing or event-level filter was applied.

The resulting events are referred to as *inclusive*: they were not selected for a particular interaction process. Instead, an electron was delivered from the upstream beam position and its subsequent interactions were determined by the transport simulation. The sample is therefore dominated by electrons without a hard target interaction, while SM interactions remain present at their simulated rates [5]. No dark-sector signal process was injected. For clarity in this report, the electrons drawn from this inclusive sample are called *non-signal electrons*, in the larger of context of LDMX these electrons can sometimes be called *pile-up electrons*, as they often serve the purpose of acting as additional interactions in an overlay.

An existing source pool of about 10^9 of such events was available for this study [hans_alin]. Most input ROOT files contained 10^4 events, although some contained slightly fewer due to some error. The measured number of entries in each file was relevant to check when constructing the overlay jobs to guarantee uniqueness of every electron present in an pile-up event.

3.2.2 Simulating Events with Electron Multiplicity ($2e$, $3e$)

Events with a multiple number of electrons, also called "*pile-up events*", were constructed by *overlaying* independently simulated single-electron interactions onto each other in space. The `ldmx-sw` overlay combines their simulated hit contributions by detector channel before digitization (detector response) and reconstruction. The particles are transported independently, but the digitization and reconstruction has to be simulated again after an overlay since the event may contain overlapping detector deposits that must be processed jointly in one event.

The configuration was based on the `ldmx-sw` in-time pile-up validation workflow [33]. It reads one ROOT file as the *main* event stream and a second as the *overlay* stream.

These names describe the streams’ software roles; both streams contain the same type of inclusive, non-signal events. For a requested electron multiplicity $p \in \{2, 3\}$, Poisson sampling was disabled and the overlay multiplicity was fixed so that each output event contained one main electron and $p - 1$ overlay electrons.

Source electrons were not reused between the generated events, as every simulated electron ought to be *unique* in training and validation. Jobs were assigned unique source inputs, and the number of events written by an overlay was bounded by

$$N_{\text{out}} \leq \min \left(N_{\text{main}}, \left\lfloor \frac{N_{\text{overlay}}}{p - 1} \right\rfloor \right), \quad (13)$$

where N_{main} and N_{overlay} are the actual entry counts of the two input files. This bound prevents the overlay reader’s wrap-around behavior from selecting an electron more than once within a job. For the typical input size of 10^4 events per file, this yielded 10^4 $2e$ events or 5×10^3 $3e$ events per output file.

The target of 10^7 events for each pile-up multiplicity was selected, following the supervisor’s recommendation, to provide a large independent pool for model development and the study of infrequent reconstruction failures. The resulting pile-up dataset therefore contained 10^7 events, for the two cases respectively. This sample size is not a direct test of the experiment-level background rejection required by LDMX: those requirements are process-specific and can reach twelve orders of magnitude for backgrounds associated with few-GeV bremsstrahlung [34]. The `ldmx-sw` Docker⁴ container release v4.7.3 was mounted on the COSMOS computing cluster. The container release also included an origin-identifier fix that was implemented by the author required to retain the source-electron labels after overlay [35]. The overlay configuration script in Python, and how it would work its way through the available $1e$ dataset and where to output the pile-up events, were set up with an `sbatch`⁵ script and afterwards sent as a job to run on the HPC cluster. Table II summarizes the dataset that was generated for this study and the overlay production completed in a few hours in total.

Table II
SIZES AND CONSTRUCTION OF THE INCLUSIVE (NON-SIGNAL) EVENT SAMPLES.

Sample	Event construction	Events per file	Total events	Release
$1e$	Inclusive event 8 GeV beam	10^4	$\sim 10^9$	<code>ldmx-sw</code> v4.6.2
$2e$	1 main + 1 overlay electron	10^4	10^7	<code>ldmx-sw</code> v4.7.3
$3e$	1 main + 2 overlay electrons	5×10^3	10^7	<code>ldmx-sw</code> v4.7.3

Figure 10 shows 3 *event displays* from the simulations. The event display is focused on the ECal detector showcasing the hits inside the detector (in Euclidian space) from incoming electrons that causes these electron showers in a typical $1e$, $2e$ and $3e$ event. Only the hits are visible in the detector and the coloring of the hits is dependent on

⁴Docker is a platform for packaging applications and their dependencies into portable, isolated environments called containers. `ldmx-sw` uses Docker containers to collect dependencies, code and updates the framework with continuous new releases over time.

⁵(Slurm batch) is the command used to submit batch jobs to a Slurm-managed computing cluster. Slurm is an open-source workload manager used to schedule and allocate computing resources on HPC clusters.

which originating electron that caused it. Note also that the coloring also represents a perfect electron shower separation.

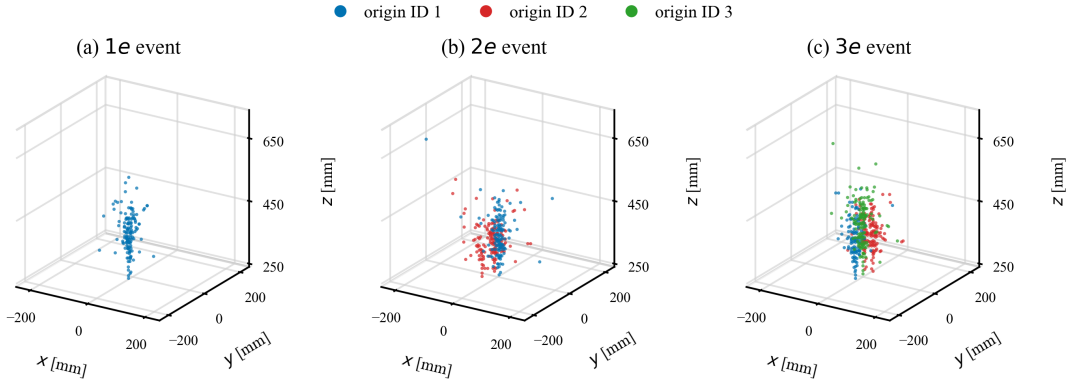


Figure 10. Three-dimensional ECal event displays of randomly selected $1e$, $2e$ and $3e$ events. Each marker represents a selected ECal hit and is coloured by the electron origin ID with the largest summed deposited-energy contribution to that hit: origin ID 1 is blue, origin ID 2 is red and origin ID 3 is green. Noise hits are omitted. Only simulation truth is shown; no model predictions are used.

3.3 Simulation Information and ROOT Extraction

This section first defines which detector quantities are used as model *inputs* and which simulation quantities are used as *targets*. It then explains how these quantities are read, matched, converted, saved and prepared for training. Some technical details are moved to the appendix but referenced from this section.

The data appear in four forms. The first is the original set of ROOT branches. The second is a temporary per-event record used during conversion. The third is the saved ML-ready tensor event. The fourth is the version passed to a model that it “sees” after the chosen target, noise and normalization steps have been applied. Figure 12 shows the separation between the detector input collection simulating what the LDMX detector would measure in the real-world scenario and simulation-truth information used to build targets and to calculate loss for the supervised ML models. Figure 38 shows a high-level overview the conversion pipeline in a visual manner.

The simulations are stored in `.root` files. ROOT is a C++ data analysis framework widely used in high-energy physics. In the LDMX files, a `TTree` stores each event as one entry and divides its quantities into branches. This compressed, column-based structure allows selected branches to be read efficiently and to be highly storage-efficient [36–38]. The Python pipeline uses the `uproot` package to read these branches without requiring the C++ ROOT framework, and `Awkward` arrays to handle the nested data whose lengths differ between events [39–41]. The data are then converted to `PyTorch` tensors and saved in `.pt` shards ready for machine-learning applications with the tools that the high-level ML tools that the python package offers [42].

ROOT reading, tensor and target construction, and model training are kept as separate stages. This is a conscious and important design choice. It keeps the target definition in-

dependent of the model architecture and allows new models to reuse the same processed data.

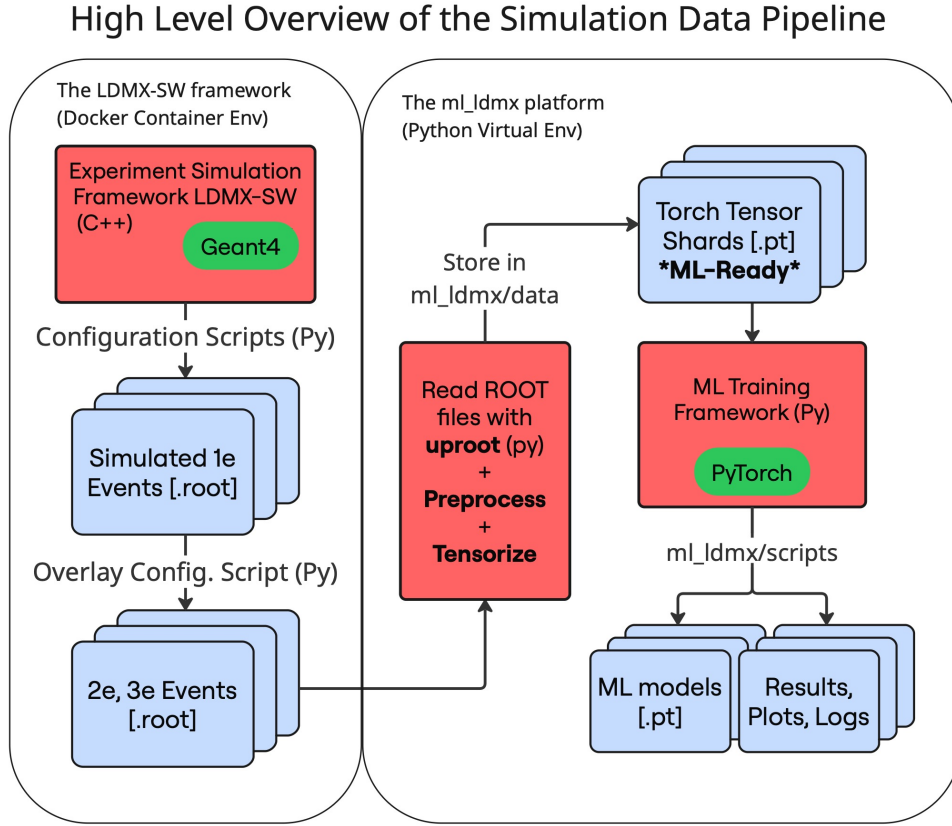


Figure 11. A flowchart of this thesis method presenting a high-level overview of how the data flows through different stages and different frameworks, from simulation to ML-model training.

3.3.1 Selected Detector Data and Simulation Truth

Only quantities available from reconstructed detector data are given to the model. For an ECal *reconstructed hit quantities* (RecHit), these are the cell position and reconstructed energy, (x_i, y_i, z_i, E_i) . For a reconstructed *trigger-pad* (TPad) track, they are the one-dimensional centroid and photoelectron count, (c_j, n_j^{pe}) .

The trigger-pad tracks provide context from another subdetector but do not have their own hit-origin targets. Only ECal hits have these targets. The same network still produces a vector of class scores, called *logits*, for every input row. Before the loss and metrics are calculated, the `ecal_mask` selects only the ECal rows. Trigger-pad rows can therefore influence the ECal predictions, but their own output scores are ignored.

Training also requires simulation truth, but this information is not given to the model, as that would break what a model in a real scenario would be able to "see". Detector-cell identifiers are used to match reconstructed and simulated ECal hits. The deposited-energy and origin arrays describe the energy in each stored contribution and the overlay

origin to which it belongs. Separate track identifiers describe the simulated tracks. The noise flag is also used only when targets and dataset views are prepared. The truth is therefore used after a prediction to calculate the loss or evaluation metrics. It is not available to the model when the prediction is made.

Table III lists the selected fields. A complete branch path has the form `collection/collection.member_`. The collection and member names are shown separately to keep the table readable while preserving their exact ROOT spelling.

Table III
MAKE THIS LOOK MORE NICE SELECTED BRANCHES FROM THE `LDMX_EVENTS` TREE AND THEIR ROLE IN THE SUPERVISED PIPELINE.

Collection	Members read	Role
EcalRecHits_ overlay	id_, xpos_, ypos_, zpos_, energy_, is_noise_	Cell matching, reconstructed ECal position and energy, and the training-time noise policy
EcalSimHitsOverlay_ overlay	id_, track_ id_contribs_, edep_contribs_, origin_contribs_, n_contribs_	Cell matching and target construction from the deposited-energy and origin arrays. Track-ID and count arrays are only temporary
TriggerPadTracks_ overlay	centroid_, pe_	Reconstructed trigger-pad context supplied to the heterogeneous detector models

The track-identifier contribution array and the stored number of contributions are kept in the temporary record for inspection. They are not used by the maintained target definitions or copied into the final tensor shards. The RecHit and *simulated hit quantities* (SimHit) identifiers are needed for matching, but they are not model features themselves. Trigger-pad tracks are an estimate of the number of electrons detected, but they are not matched to the true individual simulated electrons.

The event multiplicity and source label are also not read from detector branches. They are taken from the configured $2e$ or $3e$ input source and added as source metadata.

3.3.2 Chunked ROOT Reading and Hit Matching

The selected collections are read from the `LDMX_Events` tree with `uproot` and `Awkward` arrays [39]. Only one chunk of selected branch arrays is held in the memory of a worker at a time, and it is configurable. The converted events from the current source file are collected until its output shard is written.

RecHits and SimHit are stored in separate collections and may appear in different row orders. Both collections contain a detector-cell identifier that uniquely identifies the corresponding ECal cell. For each event, the reader builds a lookup from cell identifier to SimHit record and uses it to attach the correct simulation contributions to every RecHit. This preserves the RecHit ordering while aligning each reconstructed hit with the truth information from the same cell. A noise hit may have no simulated contribution. **DOES THIS HAPPEN? FIND OUT** A non-noise hit without a matching contribution cannot

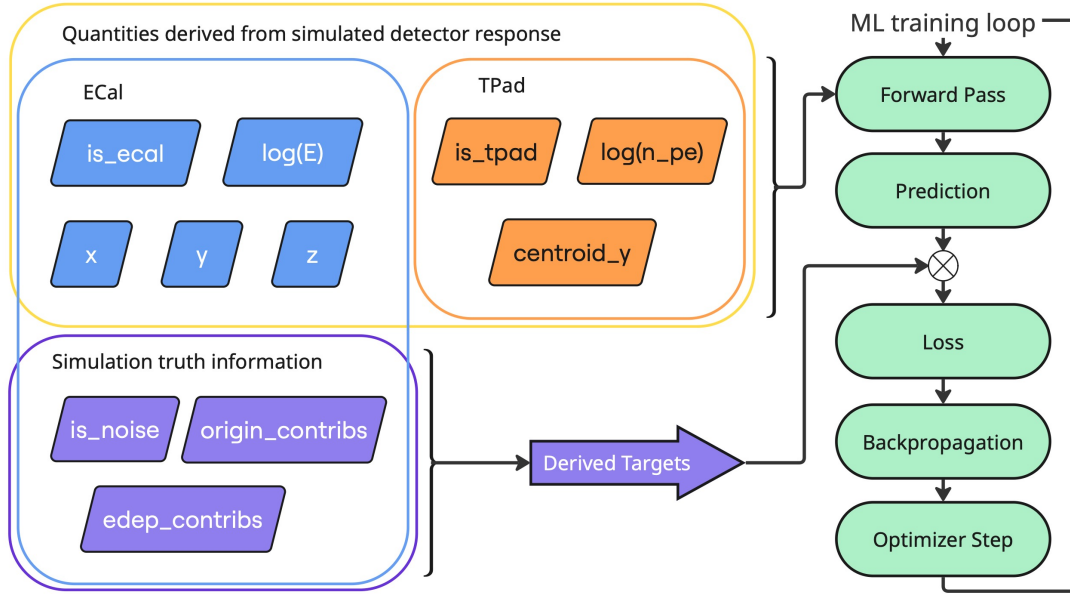


Figure 12. An illustrative flowchart showcasing what information, and at what stages, they are inserted into the ML training loop. Detector-derived quantities represent the information that can be read from the detectors in a real-world scenario, and the simulation-only truth information cannot be known in reality, but it can be utilized for supervised ML training.

be assigned a valid origin target, so conversion stops instead of creating an unsupported label.

This step produces a temporary Python dictionary of variable-length lists. It contains RecHit identifiers, positions, reconstructed energies and noise flags. It also contains the matched track, deposited-energy and origin contribution lists, together with a nested trigger-pad record for the centroids and photoelectron counts. This dictionary is only an intermediate format. It is different from both the original branch layout and the tensor dictionaries saved in the `.pt` shards.

3.4 Supervised Targets and Simplifications for the Baseline Models

The main task was to assign each selected ECal hit to its originating electron, as a successful performance of this task would mean a shower separation that would reconstruct an analysis of separate electrons without the ambiguity of pile-up. The baseline models use three main simplifications that are important to highlight:

- 1) **Known pile-up level.** Each fixed-multiplicity baseline is trained and evaluated on either $2e$ or $3e$ events. The number of electron classes is therefore indirectly known before reconstruction and this isolates the hit-assignment task. In a real-world scenario, however, the multiplicity would not necessarily be known in advance. A more desirable model, a full-event reconstruction model, would also be assigned the task of predicting the number of electrons present. This lead to the

experimental mixed-multiplicity model which also attempts to predict the number of electrons in the event.

- 2) **One hard label per hit.** An ECal cell may contain energy from several overlaid showers. For the baseline loss, the hit is assigned to only one origin label in $\{1, 2, 3\}$, called the *dominant origin*. It is the origin with the largest total deposited energy after contributions from the same origin have been summed.
- 3) **Noise-free baseline input.** Simulated noise hits are kept in the reusable production shards but removed before data are given to a baseline model. The experimental model can instead treat these hits by introducing a separate background class $\{0\}$.

3.4.1 Dominant-Origin and Energy-Fraction Targets

Let K_i be the number of simulated-particle contributions stored for reconstructed hit i . Contribution k has deposited energy d_{ik} and overlay origin identifier o_{ik} . For an electron origin j , its total contribution to the hit is

$$D_{ij} = \sum_{k=1}^{K_i} \mathbb{I}[o_{ik} = j] d_{ik}, \quad (14)$$

$$y_i^{\text{origin}} = \arg \max_{j \in \mathcal{J}} D_{ij}, \quad (15)$$

$$f_{ij} = \frac{D_{ij}}{\sum_{k=1}^{K_i} d_{ik}}. \quad (16)$$

Here, $\mathcal{J}$ is the configured set of electron origins and $\mathbb{I}[\cdot]$ is the indicator function. The baseline classifiers and the slot model use the hard target y_i^{origin} , while the slot model also uses the fractions f_{ij} to retain information about shared hits. The fractions sum to one when the configured origins account for all deposited energy in the hit. A hard label assigns the hit to one target class but does not imply that the overlapping energy deposits have been physically separated.

An earlier implementation selected the origin of the largest individual contribution, but this rule was omitted because it was qualitatively less meaningful and could make the learning task unnecessarily difficult, while the summed rule is better motivated physically despite its observed accuracy increase not being statistically significant.

3.4.2 Noise Handling and Electron Ordering

The production shards keep RecHits that carry the noise flag and mark them with a Boolean noise target. A noise hit is assigned to background class $\{0\}$, and its electron-fraction row is set to zero. These rows are removed before data are given to a baseline model. The experimental model can instead keep the background class. Noise rows are excluded when the target set of electron origins and its ordering are constructed. If they are retained, they still form part of the slot model’s event context.

Origin identifiers assigned during overlay only index each electron and therefore have no consistent physical ordering. Swapping two origin identifiers leaves the physical event unchanged, but it would swap the meaning of two output classes, potentially adding

heavy loss to successful shower separation. To obtain a consistent class definition across events, the electrons are therefore ordered according to a fixed spatial convention. For each origin represented among the hard-labelled, non-noise ECal hits, the unweighted mean y -coordinate is defined as

$$\bar{y}_j = \frac{1}{|\mathcal{E}_j|} \sum_{i \in \mathcal{E}_j} y_i, \quad \mathcal{E}_j = \left\{ i : y_i^{\text{origin}} = j \right\}. \quad (17)$$

Above math is a bit convoluted I think, better formulation that is easier to interpret but still confident and academic? Origins are sorted from the lowest to the highest value of $\bar{y}_j$ and mapped to consecutive canonical slots. The fraction-target columns are reordered in the same way.

This changes the class labels only. It does not sort the tensor rows, weight positions by energy, use trigger-pad centroids or match trigger-pad tracks to electron origins. The saved shards contain the original physical-origin labels. Canonical- y labels are constructed when the data are loaded. The original origin identifiers and fraction-column order are kept so the mapping can be traced.

3.5 Tensorization and Arriving at ML-Ready Storage

Each temporary event record is converted into numerical arrays, masks, targets and metadata. Events keep their variable size and the stored order within each detector collection. Tensorization uses fixed operations that are already implemented with `PyTorch`.

3.5.1 Combined Event Tensor

Let N_E and N_T be the total number of retained ECal hits and trigger-pad tracks in an event respectively. The ECal rows are placed first and the trigger-pad rows follow:

$$\mathbf{X} \in \mathbb{R}^{(N_E+N_T) \times 8}, \quad \mathbf{x}_n = \begin{cases} (1, 0, x_i, y_i, z_i, \tilde{E}_i, 0, 0), & n = i, \\ (0, 1, 0, 0, 0, 0, c_j, \tilde{n}_j^{\text{pe}}), & n = N_E + j. \end{cases} \quad (18)$$

The eight columns are `is_ecal`, `is_tpad`, `ecal_x`, `ecal_y`, `ecal_z`, `ecal_energy`, `tpad_centroid` and `tpad_pe`, in that order. Fields that do not apply to a row are filled with zeros. The first two columns identify the detector type and are represented with integer values either 1 (True) or 0 (False). The arrays `ecal_mask` and `tpad_mask` identify the two row types (observe that they are arrays it is a bit unconventional naming scheme here). Hit-level targets and losses apply only to the N_E ECal rows. Trigger-pad rows provide context only, but it can still enhance the models prediction if it identifies a meaningful extraction from this information.

An ECal-only model view selects $(x_i, y_i, z_i, \tilde{E}_i)$ from the same stored event rather than using a second processed dataset, so the combined event tensor can be utilized by all trained models. The collection order is preserved in storage, but the row index has no physical meaning for the models. ECal hits and TPad tracks are treated as unordered sets with variable size.

3.5.2 Preprocessing of Energy Input Features

The production shards used in this study apply a log transform of the energy-based features that are present in the input

$$\tilde{E} = \log(1 + \max(E, 0)), \quad \tilde{n}^{\text{pe}} = \log(1 + \max(n^{\text{pe}}, 0)), \quad (19)$$

using the `log1p` function in `PyTorch` [43]. By inspection, both input distributions are clearly right-skewed and this is typical for intensity-type energy features [44]. These transforms reduce their numerical range while preserving the order of the values, making it numerically more suitable for ML. Figure 13 shows their effect. The deposited energies used to construct truth targets are not transformed there is no motivation for that since they are only used to construct targets.

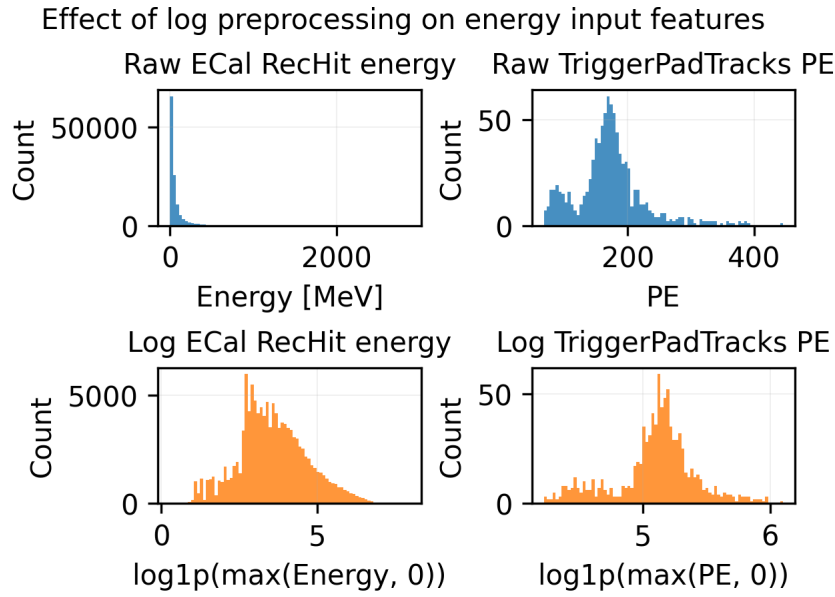


Figure 13. This figure was made to be half size wrapfigure I will update it so labels are better and ranges and so that it does not take up as much space. The plot will also tell how many events it is based on. Reconstructed ECal-energy and trigger-pad photoelectron distributions before and after the input $\log(1 + x)$ transform.

3.5.3 ML-Ready Shards and Stored Event Structure

The conversion writes one `PyTorch .pt` shard for each selected source `ROOT` file. This one-to-one correspondence makes the source of every shard clear, but it is not a complete copy of the `ROOT` file. The shards contains only the information required for the models in this study previously covered. The tensor files do not use the same optimized compression as `ROOT`. Their purpose is instead to provide data that can be loaded directly by `PyTorch`. Once the shards have been produced, they are self-contained for model training and evaluation. The original `ROOT` files are not read during training.

Each shard stores a schema version, its source information, the preprocessing settings and a list of events. Each selected entry in the source `ROOT TTree` becomes one tensor

event. A tensor event is a key–value collection in which names such as `x` and `y` refer to PyTorch tensors. Table IV summarizes the main stored information.

Table IV
NEEDS MORE WORK, OMITT ENTIRELY? ANSWER: YES, OMITT THIS ENTIRELY THE SAME INFORMATION WILL TRY TO BE GIVEN IN CONFIG/FIG/IO_AND_TENSOR.JPG MAING THIS IRRELEVANT MAIN CONTENTS OF ONE TENSOR EVENT. HERE $N = N_E + N_T$, WHERE N_E AND N_T ARE THE NUMBERS OF ECal AND TRIGGER-PAD ROWS. J IS THE NUMBER OF PHYSICAL-ORIGIN COLUMNS.

Information	Stored representation
Combined model input	<code>x</code> : float32 $[N \times 8]$ <code>ecal_mask</code> , <code>tpad_mask</code> : bool $[N]$
Detector-specific arrays	<code>ecal_pos</code> : float32 $[N_E \times 3]$ <code>tpad</code> : float32 $[N_T \times 2]$
Detector values	<code>ecal_input_energy</code> , <code>ecal_raw_energy</code> : float32 $[N_E]$ <code>tpad_raw_pe</code> : float32 $[N_T]$
Training targets	<code>y</code> , <code>physical_y</code> , <code>origin_id_y</code> : int64 $[N_E]$ <code>is_noise_target</code> : bool $[N_E]$ <code>fraction_target</code> : float32 $[N_E \times J]$
Event and source information	<code>event_idx</code> , <code>electron_count</code> , <code>source_label</code> , <code>source_file</code> , <code>source_entry</code> , <code>keep_indices</code> and <code>label maps</code>

When a shard is written, `physical_y` records the simulation-derived dominant-origin label of each retained ECal hit. Electron origins are labelled 1, 2, ..., while a retained noise hit receives 0. The field `y` stores the corresponding class index used by the training loss. Before a baseline model is trained, noise rows are removed and `y` is replaced by the canonical-`y` class described above. The field `origin_id_y` preserves the original overlay-origin identifier so that the canonical class can still be traced back to the simulated electron. For a retained noise hit, it stores a contributing origin **WHAT DOES THIS MEAN?** when this information is available and `-1` otherwise.

The event-level field `electron_count` records how many electrons were overlaid in the event. It is used as a target by the experimental count head and is not provided to the models as an input feature.

The detector-cell identifiers and the variable-length contribution lists are used while reconstructed hits are matched to simulation truth and the targets are constructed. They are not copied into the final tensor event. This does not limit training because all model inputs and targets are already present in the shard. It only means that the source ROOT file must be opened again if the original matching or additional simulation quantities need to be inspected. Together, `source_label`, `source_file` and `source_entry` identify the source event. The field `keep_indices` maps the retained ECal tensor rows to their original RecHit rows.

Two JSON files describe the complete tensor dataset. The `manifest.json` file records how the shards were produced, including the source files, accepted labels, noise treatment, input transformations, target definition and feature layout. The `index.json` file records where each shard is stored and which event range it contains. It also records the total number of events and any source files that were skipped.

During training, the loader reads a shard only when one of its events is requested. A bounded memory cache keeps a small number of recently used shards. The full dataset therefore does not need to fit in memory, and the ROOT-to-tensor conversion is not repeated for every training run. The compressed ROOT TTree remains the main simulation format [38], while the tensor shards form the task-specific working dataset used for machine learning.

3.5.4 Data Split and Training-Time Preparations

The event indices are shuffled with a fixed seed and divided into training, validation and test sets. The proportions are 80%, 10% and 10% respectively. Only the index lists are split, so the shards do not need to be rewritten. For a baseline model, noise rows are removed and the physical-origin labels are mapped to canonical- y classes when each event is loaded.

After the split, the six continuous columns (the column vector entries in the rows of the combined event tensor that are represented with float values in eq 18) are standardized. Their means and standard deviations are calculated from the training set only. The two detector-type columns with integer values are not standardized. The same training statistics are applied to the validation and test sets and saved with the model run.

Standardization is applied after the two detector types have been placed in the same tensor. A zero-filled field may therefore become non-zero after standardization. The type columns and masks still identify the detector type. The statistics are calculated from all ECal and trigger-pad rows before a model-specific view is selected. This means that the zero-filled fields take part in the calculation. The ECal-only view also uses the scaling fitted on the common mixed-detector tensor.

Parameter updates are calculated from the training split only. Validation loss controls checkpoint selection, early stopping and, where enabled, learning-rate scheduling, as detailed in the training procedure below. The test split is kept separate from these choices and is used for the final evaluation presented in the Results chapter.

3.6 Model Architectures

3.6.1 Capacity-Selection Strategy

Model capacity was selected with a performance-first strategy. The principal objective was to establish whether the proposed inputs and interaction mechanisms could achieve useful reconstruction performance, rather than to identify the smallest possible network. Widths and depths were therefore chosen conservatively from development runs so that insufficient capacity was unlikely to be the dominant limitation. The scale of contemporary Transformer-based MLPF implementations provided a qualitative reference for a reconstruction model with heterogeneous inputs, while the reconstruction task for LDMX specifically should be substantially simpler compared to the size of the hit collection this experiment generate compared to the CMS experiment [8].

This study did not either include a broad hyperparameter or architecture sweep, nor did it attempt pruning, quantisation, knowledge distillation or another model compression procedure. Thus, the reported parameter counts should not be interpreted as minimum capacities required by the reconstruction tasks. Establishing the accuracy–latency–memory trade-off is left to future outlook.

The architectures reported below are the configurations used for the million-event training campaigns found in the results. Separate $2e$ and $3e$ models were trained within each baseline family, but their architectures and hyperparameters are identical except for the width of the final classifier: $C = 2$ for the $2e$ task and $C = 3$ for the $3e$ task. Each baseline model used one million fixed-multiplicity events, whereas the contributor-set slot model used one million events balanced between the $2e$ and $3e$ sources not informing the model the multiplicity count beforehand.

3.6.2 GravNet Baseline

The GravNet baseline embeds every eight-feature ECal or trigger-pad node in a 128-dimensional latent space. Four residual GravNetConv blocks then learn event-local neighbourhoods and propagate information between nearby nodes in the learned space [6]. A linear per-node head produces the origin logits, of which only the ECal rows are supervised. The graph-assignment vector described in Section 3.7.2 prevents message passing between events.

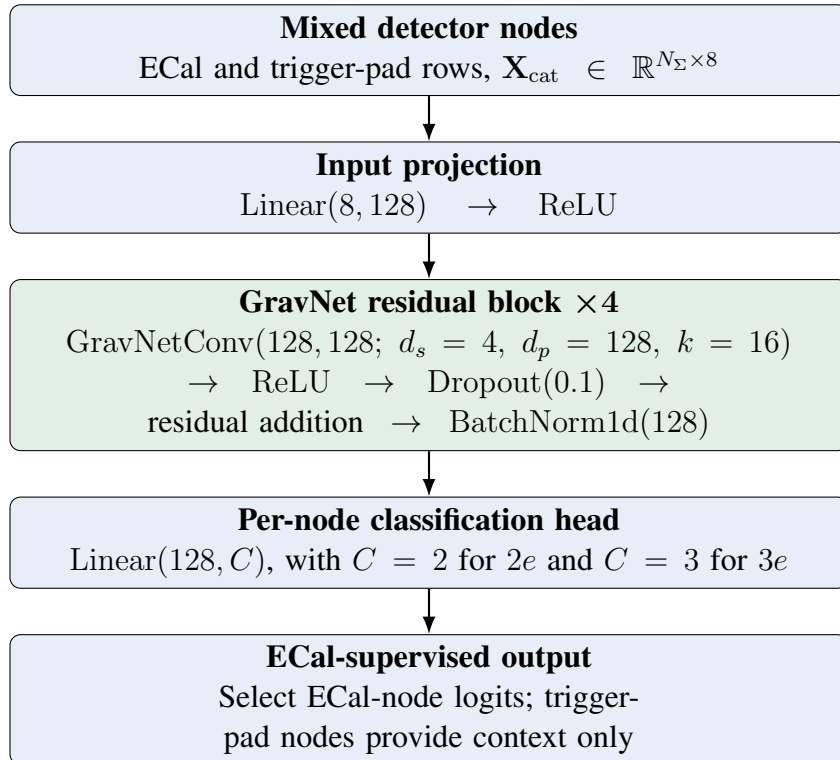


Figure 14. Architecture of the selected GravNet baseline. The learned-space dimension is $d_s = 4$, the propagated-feature dimension is $d_p = 128$, and each learned neighbourhood contains $k = 16$ nodes.

3.6.3 Transformer Baseline

The Transformer baseline maps the same eight input features to a 128-dimensional representation and applies three pre-layer-normalised encoder blocks. Within each event, full self-attention lets every valid token interact with every other valid token and the padding mask prevents padded rows from contributing, and the leading batch dimension is processed independently. A linear token head produces the ECal hit-origin logits. This model adds no sequence positional encoding and the detector coordinates are already included in the input features, and the row order carries no meaning. This set-to-set construction follows the interaction pattern used in recent MLPF models while retaining and restricting it to a single LDMX-specific output task [8].

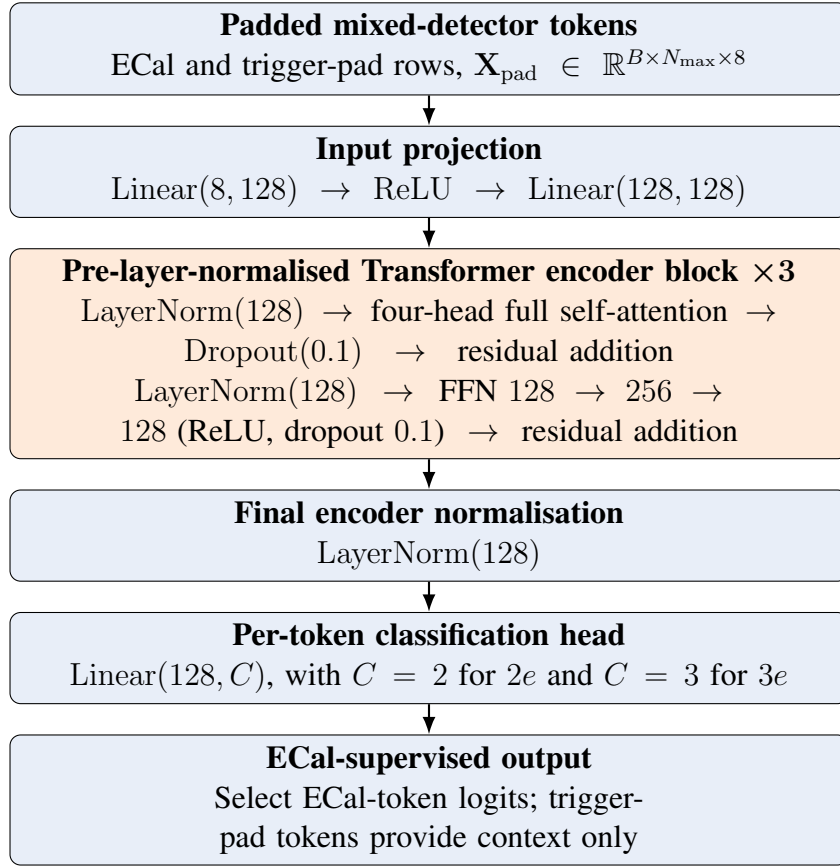


Figure 15. Architecture of the selected pre-layer-normalised Transformer baseline. Its model width is 128, the feed-forward width is 256, and its four attention heads each have dimension 32.

3.6.4 MLPF-Inspired Contributor-Set Slot Model

The proposed model uses one shared Transformer backbone for the mixed $2e/3e$ dataset. A learned detector-type embedding distinguishes ECal and trigger-pad tokens. Two ECal-supervised heads predict the subset of electron slots contributing to each hit and their energy fractions. In parallel, a mask-aware event summary predicts which of the three electron slots are valid. Coherent post-processing combines these outputs to derive the electron count, dominant hit origin and pure-versus-mixed classification and these

are not learned by separate heads. The three training losses—contributor-set, fraction and slot-validity—were assigned equal weight.

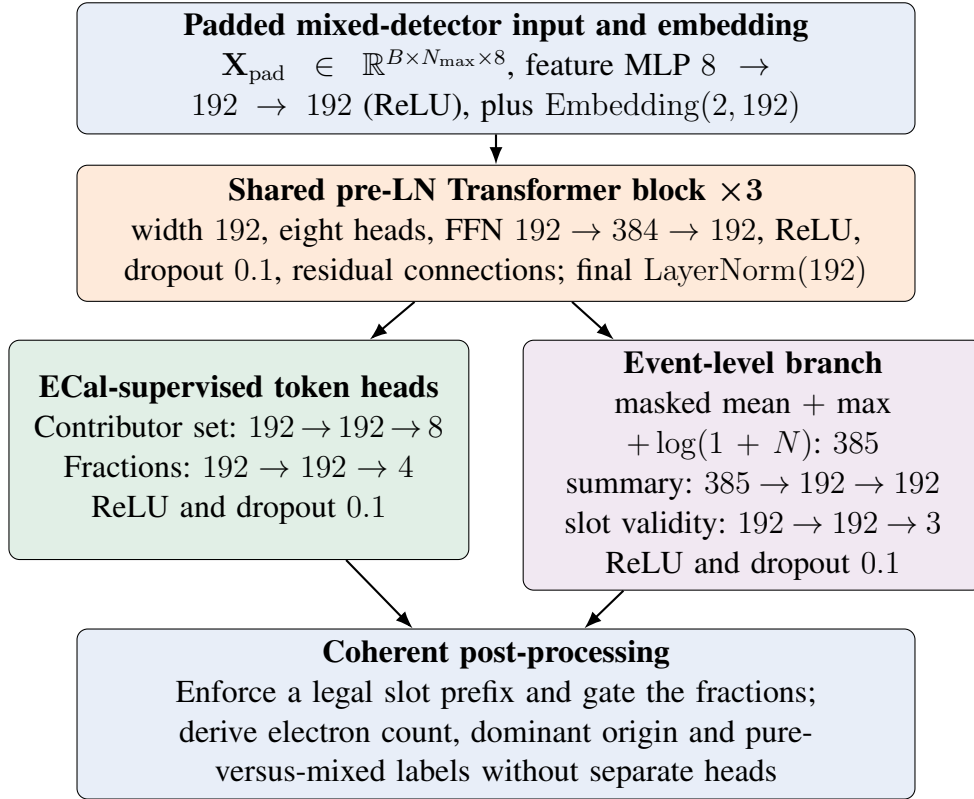


Figure 16. Architecture of the selected MLPF-inspired contributor-set slot model. The eight contributor-set classes encode all subsets of three electron slots; the four fraction components represent background and the three slots.

3.7 Training Procedure

I dont know if I should have this here or in methodology before model architectures...?
If it should be here move the content from methodology related to this here instead.

EDIT: I probably should have it here instead I guess!

3.7.1 Dataset splitting

I dont know if I should have this here or in methodology before model architectures...?
If it should be here move the content from methodology related to this here instead.

EDIT: I probably should have it here instead I guess!

3.7.2 Mini-batch construction for variable-sized events

Each mini-batch contains at most B events, where the batch size counts events rather than detector objects. For event b , the model-specific input is a variable-size matrix $\mathbf{X}_b \in \mathbb{R}^{n_b \times F}$. In the ECal-plus-trigger-pad models, trigger-pad rows are valid context objects but only ECal rows are supervised. Figure 17 shows how these event matrices are collated and processed.

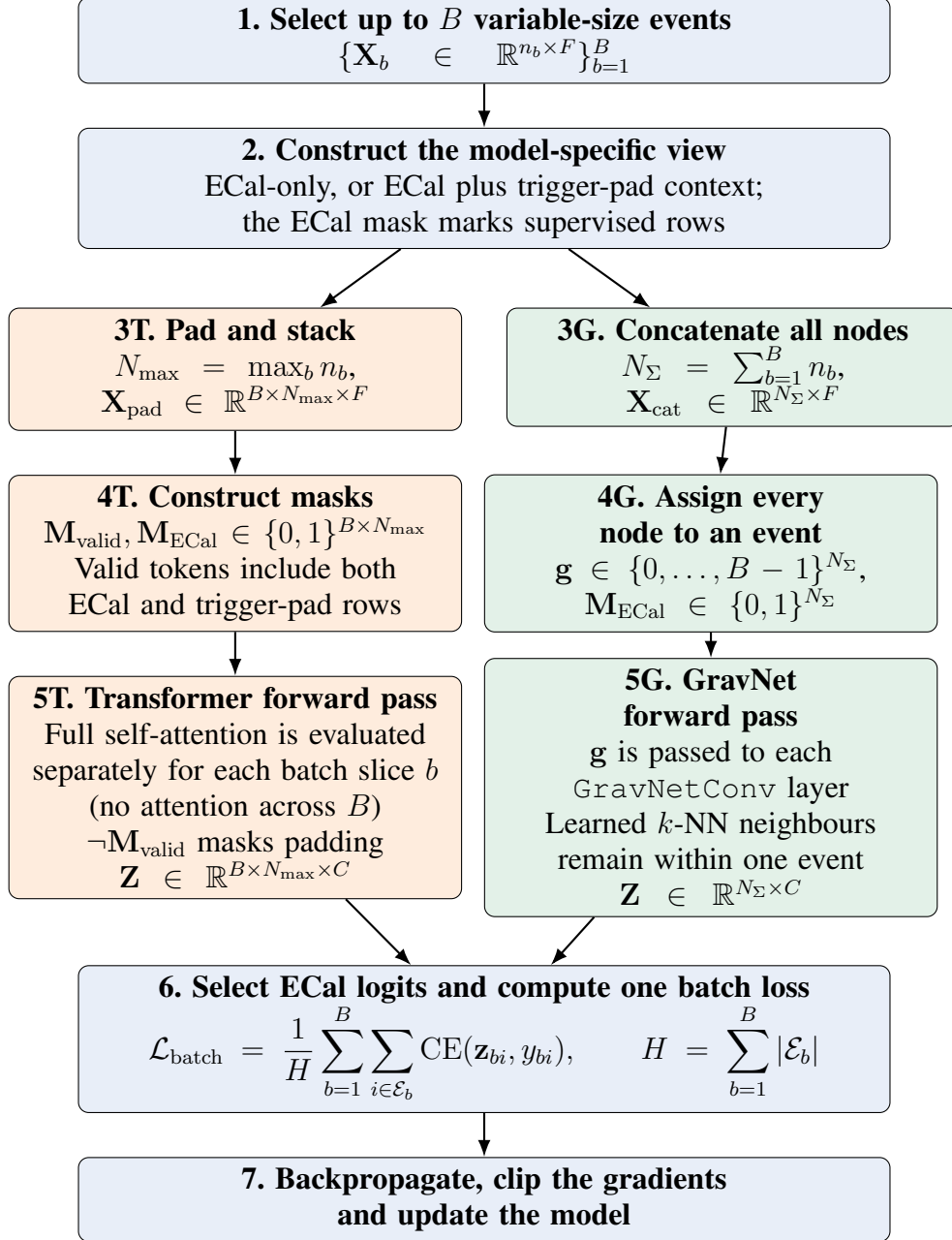


Figure 17. Mini-batch construction for the Transformer and GravNet baselines. Here F is the input-feature dimension, C is the number of output classes and $\mathcal{E}_b$ is the set of ECal rows in event b . Trigger-pad rows provide context but do not enter the hit-origin loss.

For the Transformer, the leading batch dimension indexes independent event sequences and is not part of the attention contraction. Full self-attention therefore acts only between tokens belonging to the same event, while the validity mask excludes padding tokens within that event. GravNet instead uses g to keep the learned neighbourhoods of the concatenated nodes event-specific. In both models, the ECal mask selects the supervised outputs and the loss is averaged over all H ECal hits in the mini-batch.

3.7.3 Optimizer and Regularization

All reported models were trained with AdamW, using an initial learning rate of 3×10^{-4} and a weight-decay coefficient of 10^{-4} . AdamW uses the adaptive first- and second-moment updates of Adam but decouples weight decay from the loss-gradient update. The regularisation is therefore applied directly to the parameters instead of being rescaled by the optimizer’s adaptive gradient statistics [45]. It was selected as a robust general-purpose optimizer for both the Transformer and GravNet campaigns. This was a pragmatic common choice, not the outcome of a dedicated optimizer comparison.

The reported configurations use dropout with probability 0.1 and clip the total gradient L^2 norm to at most 1.0 before each optimizer step. Weight decay and dropout regularise the model, while gradient clipping limits occasional large updates. The baseline runs retain the initial learning rate. For the multi-task slot model, validation-loss plateau scheduling is enabled with a patience of one epoch and a reduction factor of 0.5.

3.7.4 Early Stopping and Checkpoint Selection

The validation loss is evaluated after every training epoch and controls early stopping. The reported campaigns permit at most 15 epochs, require at least five, and stop after three consecutive epochs without a validation-loss improvement of at least 10^{-4} . The test split does not influence this decision.

Two related checkpoint criteria are retained deliberately. A resumable latest checkpoint and an epoch checkpoint are written after each epoch. Separately, the best checkpoint is replaced whenever the validation loss reaches a new strict minimum. The latter improvement may be smaller than the 10^{-4} threshold used to reset early-stopping patience. Thus, the best available parameters are preserved even when several improvements are too small to justify extending training. Interrupted runs can resume with the optimizer, learning-rate scheduler, data split and early-stopping history restored.

3.8 Training Objective and Optimization

For the baseline hit-origin classifiers, the loss is categorical cross-entropy evaluated only on retained ECal hits,

$$\mathcal{L}_{\text{origin}} = -\frac{1}{N_{\text{ECal}}} \sum_i \log p_i(y_i). \quad (20)$$

UPDATE THE MLPF INSPIRED MODEL TO HOW THE LOSS IS DEFINED NOW

For the mlpf-inspired multi-task model, the implemented objective is

$$\mathcal{L}_{\text{slot}} = \lambda_o \mathcal{L}_{\text{origin}} + \lambda_f \mathcal{L}_{\text{fraction}} + \lambda_s \mathcal{L}_{\text{valid}} + \lambda_c \mathcal{L}_{\text{count}}, \quad (21)$$

where $\mathcal{L}_{\text{valid}}$ is binary cross-entropy for the existence of each electron slot and $\mathcal{L}_{\text{count}}$ is categorical cross-entropy for the event-level electron multiplicity. The slot training code supports inverse-frequency class weights for the hit-origin and count terms, computed from the training split.

The weights $\lambda_o, \lambda_f, \lambda_s, \lambda_c$ are chosen by the user and allows adjusting the loss from the different supervised assignments.

3.9 Evaluation Methodology

Model selection and exploratory failure analysis are performed on the validation split. The test split remains held out for the final comparison of the selected models. Metrics are calculated both after pooling all retained ECal hits and separately for each event. The event-wise quantities are important in this problem because a single aggregate accuracy can conceal a broad distribution containing both easily separated and strongly overlapping showers.

3.9.1 Hit-origin classification metrics

Consider an event e with a set of retained ECal hits $\mathcal{H}_e$, where $N_e = |\mathcal{H}_e|$. Let y_{eh} and $\hat{y}_{eh}$ denote the target and predicted dominant-origin classes for hit h , respectively. The ordinary event hit accuracy is

$$A_{\text{hit}}^{(e)} = \frac{1}{N_e} \sum_{h \in \mathcal{H}_e} \mathbb{I}[\hat{y}_{eh} = y_{eh}], \quad (22)$$

where $\mathbb{I}[\cdot]$ is the indicator function. In addition to its mean, the distribution of $A_{\text{hit}}^{(e)}$ is retained so that its median, tails and individual failure cases can be inspected. Row-normalised confusion matrices show which origin classes are confused.

Misclassifying a low-energy hit and misclassifying a hit carrying a large amount of shower energy need not have equal physical importance. A parallel energy-weighted accuracy is therefore defined as

$$A_E^{(e)} = \frac{\sum_{h \in \mathcal{H}_e} E_{eh}^{\text{raw}} \mathbb{I}[\hat{y}_{eh} = y_{eh}]}{\sum_{h \in \mathcal{H}_e} E_{eh}^{\text{raw}}}, \quad (23)$$

where E_{eh}^{raw} is the non-transformed reconstructed ECal energy. Thus, the logarithmically transformed model input is not used as a physical weight. Equations (22) and (23) are reported in parallel rather than replacing one with the other.

The predicted class probabilities p_{ehj} are also used to calculate the maximum confidence $c_{eh} = \max_j p_{ehj}$ and the normalised entropy

$$H_{eh} = -\frac{1}{\log K} \sum_{j=1}^K p_{ehj} \log p_{ehj}, \quad (24)$$

where K is the number of classifier output classes. Their event means, together with the event loss and hit multiplicity, are used as supporting diagnostics of confident failure and event difficulty.

3.9.2 Projected shower geometry

To relate model performance to physical shower overlap, each ECal hit is projected along the beam direction onto $\mathbf{r}_{eh} = (x_{eh}, y_{eh})$. Two truth-membership definitions are evaluated. The default *any-contributor* definition is

$$m_{ehj}^{(\text{contrib})} = \mathbb{I}[f_{ehj} > 0], \quad (25)$$

where f_{ehj} is the truth energy fraction from electron origin j . Consequently, a reconstructed hit with contributions from two electrons is included once in the geometry of both showers. The magnitude of the fraction is deliberately not used as a weight. This preserves a binary geometric interpretation: the question is whether an origin contributed to a cell hit, not whether it supplied the largest contribution.

For comparison, the alternative *dominant-only* definition is

$$m_{ehj}^{(\text{dom})} = \mathbb{I}\left[j = \arg \max_k f_{ehk}\right]. \quad (26)$$

Each hit then belongs to exactly one shower. This variant quantifies how much the overlap diagnostic changes when non-dominant truth contributions are discarded; it is not the default geometry definition.

Let $q \in \{\text{contrib}, \text{dom}\}$ identify the membership definition and let $N_{ej}^{(q)} = \sum_h m_{ehj}^{(q)}$. Every included hit has equal geometric weight. The projected centroid and radial root-mean-square width of shower j are

$$\boldsymbol{\mu}_{ej}^{(q)} = \frac{1}{N_{ej}^{(q)}} \sum_{h \in \mathcal{H}_e} m_{ehj}^{(q)} \mathbf{r}_{eh}, \quad (27)$$

$$\left(s_{ej}^{(q)}\right)^2 = \frac{1}{N_{ej}^{(q)}} \sum_{h \in \mathcal{H}_e} m_{ehj}^{(q)} \left\| \mathbf{r}_{eh} - \boldsymbol{\mu}_{ej}^{(q)} \right\|_2^2. \quad (28)$$

The centroid distance and dimensionless normalised separation for a pair of showers are then

$$d_{e,ij}^{(q)} = \left\| \boldsymbol{\mu}_{ei}^{(q)} - \boldsymbol{\mu}_{ej}^{(q)} \right\|_2, \quad (29)$$

$$S_{e,ij}^{(q)} = \frac{d_{e,ij}^{(q)}}{\sqrt{\left(s_{ei}^{(q)}\right)^2 + \left(s_{ej}^{(q)}\right)^2}}. \quad (30)$$

A small value of $S_{e,ij}^{(q)}$ means that the centroid distance is small relative to the combined transverse shower widths and therefore indicates greater geometric overlap. A pair with zero combined width has no finite normalised separation and is omitted from summaries that require S .

For a two-electron event, the event separation is simply $S_e^{(q)} = S_{e,12}^{(q)}$. For $K_e \geq 2$, and in particular for a three-electron event, the primary and secondary event summaries are

$$S_{e,\min}^{(q)} = \min_{1 \leq i < j \leq K_e} S_{e,ij}^{(q)}, \quad (31)$$

$$S_{e,\text{mean}}^{(q)} = \frac{1}{\binom{K_e}{2}} \sum_{1 \leq i < j \leq K_e} S_{e,ij}^{(q)}. \quad (32)$$

For $K_e = 3$, the minimum is taken over the three pairs (1, 2), (1, 3) and (2, 3). It captures the most strongly overlapping pair and is therefore the main event-difficulty coordinate. The pair mean describes the overall separation and is retained as complementary information. Geometry uses hits from all ECal layers.

As a local ambiguity diagnostic, the dominant-only centroids are additionally used to define, for each hit,

$$M_{eh} = \min_{j \neq y_{eh}} \left\| \mathbf{r}_{eh} - \boldsymbol{\mu}_{ej}^{(\text{dom})} \right\|_2 - \left\| \mathbf{r}_{eh} - \boldsymbol{\mu}_{e,y_{eh}}^{(\text{dom})} \right\|_2. \quad (33)$$

A hit with $M_{eh} < 0$ lies closer to another shower centroid than to the centroid of its assigned origin. Both the fraction of such hits and its raw-energy-weighted counterpart are retained.

3.9.3 Difficulty profiles and event inspection

Events are ranked by event hit accuracy to select representative cases from the lower tail, near the median and from the best-performing tail. For each selected event, static and interactive three-dimensional displays compare the truth and predicted hit assignments. The interactive display preserves the detector z -coordinate and exposes hit-level energy, confidence and entropy, providing a direct way to inspect the configurations represented by the summary plots.

IS THIS STILL UPDATED OR DO WE NEED TO UPDATE AGAIN? WHAT IS THIS, EXPLAIN BETTER, REMOVE? For the multi-task fraction prediction, evaluation additionally reports

$$\text{MAE}_f = \frac{1}{N_e K_e} \sum_{h \in \mathcal{H}_e} \sum_{j=1}^{K_e} \left| \hat{f}_{ehj} - f_{ehj} \right|, \quad (34)$$

$$\text{MSE}_f = \frac{1}{N_e K_e} \sum_{h \in \mathcal{H}_e} \sum_{j=1}^{K_e} \left(\hat{f}_{ehj} - f_{ehj} \right)^2. \quad (35)$$

Predicted-against-target fraction plots complement these scalar errors. For the slot model, evaluation further records per-slot validity accuracy, exact slot-set accuracy, direct electron-count accuracy and an event-count confusion matrix, with results separated by true event

multiplicity where appropriate. The numerical outcomes of all diagnostics defined here are presented in the Results chapter.

3.9.4 Interactive 3D ECal Event Display

An interactive 3D event display of the ECal was made in order to be able to inspect hits in the ecal manually. The interactive event display uses the `INSERT HERE` package and the user can open an event with an `.html` link in a browser. In the browser there is a drop down menu where the user can select different filters such as true labels, predicted labels, confidence (logit) values, etc. The interactive event display is most meaningful when inspecting and comparing the worst performing events, median performing event and the best performing events. A screenshot of the interactive event display is shown in Figure ?? in the Appendix.

There is a high qualitative value to be able to simply "look" at data in an intuitive way, for several reasons, that is difficult to measure. The event display allowed the user during development to validate that the models seemed to work, i.e make smart predictions, it facilitated the process of finding potential failure modes by inspecting the worst-case events and it was crucial for the user to form hypotheses of what electron shower properties that made an event easy/difficult to classify the hits.

4 Results and Discussion

This chapter first characterises the simulated overlay events and the geometric shower-overlap problem. It then evaluates Transformer and GravNet baselines with fixed electron multiplicity before turning to the additional counting, contributor-set and energy-fraction tasks of the experimental model inspired by particle-flow reconstruction. Results and their immediate interpretation are presented together, while broader limitations and possible extensions are collected at the end of the chapter.

4.1 Event Characteristics and the Shower-Overlap Problem

Before comparing reconstruction models, this subsection establishes the detector, longitudinal and transverse scales of the problem that they must solve. Unless stated otherwise, the descriptive figures use deterministic cache prefixes containing 1,000,000 $2e$ events and 1,000,000 $3e$ events. These distributions characterise the simulated samples; they are not used to select a model or tune a decision threshold.

4.1.1 Detector Occupancy and Longitudinal Shower Development

Figure 18 shows the pooled transverse distribution of reconstructed ECal hits in detector coordinates. The highest occupancy and energy density occur around the beam region, although hits populate a much larger fraction of the instrumented area. The discrete pattern of populated bins follows the hexagonal cell geometry and should not be interpreted as a continuous image. Increasing the multiplicity from $2e$ to $3e$ raises both the occupancy and the summed energy density while preserving the detector-scale structure. On average, the selected $2e$ events contain 247.2 reconstructed hits and 16.01 GeV of reconstructed energy, compared with 354.6 hits and 24.02 GeV for $3e$.

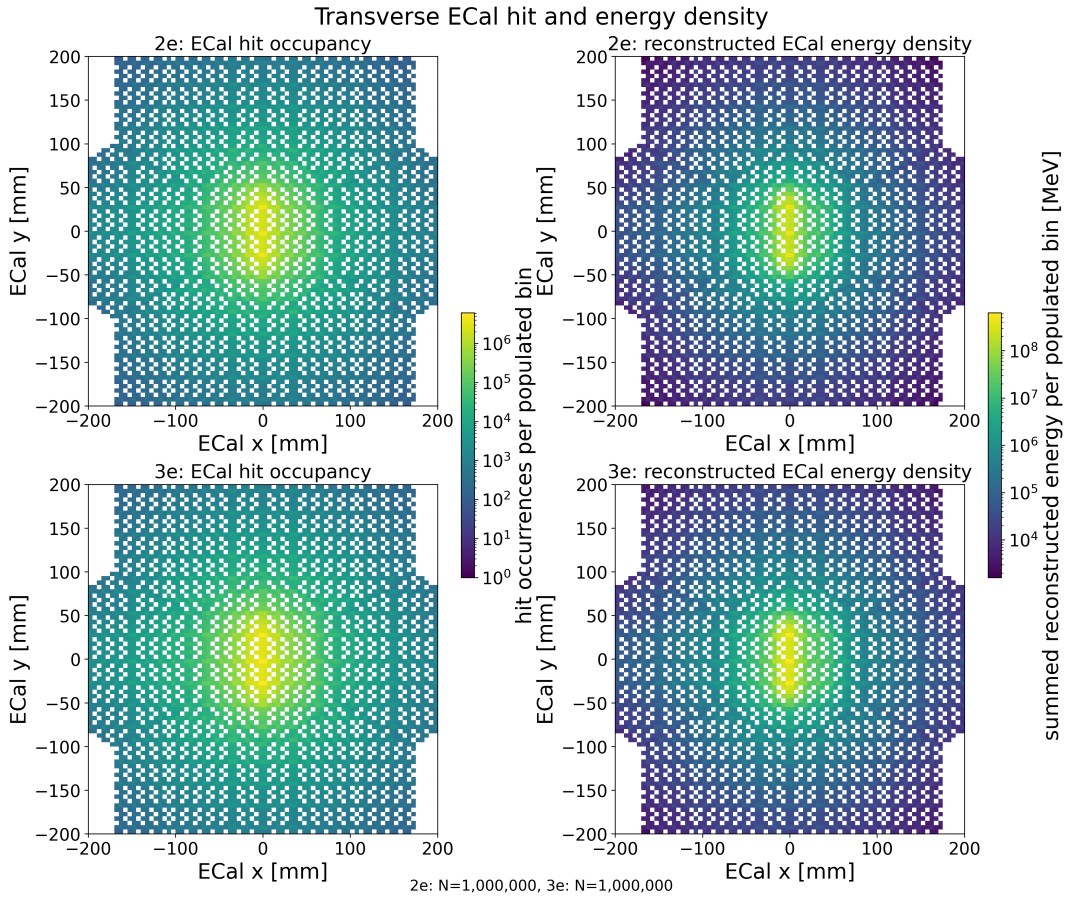


Figure 18. Transverse ECal hit occupancy (left) and summed reconstructed energy density (right) for 1,000,000 $2e$ events (top) and 1,000,000 $3e$ events (bottom). The histograms use the detector x - y coordinates and logarithmic colour scales. White regions are unpopulated spatial bins.

The same showers are resolved in depth in Figure 19. For both multiplicities, the mean hit count is largest at ECal layer 9, whereas the mean reconstructed energy reaches its maximum at layer 8. These positions correspond to $z \simeq 362$ mm and $z \simeq 343$ mm, respectively. Both quantities subsequently decrease through a long tail in the calorimeter. The $3e$ profiles have larger amplitudes but nearly the same longitudinal shape as the $2e$ profiles. Adding an electron therefore primarily increases the number and energy of simultaneous deposits rather than shifting the characteristic shower maximum. The sizeable event-to-event RMS values near the peak also demonstrate that the mean profile is not representative of every individual event.

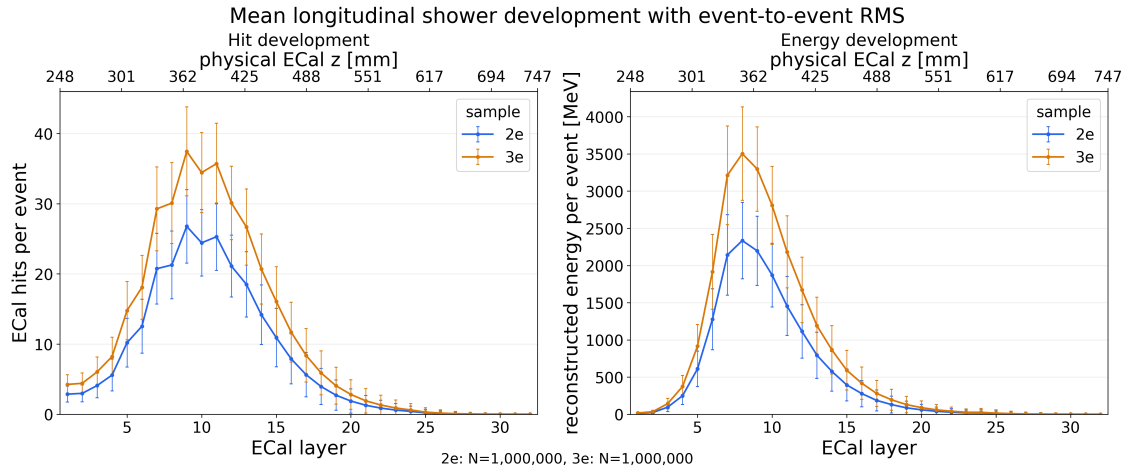


Figure 19. Mean longitudinal ECal hit multiplicity (left) and reconstructed energy (right) per event. Error bars show the event-to-event population RMS at each layer, not the uncertainty on the mean. The lower horizontal axes give ECal layer number and the upper axes give the corresponding physical z coordinate. Each profile contains 1,000,000 events.

4.1.2 Truth-Level Transverse Shower Geometry

The extent of shower overlap can be described using simulation truth. For this descriptive study, hit h contributes to the geometry of electron origin j with weight $w_{hj} = E_h f_{hj}$, where E_h is the reconstructed hit energy and f_{hj} is the simulated fraction supplied by that origin. Centroids and widths are consequently energy-weighted without forcing a mixed hit to belong entirely to one electron. These truth-assisted quantities are unavailable in measured detector data and are used only to characterise the simulated sample and interpret model performance.

Figure 20 reports the closest pair of resulting shower centroids in each event, resolved separately along the two transverse coordinates. The effective Molière radius $R_M = 25$ mm provides the reference scale. For $2e$ events, the median minimum separation is $0.253R_M$ (6.3 mm) in x and $0.946R_M$ (23.7 mm) in y . The corresponding $3e$ medians are $0.093R_M$ (2.3 mm) and $0.334R_M$ (8.3 mm). The difference between the x and y distributions is a property of the generated sample rather than a universal detector-resolution limit. Moreover, a $3e$ event contains three distinct electron pairs instead of one. Taking the closest pair therefore shifts the minimum-separation distribution towards smaller values even if the distribution for a single pair were unchanged.

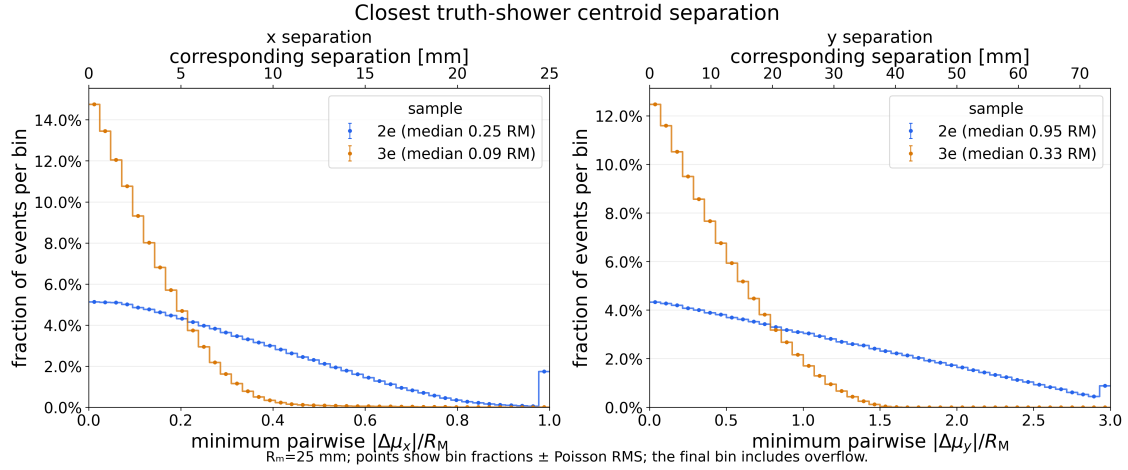


Figure 20. Minimum pairwise separation of energy-fraction-weighted truth shower centroids along x (left) and y (right). The lower axes use units of $R_M = 25$ mm, while the upper axes show the corresponding physical distance. Points and vertical bars show bin fractions and their Poisson RMS counting uncertainties. Values beyond the plotted range are collected in the final bin. Each distribution contains 1,000,000 events.

Figure 21 separates the scale of an individual shower from the distance between showers. The median transverse widths are almost independent of event multiplicity: for both samples, $\sigma_x \simeq 0.67R_M$, $\sigma_y \simeq 0.65R_M$ and $\sigma_r \simeq 0.96R_M$. In contrast, the median minimum width-normalised separation

$$S_{ij} = \frac{d_{ij}}{\sqrt{\sigma_{r,i}^2 + \sigma_{r,j}^2}}, \quad (36)$$

decreases from 0.724 for $2e$ to 0.353 for $3e$. Here, d_{ij} is the transverse centroid

distance and $\sigma_{r,i}$ is the radial width of shower i . The individual showers therefore have similar transverse dimensions in the two samples, but the closest competing showers occupy a substantially more overlapping regime when a third electron is present. This provides a physical explanation for expecting the $3e$ hit-assignment task to be more difficult.

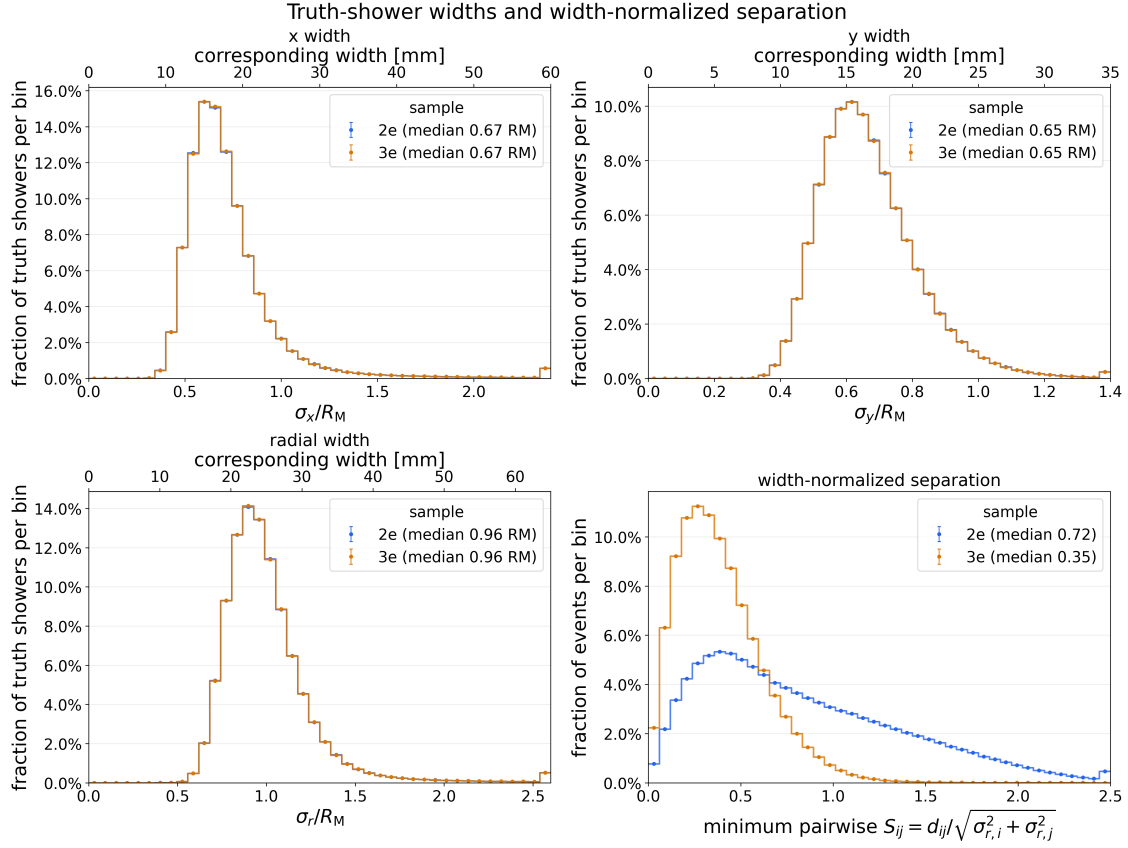


Figure 21. Energy-fraction-weighted transverse truth-shower widths in x , y and radius, together with the event minimum of the width-normalised centroid separation defined in Equation (36). The three width panels include secondary physical-distance axes. Points and vertical bars show bin fractions and Poisson RMS counting uncertainties; final bins include overflow. The $2e$ and $3e$ samples each contain 1,000,000 events.

4.1.3 Trigger-Pad Multiplicity as Event-Level Context

The TPad tracks provide a small set of upstream position and photoelectron measurements in addition to the ECal hits. Their token multiplicity is already an informative electron-count proxy, but it is not a complete one. Figure 22 shows that the observed multiplicity equals the generated count in 85.7% of $2e$ events and 74.0% of $3e$ events. Most of the remaining $2e$ events contain one token (13.9%), while 23.7% of the $3e$ events contain only two. A further 2.2% of the $3e$ sample contains only one token, whereas overcounting is negligible for both multiplicities. Across the balanced $2e/3e$ sample, direct equality gives 79.9% electron-count accuracy. The multiplicity therefore carries useful global information, but missing, merged or otherwise unresolved tracks prevent it from determining the electron count by itself. This motivates testing whether a model can combine the TPad observations with the ECal shower pattern.

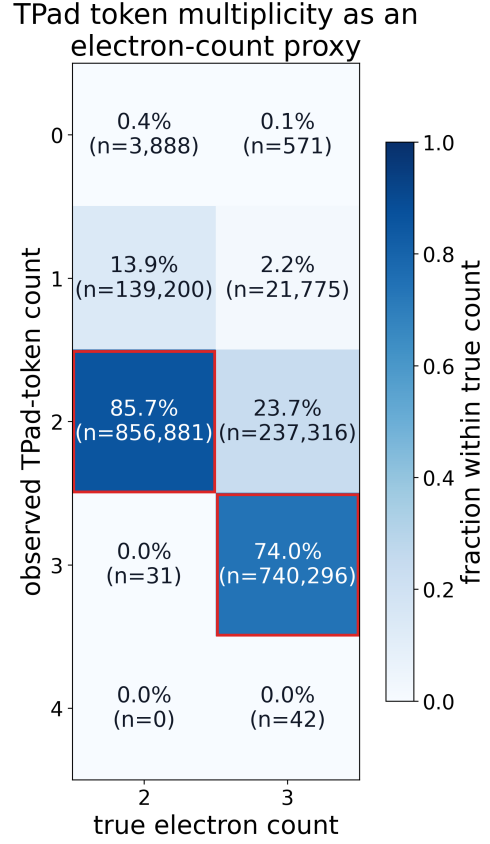


Figure 22. Confusion matrix with observed TPad-track token multiplicity vs true electron multiplicity. Each column contains 1,000,000 events. Red outlines show agreement between the token and generated counts.

4.1.4 Permutation Invariance of Electron Shower Labels

Move this and combine into one subsection in methodology it is OK that this "result" is presented there already

The physical reconstruction objective that is of value to LDMX is to partition deposits into electron showers and these does not possess any intrinsic ordering. Canonical ordering by the truth y -centroid supplies stable class targets during training, but a prediction that makes a consistent grouping but exchanges two complete shower labels (i.e a permutation) still represents the same physical partition. The permutation-invariant accuracy metric swaps the predicted labels globally and compares against the true labels and chooses the permutation that yields the highest accuracy, since this represents the reconstruction objective correctly.

Figure 23 verifies the practical effect of this convention for 3,000 $3e$ validation events. A label permutation changes the score for 297 events (9.9%); the median gain is zero, while the mean event accuracy increases by 1.50 percentage points, from 0.722 to 0.737. Most events lie on the diagonal, but a smaller set lies well above it and would otherwise be penalised for a globally exchanged label rather than a failure to separate the showers. Unless explicitly stated otherwise, the event-level hit-assignment results below consequently use this permutation-aligned definition. This is the only model-dependent diagnostic in the present event-characterisation subsection.

Taken together, the event statistics establish the expectations used to interpret the model results. The individual shower widths change little from $2e$ to $3e$, whereas the closest-pair separation decreases substantially for $3e$; the additional difficulty should therefore arise mainly from overlap rather than broader individual showers. The TPad multiplicity supplies useful but incomplete event-level context, and the permutation study confirms that shower separation must be evaluated as an unordered partition rather than as an absolute class numbering.

THE REST OF THE RESULTS SECTION WILL INCLUDE ALL THE RESULTS AND ALSO DISCUSS THEM IN A NATURAL WAY AS WE GO ALONG I WANT TO DIVIDE IT INTO SUBSECTIONS THAT FIRST COVER THE BASELINE MODELS AND INVESTIGATE THE SO-CALLED PRINCIPAL TASK OF THIS THESIS AND STUDY OF ELECTRON SHOWER SEPARATION IN THE FORM OF HIT ASSIGNMENTS. ONE SUBSUBSECTION CAN NATURALLY LEAD INTO THE OTHER. ONCE THE BASELINE MODELS HAVE

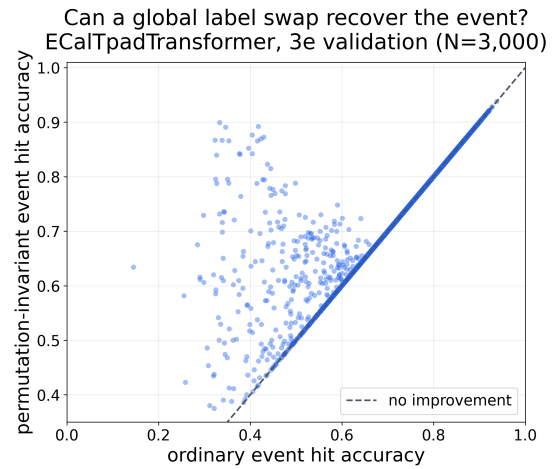


Figure 23. Ordinary canonical-label hit accuracy compared with accuracy after the best single global permutation of the predicted shower labels, evaluated on 3,000 $3e$ validation events from the Transformer development run. The dashed diagonal denotes no change. A global permutation can only preserve or increase the event score.

BEEN COVERED I HAVE ALREADY INTRODUCED MANY TYPES OF ANALYSES AND PLOTS AND I CAN REUSE ARGUMENTS AND I CAN UTILIZE THE FACT THAT STUFF HAS ALREADY BEEN EXPLAINED AND CONCEPTS DISCUSSED WHEN I GO INTO THE NEXT SECTION THE SECTION AFTER THE BASELINE MODEL SECTION IS COVERING THE MLPF INSPIRED MULTI TASK SLOT TRANSFORMER MODEL. HERE ALL THE SUBSECTIONS WILL PRESENT THE RESULTS THAT ARE RELATED TO THIS MODEL AND THIS MODEL HAS NEW TASKS THAT THE BASELINE MODELS DOES NOT HAVE SO I HAVE TO GO THROUGH THESE AND DISCUSS AND EXPLAIN/COMMENT THE RESULTS (AND NOT EXPLAIN STUFF THAT I HAVE ALREADY EXPLAINED IN THE METHODOLOGY OF COURSE). I CAN USE THE SAME ANALYSIS PLOTS THAT I USED FOR THE HIT ASSIGNMENTS FOR THIS MODEL AS WELL AND INVESTIGATE IF THEY ARE SIMILAR OR DIFFERENT AND DISCUSS WHY THAT MIGHT BE THE CASE.

4.2 Baseline Models

4.2.1 Baseline Architecture Comparison

The headline baseline comparison uses 1,000,000 selected events for each electron multiplicity: 800,000 training, 100,000 validation and 100,000 test events. Both architectures receive the same log-transformed ECal and trigger-pad-track features and use a hidden width of 128. The Transformer contains three pre-layer-normalised self-attention layers with four attention heads and a final layer normalisation. The GravNet model contains four neighbourhood aggregation blocks, each using $k = 16$ neighbours, with batch normalisation between blocks. Table VIII summarises the resulting model sizes and observed training times.

Table V
ARCHITECTURE AND PRACTICAL PROPERTIES OF THE MATCHED 1,000,000-EVENT BASELINE RUNS.
PARAMETER STORAGE ASSUMES 32-BIT FLOATING-POINT VALUES, WHEREAS THE CHECKPOINT SIZE ALSO
INCLUDES OPTIMISER AND TRAINING STATE. TRAINING TIME IS THE MEAN OPTIMISATION TIME PER EPOCH
AND EXCLUDES VALIDATION.

Model	Core configuration	Parameters	Weights (MB)	Checkpoint (MB)	Batch size	Train time/epoch $2e/3e$ (min)
Transformer	3 layers, 4 heads, $d = 128$	0.416 M	1.66	9.92	64	38.3/40.1
GravNet	4 blocks, $k = 16$, $d = 128$	0.268 M	1.07	8.15	32	46.8/49.7

As described in Section 3.6.1, these models were selected as performance-oriented reference configurations rather than through an exhaustive capacity search. The comparison below therefore evaluates the implemented architectures, not an optimal accuracy–parameter or accuracy–latency frontier. In particular, the use of contemporary MLPF models as a qualitative scale reference does not imply matched capacity: the CMS particle-flow problem has different inputs, outputs and reconstruction scope. No pruning or other compression study was performed.

The GravNet baseline has approximately 36% fewer trainable parameters and correspondingly smaller model weights and checkpoints. In these runs, however, this did not translate into shorter epochs: Transformer optimisation took approximately 38–40 minutes per epoch, compared with 47–50 minutes for GravNet. The learned neighbour search and aggregation are more expensive than the parameter count alone suggests. This timing comparison remains descriptive rather than controlled because the Transformer used a batch size of 64 and GravNet used 32.

Table IX shows that both architectures reach very similar shower-assignment performance. The $3e$ task is consistently harder than the $2e$ task, independent of architecture. Energy weighting increases all four accuracies, which indicates that misassigned hits tend to carry less energy than correctly assigned hits.

Because both models were evaluated on the same validation events, their event-wise results can be compared directly. Table X reports paired bootstrap intervals for this comparison. The Transformer has a 0.223-percentage-point advantage in mean $2e$ event accuracy and a 0.445-percentage-point advantage for $3e$. The corresponding energy-weighted advantages are 0.203 and 0.510 percentage points. All four intervals exclude

Table VI

IS THIS THE ENERGY WEIGHTED MEDIAN OR ENERGY WEIGHTED MEAN? PRELIMINARY PERFORMANCE OF THE MATCHED BASELINE MODELS. POOLED HIT ACCURACY IS EVALUATED ON THE TEST SET. THE EVENT-BALANCED MEAN, MEDIAN AND ENERGY-WEIGHTED ACCURACY ARE EVALUATED ON THE CORRESPONDING 100,000-EVENT VALIDATION SETS. ALL VALUES ARE PERCENTAGES.

Multiplicity	Model	Pooled test	Mean event	Median event	Energy-weighted
2e	Transformer	83.31	82.85	85.96	88.36
2e	GravNet	83.09	82.63	85.81	88.15
3e	Transformer	73.11	72.58	75.54	80.30
3e	GravNet	72.63	72.13	74.93	79.79

zero. The large event samples therefore resolve a consistent Transformer advantage for these trained models, but its magnitude remains approximately half a percentage point or less. Moreover, the confidence intervals describe finite-event uncertainty for fixed trained models; they do not include variation between random initialisations or training samples. The results support similar practical performance with a small advantage to the Transformer in this campaign, not a universal ranking of the two architectures.

Table VII

PAIRED TRANSFORMER-MINUS-GRAVNET DIFFERENCES ON THE COMMON 100,000-EVENT VALIDATION SAMPLES. DIFFERENCES AND PERCENTILE-BOOTSTRAP 95 % CONFIDENCE INTERVALS ARE IN PERCENTAGE POINTS. THE BOOTSTRAP RESAMPLES COMPLETE EVENTS.

Multiplicity	Event metric	Difference	95 % CI
2e	Hit accuracy	+0.223	[+0.207, +0.238]
2e	Energy-weighted accuracy	+0.203	[+0.181, +0.224]
3e	Hit accuracy	+0.445	[+0.425, +0.466]
3e	Energy-weighted accuracy	+0.510	[+0.481, +0.540]

Since the two baselines behave similarly and the Transformer gives the higher headline accuracy, the following detailed diagnostic plots use it as the representative fixed-multiplicity model instead of duplicating every plot for GravNet. These diagnostics were produced from the earlier 20,000-event development runs. They are retained to examine where assignment fails, while Tables IX and X provide the headline comparison of the larger matched runs.

4.3 Baseline Models

4.3.1 Baseline Architecture Comparison

The headline baseline comparison uses 1,000,000 selected events for each electron multiplicity: 800,000 training, 100,000 validation and 100,000 test events. Both architectures receive the same log-transformed ECal and trigger-pad-track features and use a hidden width of 128. The Transformer contains three pre-layer-normalised self-attention layers with four attention heads and a final layer normalisation. The GravNet model contains four neighbourhood aggregation blocks, each using $k = 16$ neighbours, with batch normalisation between blocks. Table VIII summarises the resulting model sizes and observed training times.

Table VIII

ARCHITECTURE AND PRACTICAL PROPERTIES OF THE MATCHED 1,000,000-EVENT BASELINE RUNS. PARAMETER STORAGE ASSUMES 32-BIT FLOATING-POINT VALUES, WHEREAS THE CHECKPOINT SIZE ALSO INCLUDES OPTIMISER AND TRAINING STATE. TRAINING TIME IS THE MEAN OPTIMISATION TIME PER EPOCH AND EXCLUDES VALIDATION.

Model	Core configuration	Parameters	Weights (MB)	Checkpoint (MB)	Batch size	Train time/epoch $2e/3e$ (min)
Transformer	3 layers, 4 heads, $d = 128$	0.416 M	1.66	9.92	64	38.3/40.1
GravNet	4 blocks, $k = 16$, $d = 128$	0.268 M	1.07	8.15	32	46.8/49.7

The GravNet baseline has approximately 36% fewer trainable parameters and correspondingly smaller model weights and checkpoints. In these runs, however, this did not translate into shorter epochs: Transformer optimisation took approximately 38–40 minutes per epoch, compared with 47–50 minutes for GravNet. The learned neighbour search and aggregation are more expensive than the parameter count alone suggests. This timing comparison remains descriptive rather than controlled because the Transformer used a batch size of 64 and GravNet used 32.

Table IX shows that both architectures reach very similar shower-assignment performance. The $3e$ task is consistently harder than the $2e$ task, independent of architecture. Energy weighting increases all four accuracies, which indicates that misassigned hits tend to carry less energy than correctly assigned hits.

Table IX

IS THIS THE ENERGY WEIGHTED MEDIAN OR ENERGY WEIGHTED MEAN? PRELIMINARY PERFORMANCE OF THE MATCHED BASELINE MODELS. POOLED HIT ACCURACY IS EVALUATED ON THE TEST SET. THE EVENT-BALANCED MEAN, MEDIAN AND ENERGY-WEIGHTED ACCURACY ARE EVALUATED ON THE CORRESPONDING 100,000-EVENT VALIDATION SETS. ALL VALUES ARE PERCENTAGES.

Multiplicity	Model	Pooled test	Mean event	Median event	Energy-weighted
$2e$	Transformer	83.31	82.85	85.96	88.36
$2e$	GravNet	83.09	82.63	85.81	88.15
$3e$	Transformer	73.11	72.58	75.54	80.30
$3e$	GravNet	72.63	72.13	74.93	79.79

Because both models were evaluated on the same validation events, their event-wise results can be compared directly. Table X reports paired bootstrap intervals for this comparison. The Transformer has a 0.223-percentage-point advantage in mean $2e$ event accuracy and a 0.445-percentage-point advantage for $3e$. The corresponding energy-weighted advantages are 0.203 and 0.510 percentage points. All four intervals exclude zero. The large event samples therefore resolve a consistent Transformer advantage for these trained models, but its magnitude remains approximately half a percentage point or less. Moreover, the confidence intervals describe finite-event uncertainty for fixed trained models; they do not include variation between random initialisations or training samples. The results support similar practical performance with a small advantage to the Transformer in this campaign, not a universal ranking of the two architectures.

Since the two baselines behave similarly and the Transformer gives the higher head-line accuracy, the following detailed diagnostic plots use it as the representative fixed-

Table X
 PAIRED TRANSFORMER-MINUS-GRAVNET DIFFERENCES ON THE COMMON 100,000-EVENT VALIDATION
 SAMPLES. DIFFERENCES AND PERCENTILE-BOOTSTRAP 95 % CONFIDENCE INTERVALS ARE IN PERCENTAGE
 POINTS. THE BOOTSTRAP RESAMPLES COMPLETE EVENTS.

Multiplicity	Event metric	Difference	95 % CI
$2e$	Hit accuracy	+0.223	[+0.207, +0.238]
$2e$	Energy-weighted accuracy	+0.203	[+0.181, +0.224]
$3e$	Hit accuracy	+0.445	[+0.425, +0.466]
$3e$	Energy-weighted accuracy	+0.510	[+0.481, +0.540]

multiplicity model instead of duplicating every plot for GravNet. These diagnostics were produced from the earlier 20,000-event development runs. They are retained to examine where assignment fails, while Tables IX and X provide the headline comparison of the larger matched runs.

4.3.2 Representative Training Behaviour

Figure 24 shows the training history of the $2e$ pre-layer-normalised Transformer trained on 1,000,000 events. Of these, 800,000 were used for training and 100,000 for validation, in line with the split 80/10/10. Each displayed point follows one complete epoch and therefore does not include the untrained model. The presented training behavior is representative for all models that perform the principal hit assignment task.

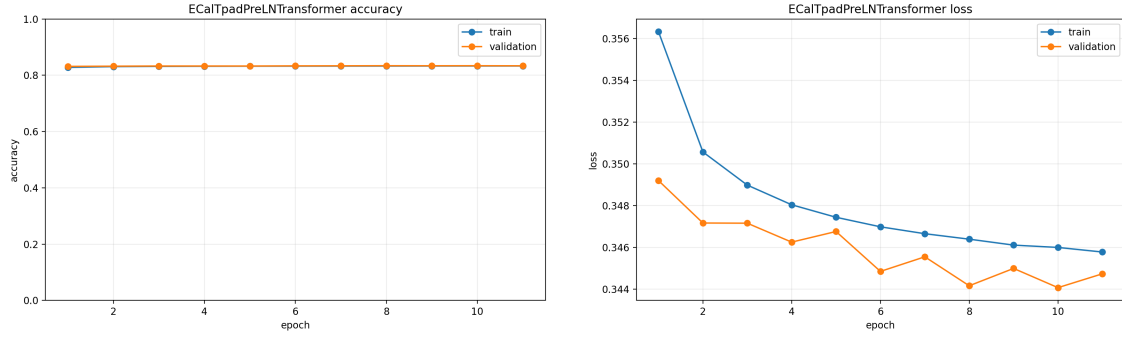


Figure 24. Training history for the 1,000,000-event $2e$ pre-layer-normalised Transformer. The left panel shows pooled native-label hit accuracy and the right panel shows cross-entropy loss for the 800,000-event training and 100,000-event validation sets. Validation-loss early stopping ended training after epoch 11 and the best validation loss was extracted from epoch 10.

The native-label accuracy is already 0.828 for training and 0.832 for validation after the first epoch. By epoch 11, the training accuracy has increased to 0.833, while the highest validation accuracy is 0.834 at epoch 8. The accuracy curve is clearly almost flat on the displayed scale. The loss gives a more sensitive view of the remaining optimisation: the training loss decreases from 0.3563 to 0.3458, a change of only 0.0106, and the validation loss decreases from 0.3492 to its minimum of 0.3441 at epoch 10. The small subsequent change in accuracy shows that most class decisions made by the final model were already established after the first recorded epoch and later epochs mainly adjusted the predicted class probabilities.

Early stopping monitored validation loss, with a minimum of five epochs, a patience of three epochs and a minimum improvement of 10^{-4} . The run stopped after epoch 11 rather than reaching the maximum of 15 epochs, and the epoch-10 parameters were retained as the best checkpoint. The validation loss is slightly lower than the training loss throughout. This is consistent with dropout being active while training metrics are accumulated, but disabled for validation, which is evaluated once after each epoch with fixed parameters. Smaller development runs of about 10,000 events (also with splitting 80/10/10) showed similar qualitative behavior and this could be utilized during development, but for the final reported models the larger dataset of 1,000,000 was used instead to expose the model to a sophisticated size of data.

4.3.3 Hit Assignment Performance

THIS SUBSECTION SHOULD COVER THE OVERALL ACCURACIES AND PERFORMANCES OF THE BASELINE MODELS FOR THE PRINCIPAL TASK OF HIT ASSIGNMENTS (OR NODE ASSIGNMENTS AS IT IS CALLED GENERALLY IN MACHINE LEARNING). IT PRESENTS AN EVENT-AVERAGED (PERMUTATION INVARIANT) ACCURACY FOR THE 2E AND 3E MODELS THAT IS RELEVANT TO THE LDMX WHERE CAN SEE THAT THE DISTRIBUTION IS WIDE AND WHERE WE CAN SEE WHAT THE MOST COMMON EVENT WIDE ACCURACY PERCENTAGE IS AND WHERE WE SEE CLEARLY THAT THE 2E EVENTS ARE EASIER TO ASSIGN AND THAT THE 3E EVENTS ARE MORE DIFFICULT TO ASSIGN. WE PRESENT ALSO HIT CONFUSION MATRICES THAT SHOW **POOLED**** HIT ACCURACIES AND ALSO 4X4 CONFUSION MATRICES THAT IMPLY THAT THE HIT ACCURACY SEEMS TO BE LOWER DEEPER INTO THE ECAL LAYERS (BUT ALSO THAT MOST HITS ARE FOUND IN THE FIRST 20 ECAL LAYERS SINCE THAT ACCURACY IS ABOUT THE SAME AS THE ACCURACY ACROSS ALL ECAL LAYERS). THE ECAL LAYERS 1-20 HAS BEEN CHOSEN SPECIFICALLY BECAUSE THESE ARE USED FOR THE TRIGGER-LEVEL ONLINE RECONSTRUCTION, BUT OF COURSE IT THIS SCENARIO THE LOWER GRANULARITY TRIGGER CELLS ARE USED INSTEAD OF THE FULL RESOLUTION CELLS THAT WE HAVE NOW. THE ECAL LAYER POOLED HIT CONFUSION MATRICES LEAD PERFECTLY INTO THE NEXT SUBSECTION WHERE WE SHOW ACCURACY ACROSS ECAL DEPTH**

The event-wise distributions in Figure 25 show that reconstruction quality varies substantially between events. The median permutation-aligned event hit accuracy is 0.854 for $2e$ and 0.757 for $3e$ events. The corresponding event means are 0.829 and 0.744, while the pooled hit accuracies are 0.833 and 0.748. The additional electron therefore makes the assignment task more difficult, but the broad distributions also show that electron multiplicity alone does not determine whether an event is reconstructed well.

Figure 26 separates the same result by true and predicted electron group. For $2e$, the two groups have similar overall recall. For $3e$, the middle canonical group is confused more frequently with both neighbouring groups than either outer group. The decrease in the last twelve ECal layers is visible for both multiplicities: pooled accuracy is 0.693 for $2e$ and 0.556 for $3e$, compared with 0.836 and 0.751 in layers 1–20. The last-layer group contains only about 1.5 % of the hits in each sample and should not be given the same statistical weight as the all-layer result.

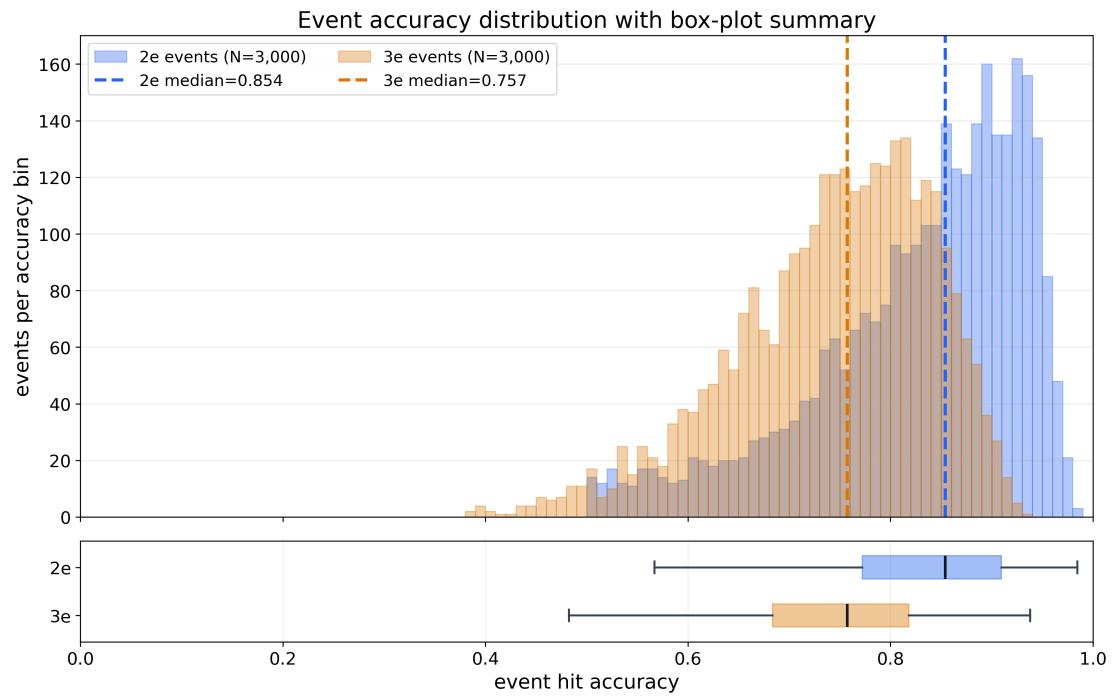


Figure 25. Distribution of permutation-aligned event hit accuracy for 3,000 $2e$ and 3,000 $3e$ validation events. Every event contributes equally to the histogram and box-plot summary. Dashed lines indicate the sample medians.

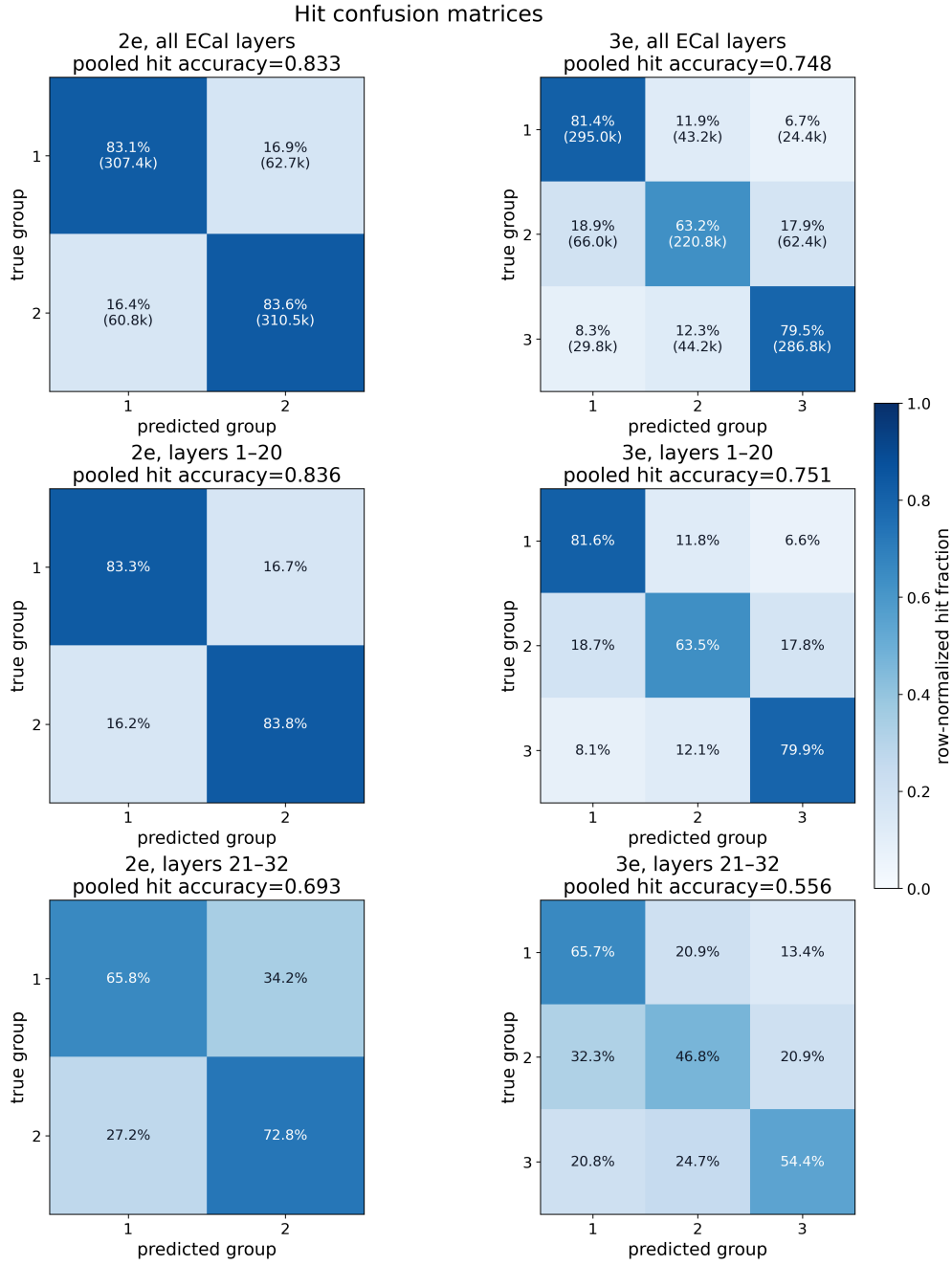


Figure 26. **WE SHOULD HAVE AN ENERGY-WEIGHTED CONFUSION MATRIX CORRESPONDING TO THIS AS WELL?** Row-normalized hit confusion matrices after one event-wide label alignment. Rows are true groups and columns are predicted groups. The top row uses all ECal layers, the middle row uses layers 1–20 and the bottom row uses layers 21–32. Percentages are conditional on the true group, and counts are pooled over validation hits.

4.3.4 Performance Across the Electron Shower Depth

Figures 27 and 28 resolve the hit assignment performance layer by layer of the ECal, also representing the hit assignment performance along the depth of the electron showers. The mean hit accuracy decreases with ECal depth for both event multiplicities, with the $3e$ sample remaining below the $2e$ sample. The late-layer fluctuations and confidence intervals should be interpreted cautiously because relatively few event-layer observations remain there.

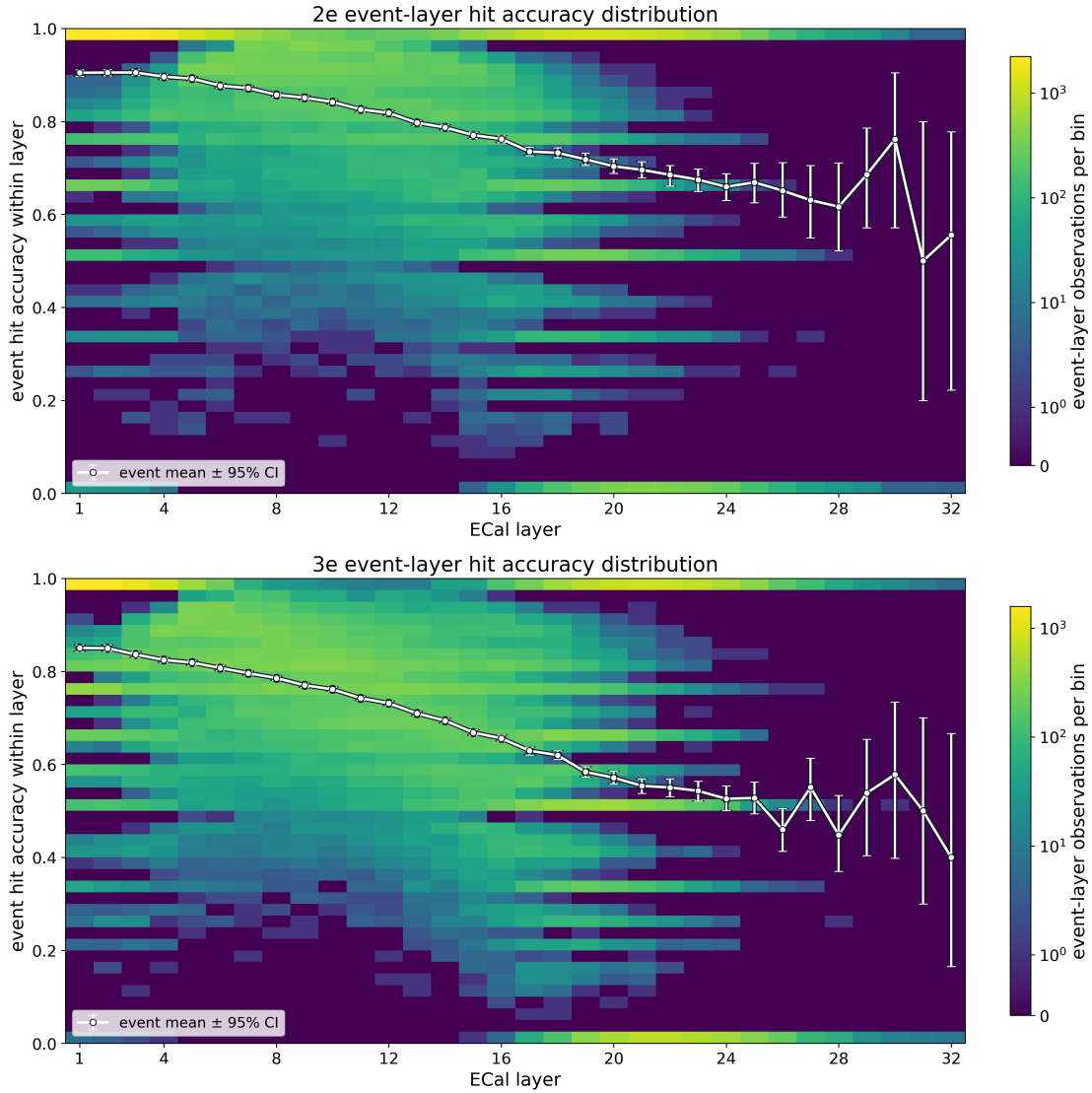


Figure 27. Distribution of permutation-aligned event-layer hit accuracy for $2e$ events (top) and $3e$ events (bottom). Color shows the number of event-layer observations in each rectangular bin on a logarithmic scale. White points show event-balanced means with 95 % bootstrap confidence intervals.

Over all layers, the energy-weighted accuracies are 0.888 for $2e$ and 0.825 for $3e$, both higher than their corresponding unweighted hit accuracies. Within this development sample, incorrectly assigned hits therefore carry less reconstructed energy on average

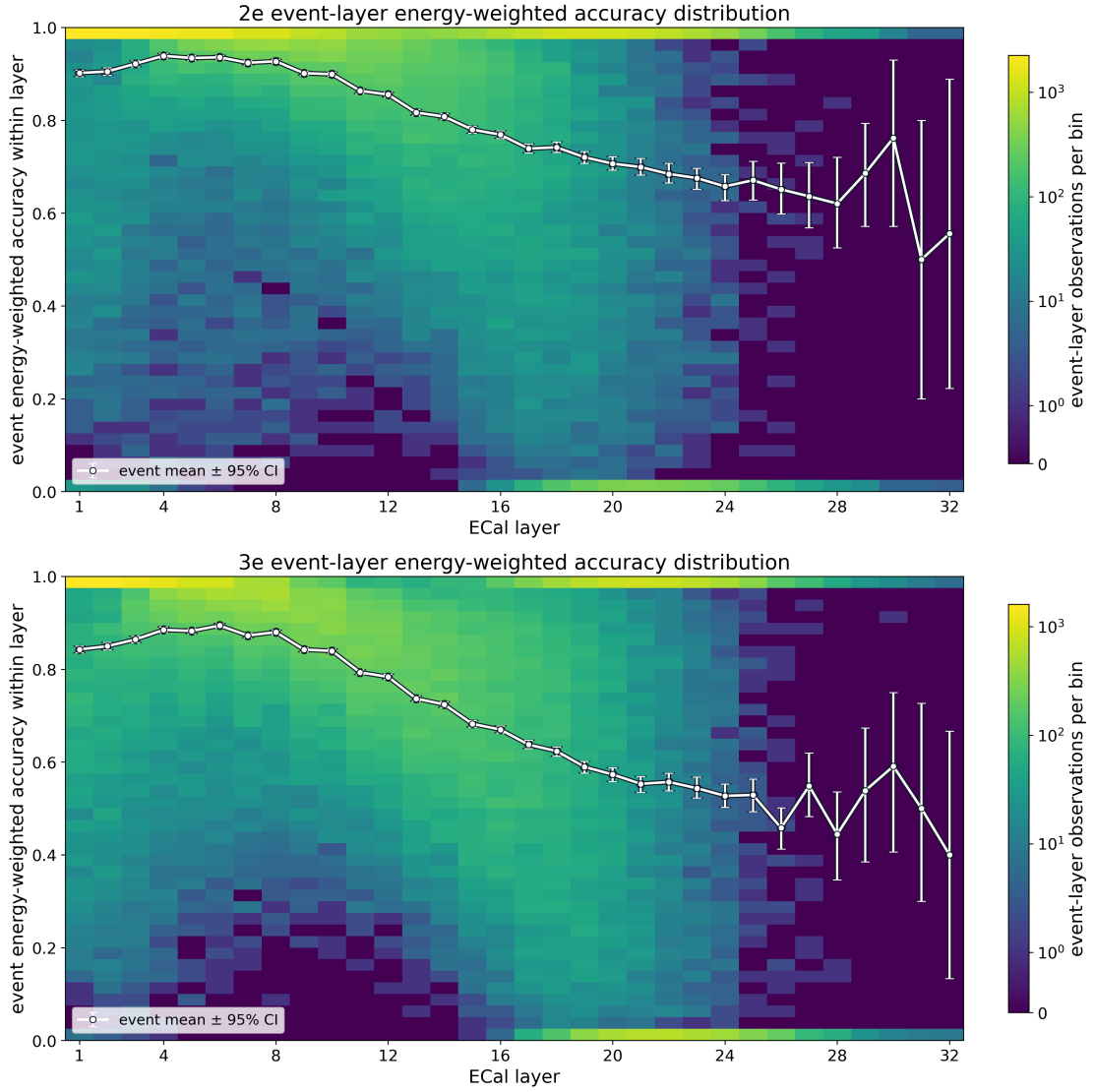


Figure 28. Distribution of raw-energy-weighted event-layer accuracy for $2e$ events (top) and $3e$ events (bottom), using the same event-wide label alignment as Figure 27. White points show event-balanced means with 95 % bootstrap confidence intervals.

than correctly assigned hits. The depth dependence nevertheless remains, showing that energy weighting does not remove the loss of separation quality in the later shower region.

4.3.5 Dependence on Shower Separation

The following plots relate event accuracy to truth-assisted geometric descriptions of shower overlap. For these development diagnostics, the weight of hit h in the geometry of origin j is the product of its raw reconstructed energy and the simulated deposited-energy fraction from that origin. These fractions are simulation truth, so the resulting quantities are oracle difficulty diagnostics rather than observables available in measured detector data. Figure 29 uses the smallest pairwise distance between these energy-weighted shower centroids. Dividing by the reference Molière radius $R_M = 25$ mm supplies an interpretable detector scale, but the line at one Molière radius is not a hard resolution boundary.

The event mean rises as the minimum centroid distance increases. The trend is present for both multiplicities and is consistent with shower overlap being a major source of assignment ambiguity. The sizeable accuracy spread at a fixed distance also shows that centroid separation is not a complete description of event difficulty.

Figure 30 instead divides the minimum centroid distance by the combined energy-weighted root-mean-square widths of the two showers. This produces a dimensionless overlap measure and separates the all-layer geometry from the geometry in the first three layers.

The all-layer profiles again show increasing accuracy with separation. The first-three-layer profiles retain a weaker trend, indicating that the initial shower geometry contains useful information but does not fully determine the separability of the developed shower.

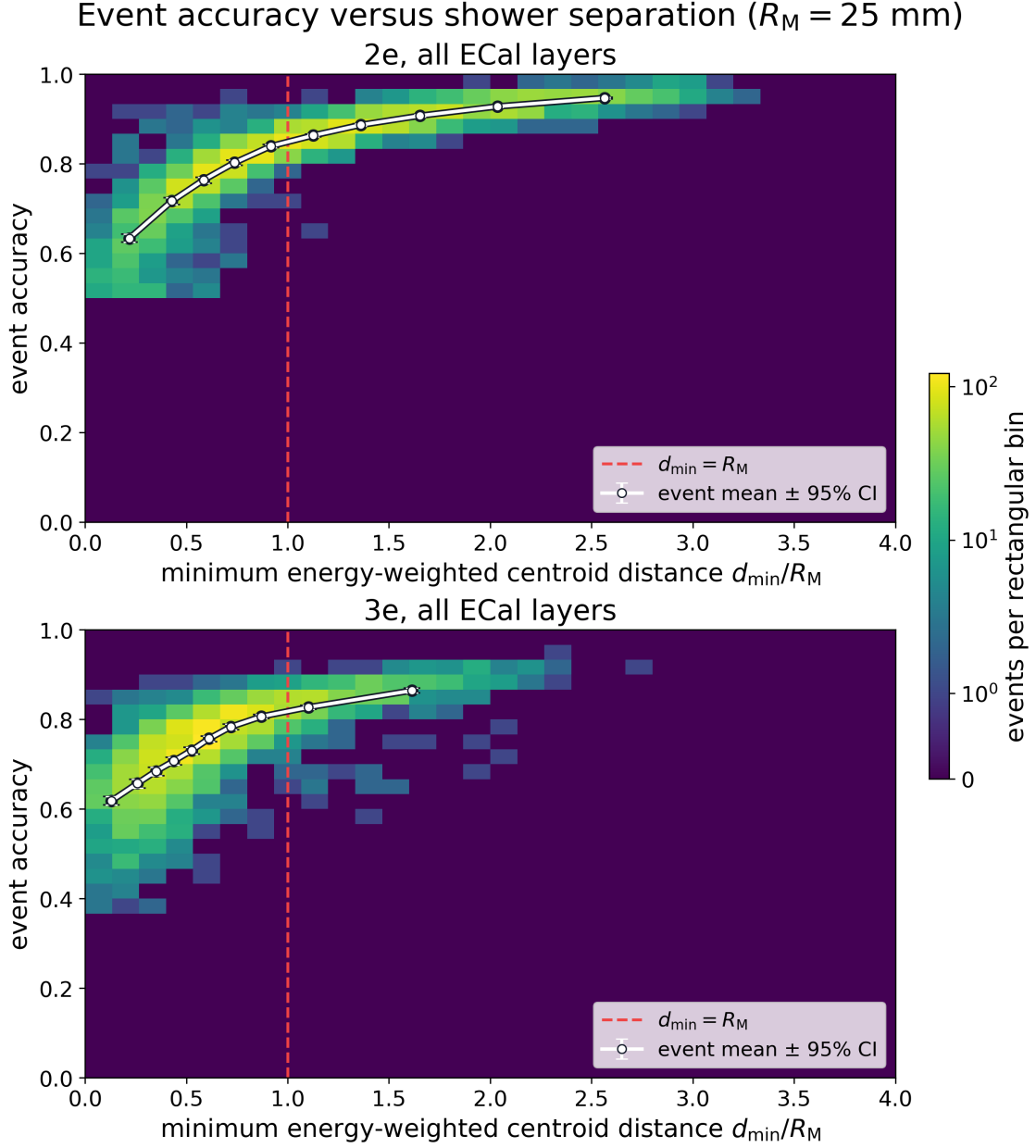


Figure 29. Permutation-aligned event hit accuracy as a function of the minimum energy-weighted centroid separation in units of $R_M = 25$ mm, for $2e$ events (top) and $3e$ events (bottom). White points show equal-population bin means with 95 % bootstrap confidence intervals. The red line marks one Molière radius as an interpretive scale.

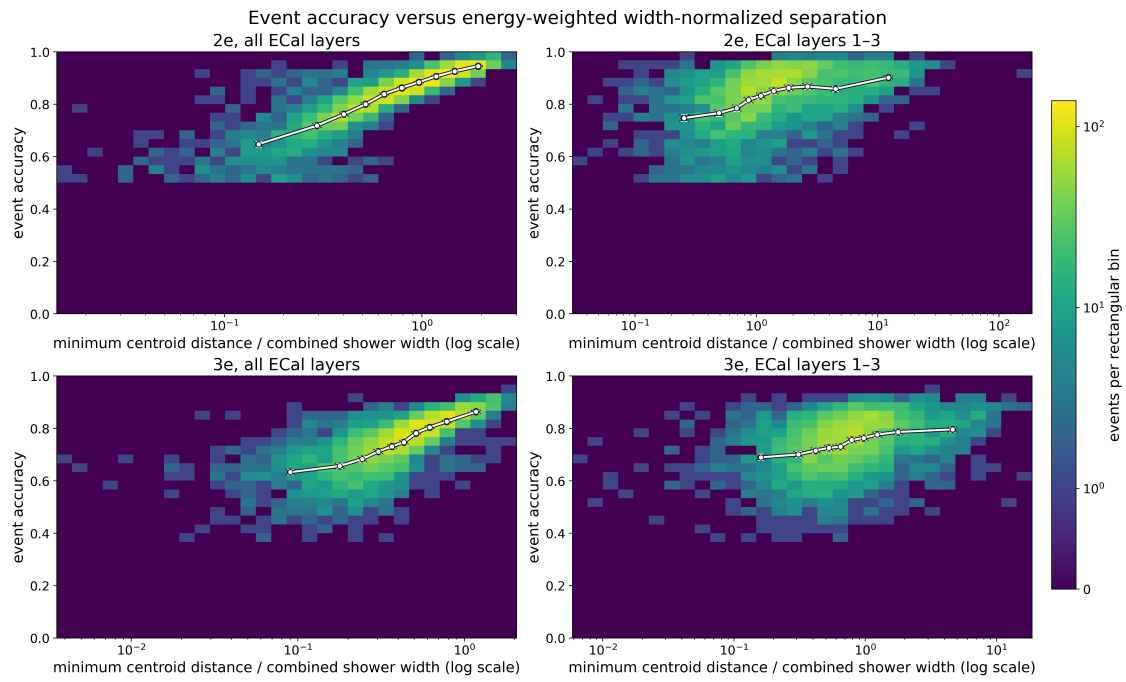


Figure 30. Permutation-aligned event hit accuracy as a function of minimum energy-weighted centroid distance divided by combined shower width. The left column uses all ECal layers and the right column uses layers 1–3; $2e$ events are shown above $3e$ events. The horizontal axes are logarithmic.

4.3.6 Confidence and Selective Performance

The negative relation between prediction entropy and event accuracy in Figure 31 shows that the model is generally less certain on events that it reconstructs poorly. Mean normalised entropy can therefore help rank difficult events, although this correlation does not make it a complete or independently calibrated measure of reconstruction quality.

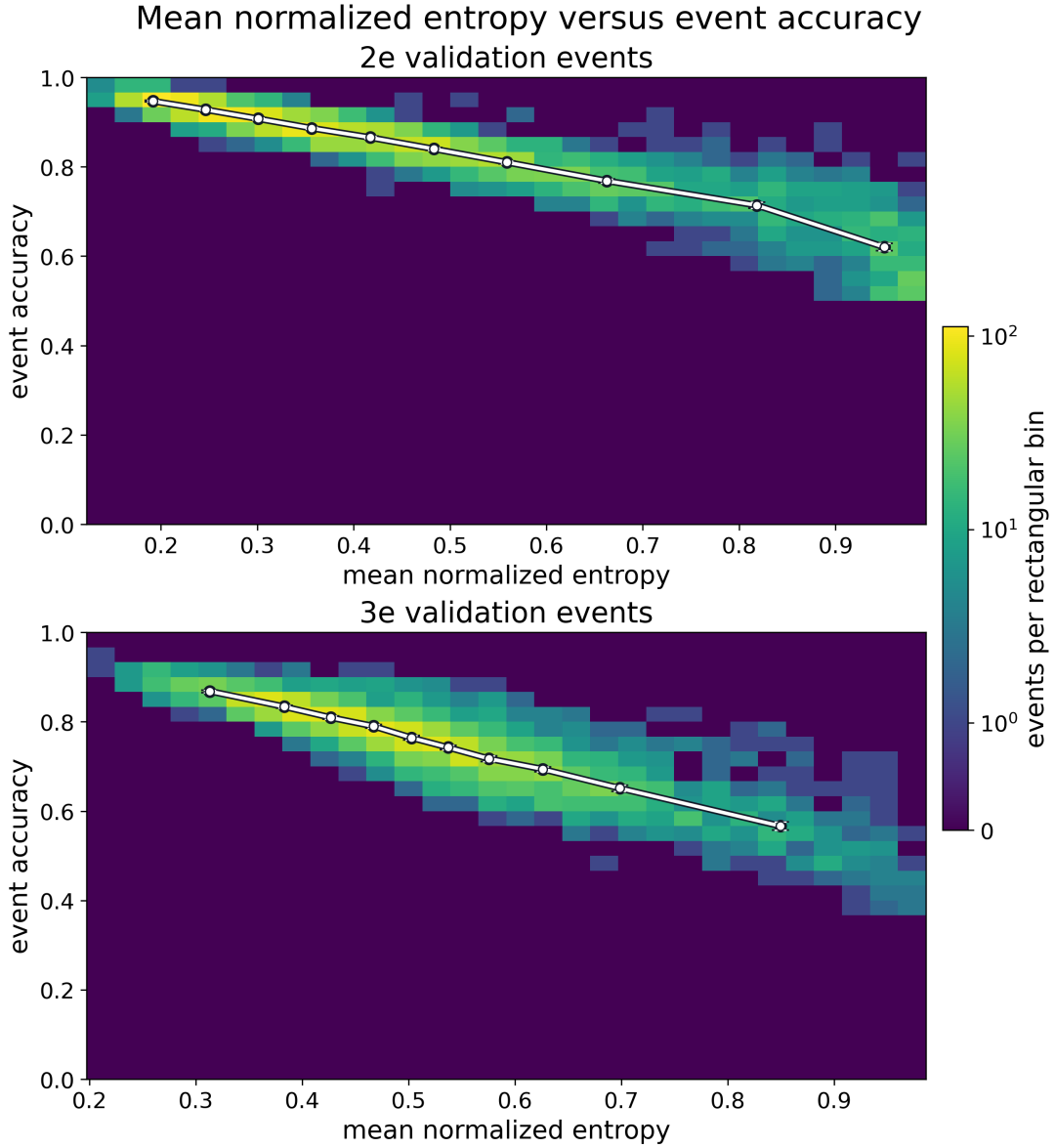


Figure 31. Mean normalized prediction entropy against permutation-aligned event hit accuracy for 2e events (top) and 3e events (bottom). White points show equal-population bin means with 95 % bootstrap confidence intervals.

Figure 32 makes the corresponding selection trade-off explicit. Retaining only high-confidence hits increases their accuracy, but reduces hit coverage. The retained energy fraction decreases more slowly than the retained hit fraction, indicating that many of

the first hits removed carry relatively little reconstructed energy. Selecting complete events by confidence gives a smaller but consistent increase in mean event accuracy as coverage is reduced.

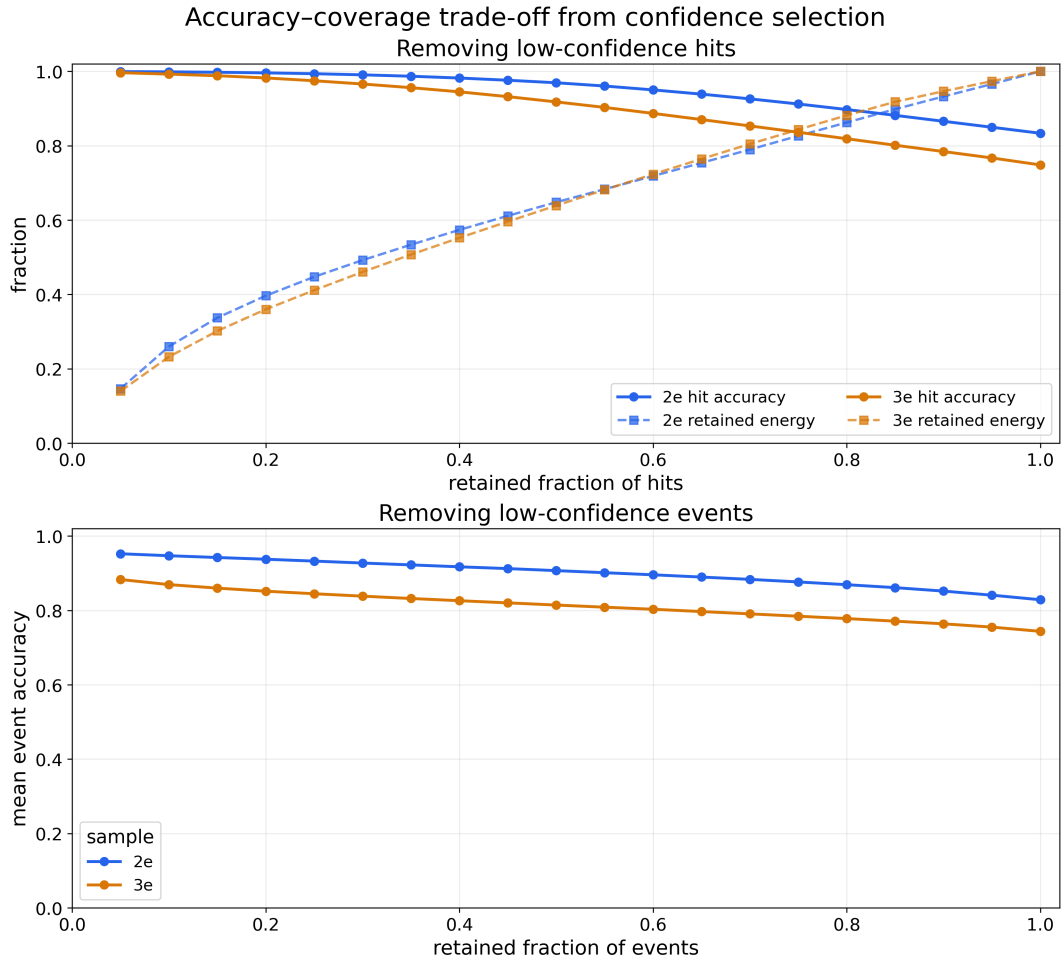


Figure 32. Accuracy-coverage trade-off obtained by ordering predictions by confidence. The upper panel removes low-confidence hits and shows both the accuracy and retained raw-energy fraction. The lower panel removes complete low-confidence events and shows the mean accuracy of the retained events.

The layer profiles in Figure 33 show that mean confidence and mean accuracy follow each other closely through most of the ECal. Their separation and uncertainty grow in the sparsely populated final layers, where the per-layer estimate is least stable. Agreement between the two layer means does not by itself establish hit-level probability calibration.

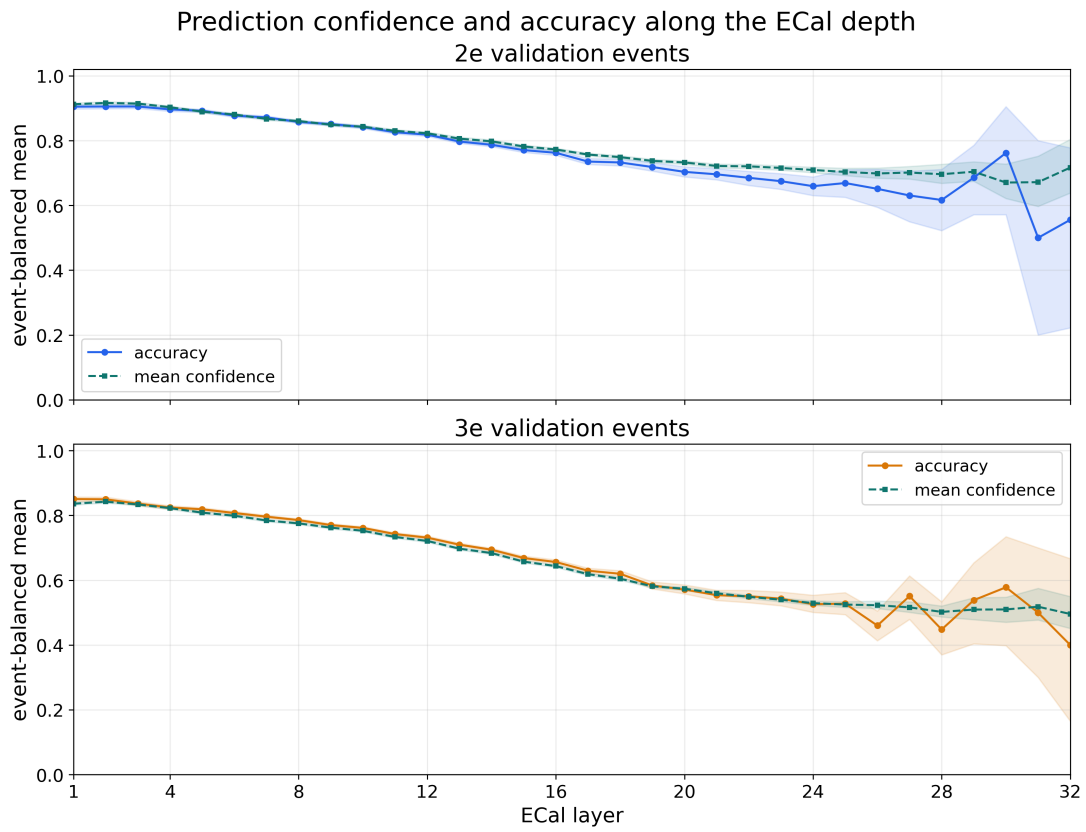


Figure 33. Event-balanced mean prediction confidence and permutation-aligned accuracy across the ECal depth for $2e$ events (top) and $3e$ events (bottom). Shaded bands show 95% bootstrap confidence intervals.

4.3.7 Trigger-Pad Ablation

THIS SUBSECTION SHOULD COVER TRIGGER-PAD ABLATION-RELATED RESULTS AND DISCUSS IF THE BASELINE MODELS ARE ABLE TO SQUEEZE SOMETHING OUT OF THE TPad INFORMATION OR NOT

Figure 34 removes trigger-pad-track tokens at inference time from one trained $3e$ ECalTpadTransformer. The reference and ablated event accuracies are almost identical. Across the 3,000 validation events, the pooled hit-accuracy gain from retaining the trigger-pad inputs is 1.26×10^{-4} , the mean event-accuracy gain is 1.32×10^{-4} , and the median event-accuracy gain is zero.

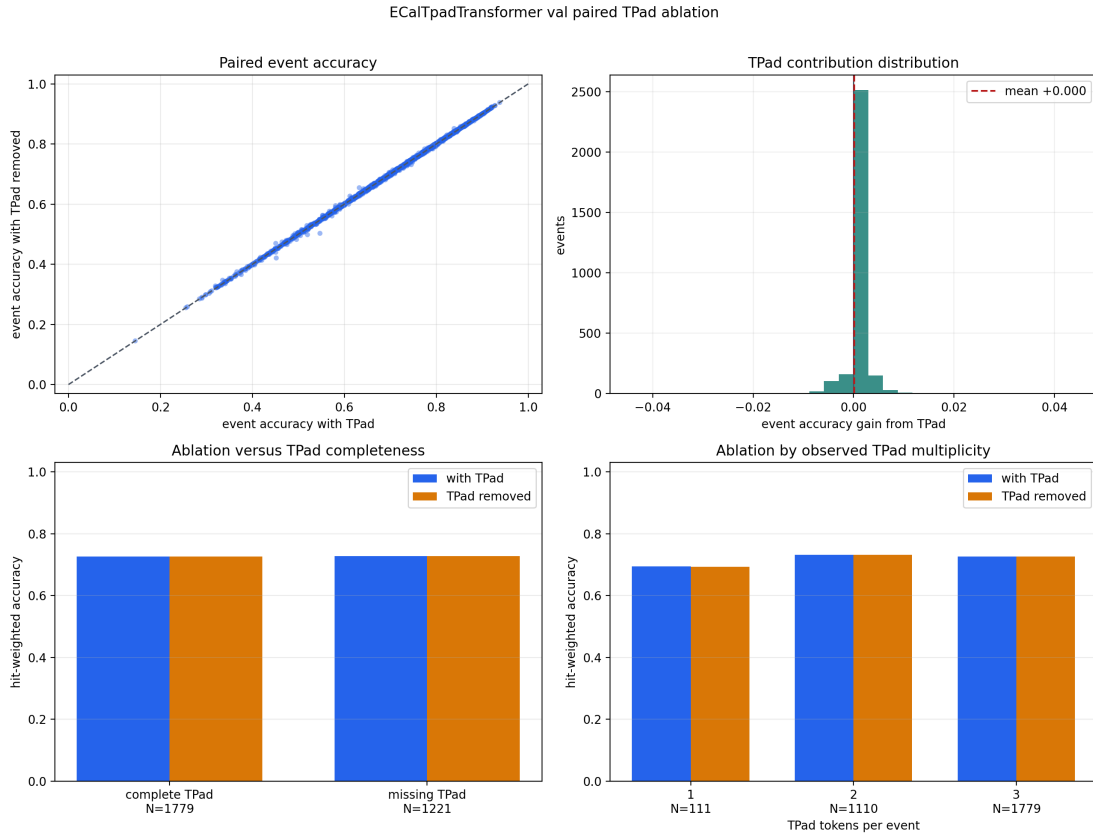


Figure 34. **THIS FIG SUCKS I WILL REPLACE** Paired inference-time ablation of trigger-pad-track tokens for a trained $3e$ ECalTpadTransformer on 3,000 validation events. The upper panels compare event accuracy with and without the trigger-pad inputs. The lower panels divide the comparison by completeness and observed trigger-pad multiplicity.

For this trained model and validation sample, the available trigger-pad inputs therefore have negligible influence on fixed-multiplicity hit assignment. The ablation removes tokens from an already trained model and does not replace a controlled comparison between separately trained ECal-only and ECal-plus-trigger-pad models. More importantly, the result is specific to a task for which the electron multiplicity is already fixed. It does not imply that trigger-pad information is uninformative when the model must also infer the number of electrons in an event.

4.4 MLPF-Inspired Slot Model

4.4.1 MLPF-Inspired Contributor-Set Reconstruction

The experimental contributor-set slot model extends the fixed-multiplicity assignment problem to a particle-flow-like multi-task setting. It was trained on 50,000 $2e$ and 50,000 $3e$ events and evaluated here using the best checkpoint, selected at epoch 14. The backbone has hidden width 192, three pre-layer-normalised Transformer layers and eight attention heads, corresponding to approximately 1.16 million trainable parameters. The test set contains 10,000 events and 3,025,349 ECal hits.

The model predicts three electron-slot validity scores for each event and a contributor-set distribution and energy fractions for each hit. The electron count is derived from the valid slots; it is not produced by a separate count head. Similarly, the dominant-origin label is derived from the predicted fractions. This post-processing keeps the reported quantities consistent with the model’s underlying set-valued prediction.

Table XI
TEST-SET PERFORMANCE OF THE CONTRIBUTOR-SET SLOT MODEL SELECTED AT EPOCH 14. COUNT, CONTRIBUTOR-SET AND DOMINANT-ORIGIN RESULTS ARE ACCURACIES; THE FRACTION ERRORS ARE MEAN ABSOLUTE ERRORS.

Metric	Value
Derived electron-count accuracy, all events	99.79 %
Derived electron-count accuracy, true $2e$	99.98 %
Derived electron-count accuracy, true $3e$	99.61 %
Exact per-hit contributor-set accuracy	72.39 %
Per-hit dominant-origin accuracy	77.09 %
Mixed-hit precision	46.49 %
Mixed-hit recall	64.69 %
Mixed-hit F_1 score	0.541
Fraction MAE, raw prediction	0.144
Fraction MAE, validity-masked prediction	0.115

Figure 35 resolves the exact contributor-set task. Pure one-electron contributions dominate the data and are generally reconstructed more reliably than mixed sets. The overall exact set accuracy is 72.39 %, so this is substantially more demanding than electron counting. In particular, a contributor-set error can occur even when the predicted dominant origin and the event electron count remain correct.

Figure 36 shows two useful reductions of the same predictions. Dominant-origin accuracy reaches 77.09 %, which is comparable in scale to the fixed-multiplicity $3e$ baseline despite the additional event-count and fraction tasks. Mixed-hit detection reaches 64.69 % recall but only 46.49 % precision, giving an F_1 score of 0.541. The model therefore identifies a majority of truth-mixed hits but also labels a sizeable number of pure hits as mixed. This task should not be summarised by ordinary accuracy alone because pure hits are much more common.

Applying the predicted slot-validity mask before normalising the energy fractions reduces their mean absolute error from 0.144 to 0.115. This improvement illustrates one purpose of coupling the tasks: the event-level validity prediction constrains which

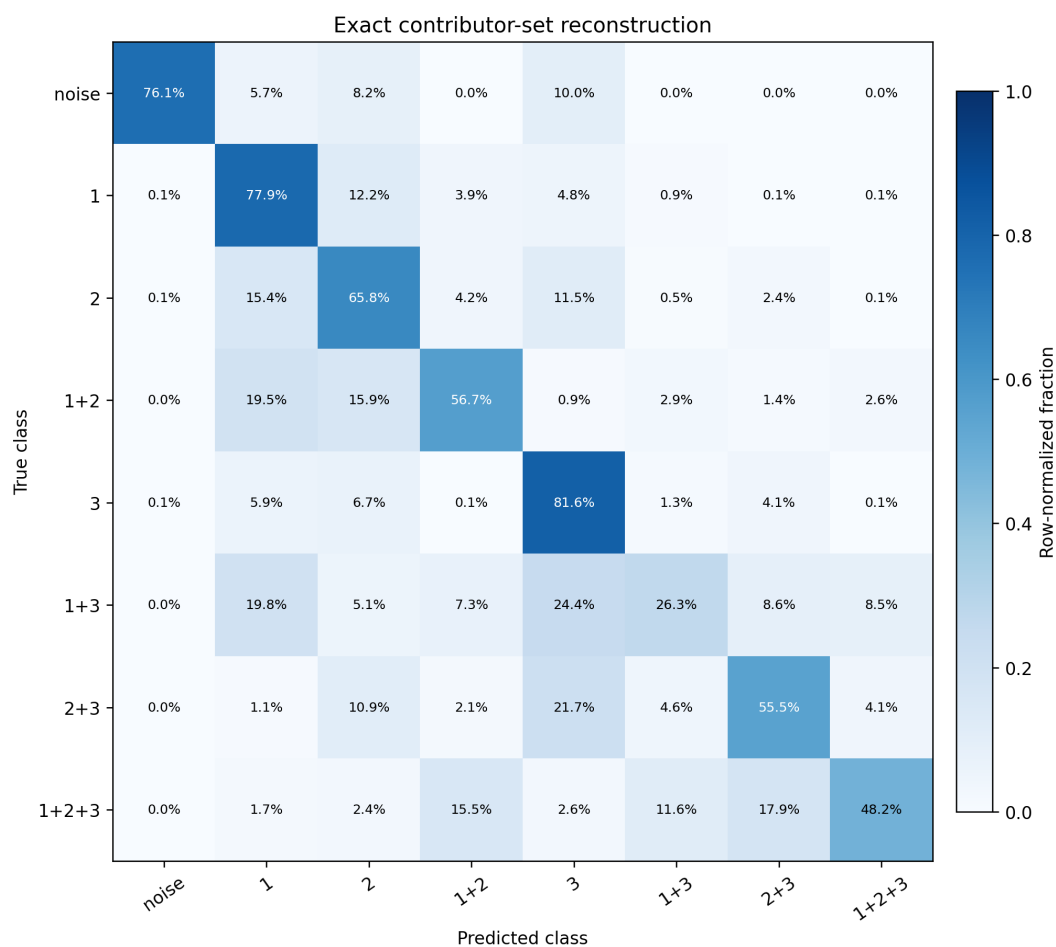


Figure 35. Row-normalised test-set confusion matrix for the per-hit contributor set predicted by the slot model. Each class denotes the exact subset of electron origins contributing simulated deposited energy to a hit. Cell annotations show row percentages.

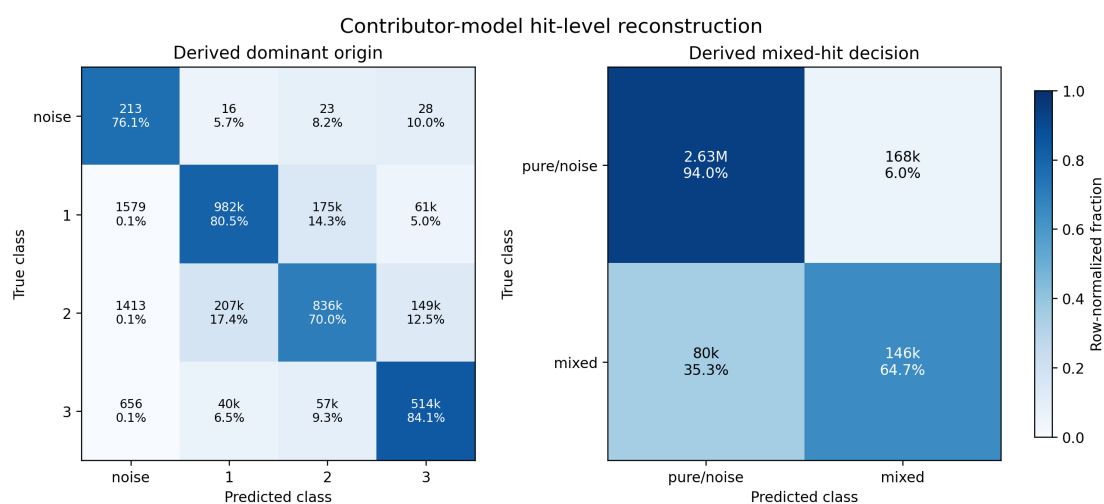


Figure 36. Test-set confusion matrices derived from the contributor-slot outputs. Left: dominant electron origin inferred from the predicted energy fractions. Right: binary pure-versus-mixed classification derived from the predicted contributor set.

electron fractions are physically available during post-processing. Nevertheless, the contributor-set and mixed-hit results show that resolving overlap at hit level remains the main weakness of this preliminary model.

4.4.2 Electron Counting and the Contribution of Trigger-Pad Information

Electron counting is the strongest result of the contributor-set model. The derived count is correct for 99.79 % of test events, with similar performance for true $2e$ and $3e$ samples. Table XII compares this result with both an inference-time removal of the trigger-pad tokens and a direct heuristic that equates observed trigger-pad multiplicity with electron count.

Table XII
ELECTRON-COUNT PERFORMANCE AND PAIRED DIFFERENCES ON THE 10,000-EVENT CONTRIBUTOR-MODEL TEST SET. ACCURACY VALUES ARE PERCENTAGES. DIFFERENCES ARE PERCENTAGE POINTS WITH EVENT-BOOTSTRAP 95 % CONFIDENCE INTERVALS. “TPAD REMOVED” IS AN INFERENCE-TIME ABLATION OF THE SAME TRAINED CHECKPOINT, NOT AN INDEPENDENTLY RETRAINED ECal-ONLY MODEL.

Method or comparison	All	True $2e$	True $3e$
Neural model, ECal+TPad	99.79	99.98	99.61
Same model, TPad removed	98.15	100.00	96.36
TPad multiplicity heuristic	84.31	100.00	69.12
ECal+TPad minus TPad removed	+1.64	−0.02	+3.25
95 % CI	[1.39, 1.89]	[−0.06, 0.00]	[2.78, 3.74]
Neural minus heuristic	+15.48	−0.02	+30.49
95 % CI	[14.79, 16.22]	[−0.06, 0.00]	[29.21, 31.77]

The paired ablation reduces overall count accuracy by 1.64 percentage points, with a 95 % event-bootstrap interval of [1.39, 1.89] percentage points. Almost the entire effect occurs in true $3e$ events, for which removing trigger-pad information reduces accuracy by 3.25 percentage points. The interval excludes zero, providing evidence that this trained network uses the trigger-pad input for the variable-count task.

The trigger-pad multiplicity by itself is not an adequate electron counter. As Figure 37 shows, it classifies every $2e$ event correctly but misclassifies 1,569 of the 5,081 true $3e$ events. The neural model makes only 21 count errors over the full 10,000 events. Its overall advantage over the heuristic is 15.48 percentage points, with a 95 % interval of [14.79, 16.22] percentage points. The result indicates that the model combines ECal shower information and trigger-pad observations rather than merely reproducing the observed number of trigger-pad tokens.

Together with the fixed-count ablation in Figure 34, these results show a task-dependent role for trigger-pad information: it has negligible measured effect on the baseline’s hit assignment when multiplicity is given, but a clear effect when the contributor model must infer multiplicity. The present ablation still introduces a distribution shift because trigger-pad inputs are removed only at inference time. A separately trained ECal-only contributor model is needed to complete the controlled architecture comparison and is treated as follow-up work rather than folded into the current estimate.

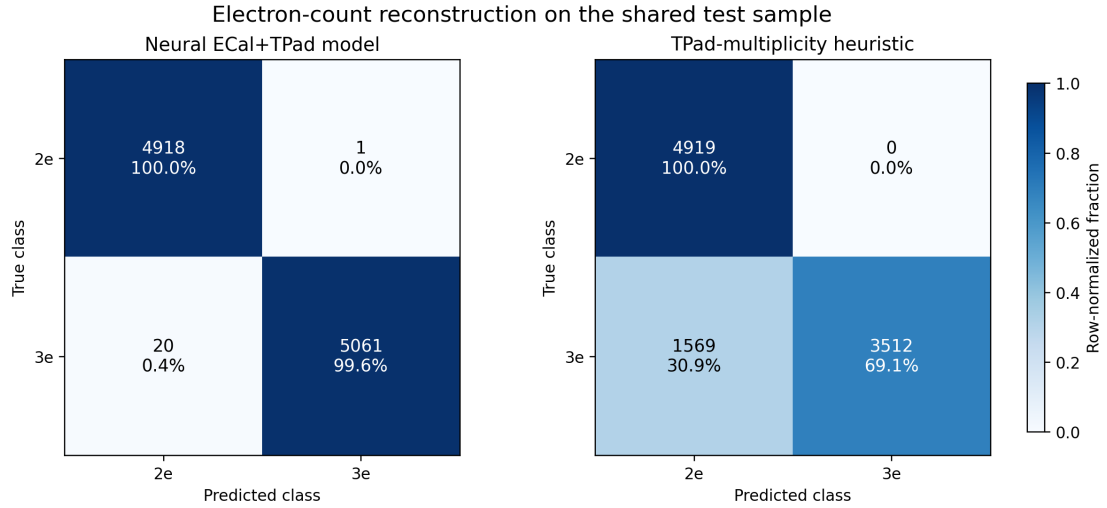


Figure 37. Row-normalised electron-count confusion matrices on the common test events. Left: count derived from the contributor model’s slot-validity predictions using ECal and trigger-pad inputs. Right: direct trigger-pad multiplicity heuristic. Counts and row percentages are shown in each cell.

5 Conclusions and Outlook

5.1 Summary of Contributions

Summarize main findings and improvements over baseline.

5.2 Future Work

Discuss architectural improvements. Discuss scalability. Discuss transformers and potential benefits with hardware specifically made for this.

Explore integration into high-level trigger.

Discuss training per epoch being a bit wierd.

Discuss noisy loss with experimental model due to incorrect 2e/3e predictions and wonder how one handles that.

Discuss trimming the model to minimal size but preserving performance.

Discuss how MORE information can be fed and how that has not been tested yet. Discuss that MORE information can mean new unique information but it can also include more object-level information that is already processed beforehand somewhat.

Discuss the 3 highlighted simplifications and why they were made.

Discuss the comparison between transformer and gnn based models and why transformers seem to be the preffered choice given this study.

Discuss the prospect of general optimization trimming the size of the network: parameters, inference time, size on disk.

Discuss prospect of adding tasks to the model (multi-task already or not).

Discuss the canonical y-ordering and potential of using Hungarian matching instead for loss calculation.

Discuss implementing LSH or was it called LHS as in mlpf 2021 or flash attention reducing the sizes and reducing inference time. Discuss why this is probably overkill for this project with relatively few inputs.

Put emphasis on that the codebase that was the basis for this report is meant to be logically structured and documented in such a way so that it can be handed over and that it can act as a platform for deep learning ML based tasks connected to LDMX. One of the intended purposes of the master thesis was to create something that could be picked up at a later stage. It should be a platform where resources can be reused to see if ML based methods can perform different reconstruction tasks, by students or researchers alike.

Discuss the inclusion of TS and tpad information. Why heterogenous detector information is interesting, and discuss why the tpad information in this case seem to matter little to nothing at all. Discuss what that could mean, discuss if we think that we can squeeze more information out or not. When do we expect this information to be important? When is it not important?

Alternative geometry for trigger scintillator can be used to achieve resolution in both x and y directions.

6 References

- [1] Eliot M. Petrén. *Machine Learning-Based Reconstruction for LDMX (Master's Thesis Repository)*. <https://github.com/cat2code/mpetren-msceng-ldmx>. Accessed: 2026-03-03. 2026.
- [2] Huilin Qu and Loukas Gouskos. *ParticleNet: Jet Tagging via Particle Clouds*. Accessed: 2025-11-19. 2019. arXiv: 1902.08570 [hep-ph]. URL: <https://arxiv.org/pdf/1902.08570>.
- [3] Marco Cirelli, Alessandro Strumia and Jure Zupan. 'Dark Matter'. In: *SciPost Physics Reviews*, 1 (2026). DOI: 10.21468/SciPostPhysRev.1. arXiv: 2406.01705 [hep-ph].
- [4] Torsten Åkesson et al. *Light Dark Matter eXperiment (LDMX)*. 2018. arXiv: 1808.05219 [hep-ex]. URL: <https://arxiv.org/pdf/1808.05219>.
- [5] Torsten Åkesson et al. *LDMX – The Light Dark Matter eXperiment*. 2025. arXiv: 2508.11833 [hep-ex]. URL: <https://arxiv.org/pdf/2508.11833>.
- [6] Shah Rukh Qasim et al. 'Learning Representations of Irregular Particle-Detector Geometry with Distance-Weighted Graph Networks'. In: *European Physical Journal C* 79.7 (2019), p. 608. DOI: 10.1140/epjc/s10052-019-7113-9. arXiv: 1902.07987 [physics.data-an].
- [7] Joosep Pata et al. *MLPF: Efficient machine-learned particle-flow reconstruction using graph neural networks*. 2021. DOI: 10.1140/epjc/s10052-021-09158-w. arXiv: 2101.08578 [physics.data-an].
- [8] CMS Collaboration. *Full event interpretation with machine-learning-based particle-flow reconstruction in the CMS detector*. Submitted to the European Physical Journal C. 2026. arXiv: 2601.17554 [physics.ins-det].
- [9] Eliot M. Petrén. *ML-based Particle Reconstruction Platform for LDMX (built for Eliot's Master Thesis)*. https://github.com/cat2code/ml_ldmx. Accessed: 2026-07-30. 2026.
- [10] Gianfranco Bertone and Dan Hooper. 'History of Dark Matter'. In: *Reviews of Modern Physics* 90.4, 045002 (2018). DOI: 10.1103/RevModPhys.90.045002. arXiv: 1605.04909 [astro-ph.CO].
- [11] Knut Lundmark. 'Über die Bestimmung der Entfernungen, Dimensionen, Massen und Dichtigkeit für die nächstgelegenen anagalaktischen Sternsysteme'. In: *Meddelanden från Lunds Astronomiska Observatorium, Serie I* 125 (1930). Bibcode: 1930MeLuF.125....1L, pp. 1–13.
- [12] Odrodrui. *Knut Lundmark and the Forgotten Dark Matter Discovery*. <https://www.physicsforums.com/insights/knut-lundmark-prehistory-dark-matter/>. Physics Forums Insights, Accessed: 2026-05-08. Jan. 2016.
- [13] Marco Battaglieri et al. *US Cosmic Visions: New Ideas in Dark Matter 2017: Community Report*. 2017. DOI: 10.48550/arXiv.1707.04591. arXiv: 1707.04591 [hep-ph].
- [14] Gréta Gajdán. 'LDMX HCal Prototype Data Analysis'. Master's thesis. Lund University, Department of Physics, 20th June 2025.

- [15] Gordan Krnjaic. ‘Testing Thermal-Relic Dark Matter with a Dark-Photon Mediator’. In: *Physical Review D* 113.11, 115057 (2026). DOI: 10.1103/sd6p-8y9p. arXiv: 2505.04626 [hep-ph].
- [16] Axel Helgstrand. ‘Towards Quantifying Veto Power with the LDMX HCal Prototype’. Master’s thesis. Lund University, Department of Physics, 2025. URL: <http://lup.lub.lu.se/student-papers/record/9202508> (visited on 02/08/2026).
- [17] Riccardo Catena and Taylor R. Gray. ‘Spin-1 Thermal Targets for Dark Matter Searches at Beam Dump and Fixed Target Experiments’. In: *Journal of Cosmology and Astroparticle Physics* 2023.11 (2023), p. 058. DOI: 10.1088/1475-7516/2023/11/058. arXiv: 2307.02207 [hep-ph].
- [18] Torsten Åkesson et al. *Current Status and Future Prospects for the Light Dark Matter eXperiment*. 2022. arXiv: 2203.08192 [hep-ex]. URL: <https://arxiv.org/pdf/2203.08192>.
- [19] D. E. Groom and S. R. Klein. *Passage of Particles Through Matter*. Review in S. Navas et al., Review of Particle Physics, *Physical Review D* 110, 030001. Particle Data Group. 2024. URL: <https://pdg.lbl.gov/2024/reviews/rpp2024-rev-passage-particles-matter.pdf>.
- [20] IML Working Group. *Living Review of Machine Learning for Particle Physics*. <https://github.com/iml-wg/HEPML-LivingReview>. GitHub repository, Accessed: 2026-05-06.
- [21] Nairit Sur. ‘From CPU Grids to GPUs and FPGAs: Heterogeneous Computing in Collider Physics’. Science Coffee seminar, Department of Physics, Lund University. Teorilabbet (K308), Lund University, 16th Apr. 2026.
- [22] Georgia Karagiorgi et al. *Machine Learning in the Search for New Fundamental Physics*. 2021. arXiv: 2112.03769 [hep-ph]. URL: <https://arxiv.org/pdf/2112.03769>.
- [23] Matthew Feickert and Benjamin Nachman. *A Living Review of Machine Learning for Particle Physics*. 2021. arXiv: 2102.02770 [hep-ph]. URL: <https://arxiv.org/pdf/2102.02770>.
- [24] Javier Duarte and Jean-Roch Vlimant. *Graph Neural Networks for Particle Tracking and Reconstruction*. 2020. arXiv: 2012.01249 [hep-ex]. URL: <https://arxiv.org/pdf/2012.01249>.
- [25] Justin Gilmer et al. ‘Neural Message Passing for Quantum Chemistry’. In: *Proceedings of the 34th International Conference on Machine Learning*. Vol. 70. Proceedings of Machine Learning Research. PMLR, 2017, pp. 1263–1272. arXiv: 1704.01212 [cs.LG]. URL: <https://proceedings.mlr.press/v70/gilmer17a.html>.
- [26] Ashish Vaswani et al. ‘Attention Is All You Need’. In: *Advances in Neural Information Processing Systems*. Vol. 30. 2017, pp. 5998–6008. arXiv: 1706.03762 [cs.CL].
- [27] Juho Lee et al. ‘Set Transformer: A Framework for Attention-Based Permutation-Invariant Neural Networks’. In: *Proceedings of the 36th International Conference on Machine Learning*. Vol. 97. Proceedings of Machine Learning Research.

- PMLR, 2019, pp. 3744–3753. arXiv: 1810.00825 [cs.LG]. URL: <https://proceedings.mlr.press/v97/lee19d.html>.
- [28] Chaitanya K. Joshi. *Transformers are Graph Neural Networks*. 2025. DOI: 10.48550/arXiv.2506.22084. arXiv: 2506.22084 [cs.LG].
 - [29] Sara Hooker. *The Hardware Lottery*. 2020. DOI: 10.48550/arXiv.2009.06489. arXiv: 2009.06489 [cs.CY].
 - [30] LUNARC. *COSMOS*. URL: <https://www.lunarc.lu.se/systems/cosmos> (visited on 24/07/2026).
 - [31] LDMX Collaboration. *LDMX Software Repository*. <https://github.com/LDMX-Software/ldmx-sw/>. Accessed: 2025-11-19.
 - [32] S. Agostinelli et al. ‘GEANT4—a simulation toolkit’. In: *Nuclear Instruments and Methods in Physics Research Section A* 506.3 (2003), pp. 250–303. DOI: 10.1016/S0168-9002(03)01368-8.
 - [33] LDMX Collaboration. *ldmx-sw: pile-up validation samples*. GitHub repository directory: [.github/validation_samples/it_pileup](https://github.com/LDMX-Software/ldmx-sw/tree/trunk/.github/validation_samples/it_pileup). URL: https://github.com/LDMX-Software/ldmx-sw/tree/trunk/.github/validation_samples/it_pileup (visited on 24/07/2026).
 - [34] LDMX Collaboration. ‘A High Efficiency Photon Veto for the Light Dark Matter eXperiment’. In: *Journal of High Energy Physics* 2020.4 (2020), p. 003. DOI: 10.1007/JHEP04(2020)003. arXiv: 1912.05535 [physics.ins-det].
 - [35] Eliot M. Petré. *Fix origin ID handling in OverlayProducer*. <https://github.com/LDMX-Software/ldmx-sw/pull/2032>. Pull request to the LDMX Software Repository, merged 2026-04-22. Accessed: 2026-05-08. 2026.
 - [36] Rene Brun and Fons Rademakers. ‘ROOT: An Object Oriented Data Analysis Framework’. In: *Nuclear Instruments and Methods in Physics Research Section A* 389 (1997), pp. 81–86. DOI: 10.1016/S0168-9002(97)00048-X.
 - [37] ROOT Team. *About ROOT*. URL: <https://root.cern/about/> (visited on 16/07/2026).
 - [38] ROOT Collaboration. *TTree: ROOT Trees*. ROOT documentation. URL: <https://root.cern/manual/trees/> (visited on 13/07/2026).
 - [39] Uproot developers. *Uproot*. 13th Sept. 2017. DOI: 10.5281/zenodo.4340632. URL: <https://github.com/scikit-hep/uproot5>.
 - [40] Uproot developers. *Uproot documentation*. URL: <https://uproot.readthedocs.io/en/stable/> (visited on 27/07/2026).
 - [41] Awkward Array development team. *Awkward Array documentation*. URL: <https://awkward-array.org/doc/main/> (visited on 27/07/2026).
 - [42] PyTorch. *PyTorch documentation*. URL: <https://docs.pytorch.org/docs/stable/index.html> (visited on 27/07/2026).
 - [43] PyTorch. *torch.log1p — PyTorch 2.13 documentation*. URL: <https://docs.pytorch.org/docs/2.13/generated/torch.log1p.html> (visited on 09/07/2026).
 - [44] S. Lai et al. ‘Shower separation in five dimensions for highly granular calorimeters using machine learning’. In: *Journal of Instrumentation* 19.10 (2024), P10027. DOI: 10.1088/1748-0221/19/10/P10027. arXiv: 2407.00178 [physics.ins-det].

- [45] Ilya Loshchilov and Frank Hutter. ‘Decoupled Weight Decay Regularization’. In: *International Conference on Learning Representations*. 2019. arXiv: 1711.05101 [cs.LG]. URL: <https://openreview.net/forum?id=Bkg6RiCqY7>.

Abbreviations and Glossary

Glossary

Bremsstrahlung	Photon radiation emitted by electrons traversing material. It constitutes a major background and drives electromagnetic shower development. 11
Dark Photon	A hypothetical dark-sector gauge boson A' that acts as a mediator between SM and dark-sector particles; one of the primary physics targets of LDMX. 5, 6
Electron Shower	An electromagnetic cascade initiated when electrons enter the ECal. Its topology and energy profile are central to background rejection. 11
Pile-Up	The presence of multiple overlapping electrons or interactions within the same detector readout window, leading to overlapping showers and increased ambiguity in reconstruction. 14

Acronyms

ldmx-sw	LDMX-Software 24, 26, 27, 83
CMS	Compact Muon Solenoid 11, 17–19, 36
CNN	Convolutional Neural Network 18
CPU	Central Processing Unit 17
DM	Dark Matter 2–6, 8, 15
ECal	Electromagnetic Calorimeter , 2, 9, 11, 13, 19, 20, 24, 25, 27, 29–33, 41, 42, 45–47, 50, 61, 86
EoT	Electrons On Target 7, 14, 15
GAT	Graph Attention Networks 21
Geant4	GEometry ANd Tracking 4 24, 26
GNN	Graph Neural Network 2, 18, 19, 21, 22, 24, 25
GPU	Graphics Processing Unit 15, 17, 22
GravNet	A Graph Neural Network Architecture For Learned Clustering And Representation Learning In High-Granularity Particle Detectors 20
HCal	Hadronic Calorimeter 13
HEP	High-Energy Physics 15–17, 19
HGCAL	High Granularity Calorimeter 11
HPC	High-Performance Computing 24, 26, 27
LDM	Light Dark Matter 4, 5
LDMX	Light Dark Matter EXperiment 2–4, 6, 11, 17–19, 24, 26–28, 36, 38, 51, 80
LHC	Large Hadron Collider 17, 18
ML	Machine Learning 2, 15–18, 22, 24, 25, 28, 31, 34

MLPF	Machine Learning Particle Flow 17–19, 22, 36, 38, 53
RecHit	Reconstructed Hit Quantities 29–32, 35
SimHit	Simulated Hit Quantities 30
SLAC	SLAC National Accelerator Laboratory 6
SM	Standard Model 2–7, 15, 26, 80
TPad	Trigger-Pad 29, 33, 50, 51
TPU	Tensor Processing Unit 17, 22
TS	Trigger Scintillator 2, 8, 12, 13, 25
WIMP	Weakly Interacting Massive Particle 5

7 Appendix

Additional plots and tables. Hyperparameter settings. Extended implementation details.

7.1 WILL REMOVE THIS: LDMX-SW pull request #2032 - *Fix origin ID handling in OverlayProducer*

This appendix summarizes the code changes introduced in pull request [35] that got merged into `ldmx-sw`. The purpose of the changes was to preserve the `origin_id` information in overlaid simulated calorimeter hits, while keeping the new bitwise track-ID encoding introduced in `ldmx-sw` separate from the origin labels used for downstream truth-level analysis. The origin labels were necessary truth labels to extract from simulations for the scope of this thesis.

The pull request modifies two files:

- `Recon/src/Recon/OverlayProducer.cxx`
- `SimCore/src/SimCore/Event/SimCalorimeterHit.cxx`

7.1.1 Changes in `OverlayProducer.cxx`

The main change in `OverlayProducer.cxx` is that the overlay-event index is now taken directly from the loop variable `i_ev`, and the corresponding `origin_id` value is propagated when adding calorimeter hit contributions. A guard was also added to prevent more than seven overlay events, since the bitwise track-ID encoding reserves three bits for the overlay-event index.

```

1 if (i_ev + 1 > 7) {
2     ldmx_log(error) << "Too_many_overlay_events!_Maximum_events_that_can_
        "
3         "be_overlaid_is_7.";
4     return;
5 }
6
7 ...
8
9 this_coll_hit_map[overlay_hit_id].addContrib(
10     contrib.incident_id_, contrib.track_id_, contrib.pdg_code_,
11     contrib.edep_, contrib.time_ + time_offset,
12     contrib.origin_id_ + i_ev + 1);

```

Listing 1. Excerpt of the origin-ID handling change in `OverlayProducer.cxx`.

7.1.2 Changes in `SimCalorimeterHit.cxx`

The change in `SimCalorimeterHit.cxx` removes `origin_id` from the track-ID encoding step. This preserves `origin_id` as a direct truth label, rather than treating it as an encoded track-like identifier.

```

1 void SimCalorimeterHit::encodeTracks(
2     const std::function<int(int, unsigned, unsigned)> &encodeFunc,
3     unsigned int encoding_version, unsigned int event_index) {
4     this->track_id_ = encodeFunc(this->track_id_, encoding_version,
5         event_index);
6     for (int i_contrib{0}; i_contrib < this->getNumberOfContribs();
7         i_contrib++) {
8         this->track_id_contribs_[i_contrib] = encodeFunc(
9             this->track_id_contribs_[i_contrib], encoding_version,
10            event_index);
11         this->incident_id_contribs_[i_contrib] = encodeFunc(
12             this->incident_id_contribs_[i_contrib], encoding_version,
13            event_index);
14     }
15 }

```

Listing 2. Origin ID is no longer encoded in `SimCalorimeterHit::encodeTracks`.

7.2 ROOT reading and tensorization logic

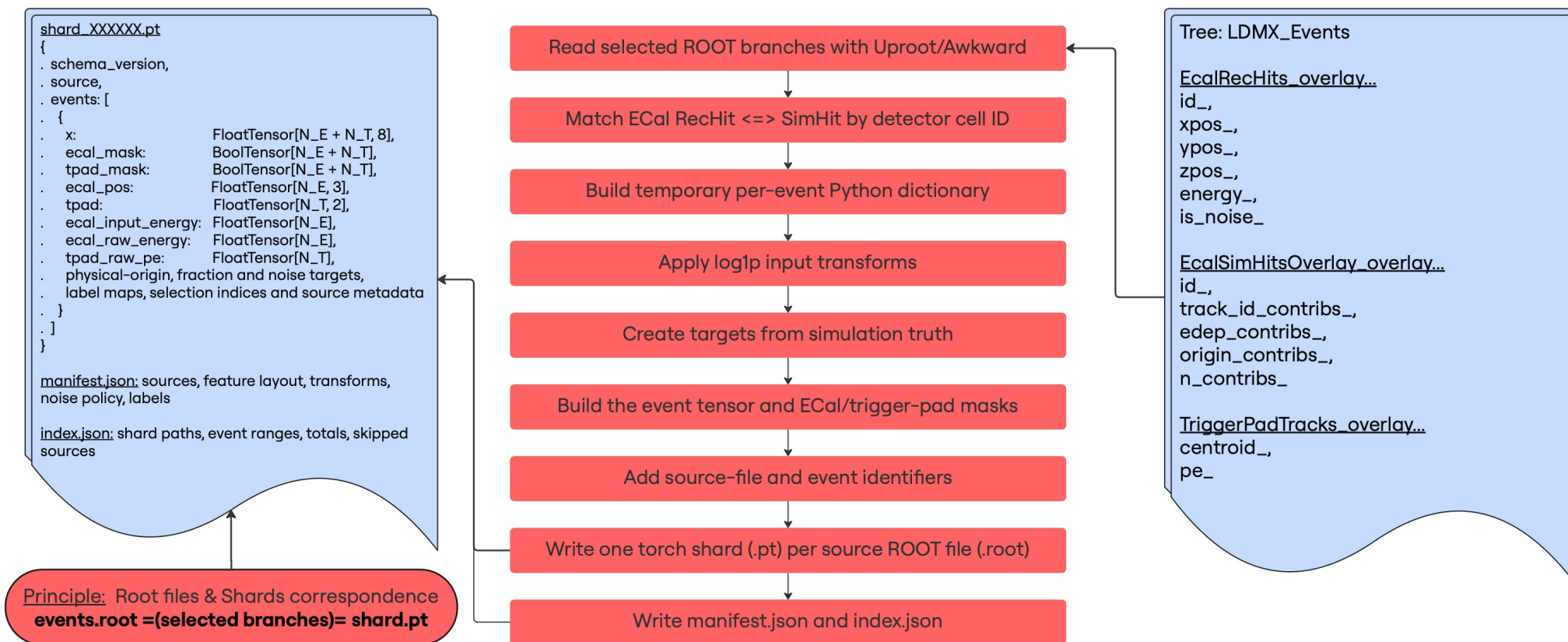


Figure 38. Write caption here...

7.3 Interactive 3D ECal Event Display

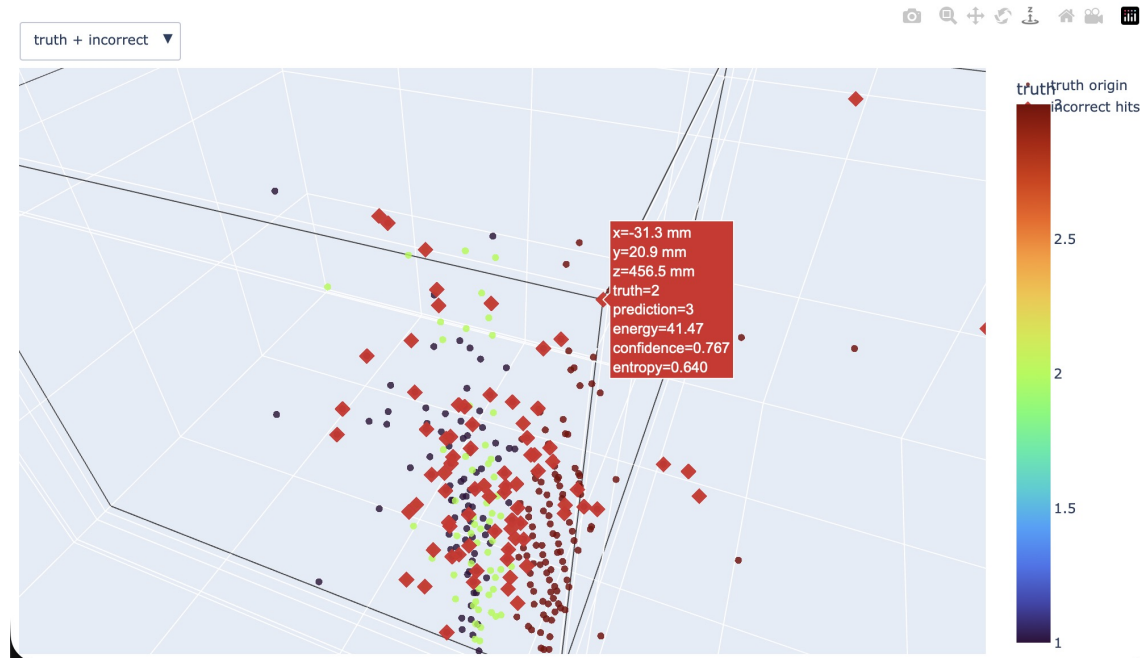


Figure 39. Screenshot of the 3D interactive ECal event display of a 3e event opened in a browser. The truth + incorrect view is selected in the drop down menu and correctly predicted hits have their respective true label coloring and incorrect predictions are marked in red squares. The user hovers the mouse over an incorrectly predicted ECal hit that displays more information about it.